KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Approved by AICET, New Delhi & Affiliated to Anna University, Chennai
Accredited by NBA (CSE, ECE, EEE, Mech), NAAC,
Recognized by UGC with 2(f) &12(B) An ISO 9001:2005 Certified Institution
Namakkal - Trichy Main Road,
Tholurpatti (post), Thottiam, Trichy – 621215, Tamil Nadu

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

COURSE MATERIAL

Course Name with Code : **24CS302 – OBJECT ORIENTED PROGRAMMING USING JAVA**

Course Faculty : **Mrs.S.VIMALA**

Batch : **2024-2028**

Year / Semester / Section : **II /III**

S.NO	TOPICS COVERED	PAGE NO
	UNIT – I- OOPS AND JAVA FUNDAMENTALS	
1.	Programming Language types	**01**
2.	Programming Paradigms	**01**
3.	Object Oriented Programming Concepts	**02**
4.	History of Java	**03**
5.	Java buzzwords	**05**
6.	Data Types	**07**
7.	Variables	**08**
8.	Operators	**09**
9.	Control Statements in Java	**12**
10.	Type Conversion and Casting	**25**
11.	Array	**26**
12.	Introducing Classes: Class Fundamentals	**31**
13.	Declaring Objects	**33**
14.	Methods	**34**
15.	Constructors	**37**
16.	Garbage Collection	**39**
17.	this keyword	**40**

UNIT – II - METHOD OVERLOADING AND INHERITANCE		
18.	Method overloading	**41**
19.	Object as parameters	**46**
20.	Returning Objects	**48**
21.	Recursion	**49**
22.	Access Control	**50**
23.	Static	**53**
24.	Final	**56**
25.	Constructor Overloading	**57**
26.	Inheritance Basis	**60**
27.	Using super	**62**
28.	Single Inheritance	**62**
29.	Multilevel Inheritance	**63**
30.	Hierarchical Inheritance	**65**
31.	The Cosmic Superclass	**68**
32.	Method Overriding	**68**
33.	Abstract Class	**69**
34.	Final with Inheritance.	**71**
UNIT – III - PACKAGES, INTERFACES AND EXCEPTION HANDLING		
35.	Packages — Importing packages	**72**
36.	Interfaces	**76**
37.	Static, Default and Private Methods.	**78**
38.	Exception — Handling Fundamentals	**79**
39.	Exception Types	**81**
40.	Uncaught Exceptions	**83**
41.	Using Try and Catch	**84**
42.	Built-in Exceptions	**85**
43.	User defined exceptions.	**86**
44.	String: String Methods	**87**
45.	Operations	**89**
UNIT – IV- MULTITHREAD PROGRAMMING AND INPUT / OUTPUT STREAMS		
46.	Multithreaded Programming	**91**
47.	Thread Model	**92**
48.	Thread Creation	**93**
49.	Life Cycle	**94**
50.	Thread Priorities	**98**
51.	Synchronization of Threads	**99**
52.	Inter thread Communication	**102**

53.	Suspending, Resuming and Stopping Threads	**104**
54.	I/O Basics	**107**
55.	Streams	**107**
56.	Byte streams and Character streams	**108**
57.	Reading and Writing Console	**109**
58.	The Print Writer Class	**113**
59.	Reading and Writing Files	**114**
60.	Scanner Class.	**117**
UNIT – V-EVENT DRIVEN PROGRAMMING AND NETWORK PROGRAMMING		
61.	Event Driven Programming: Event Listeners	**118**
62.	Delegation event model: Handling mouse	**119**
63.	keyboard events	**120**
64.	Adapter classes	**121**
65.	Swing: Introduction	**122**
66.	MVC Architecture	**124**
67.	Components and containers	**128**
68.	Exploring Swing Components: JLabel and Image icon	**130**
69.	JTextField	**131**
70.	The swing Button	**132**
71.	JScrollPane	**134**
72.	JList and JComboBox	**134**
73.	Handling menus	**136**
74.	Networking Basics	**139**
75.	classes and interfaces	**139**
76.	InetAddress: TCP/IP Client	**140**
77.	Cookies	**142**
78.	TCP/IP Server Sockets	**142**
79.	Datagrams.	**143**

Course Faculty Signature **HOD**

UNIT – I- OOPS AND JAVA FUNDAMENTALS

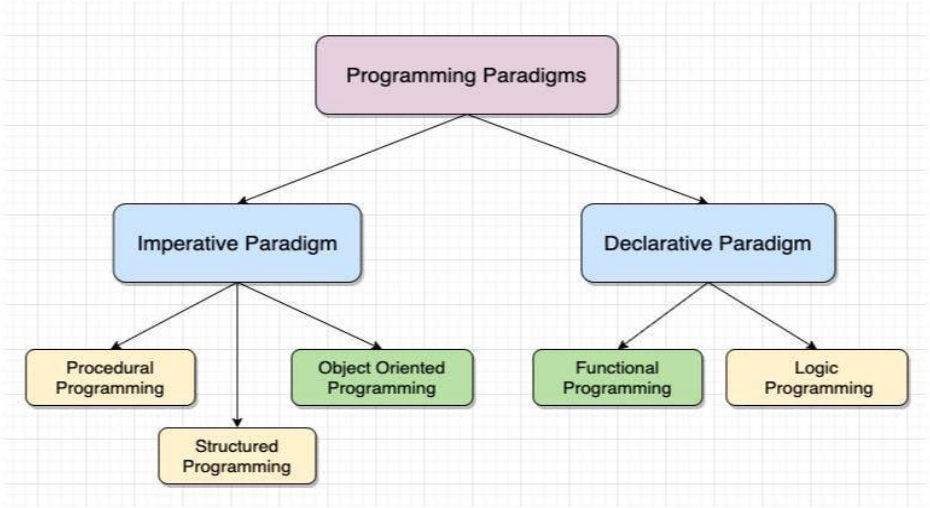
1. Programming Language types

Programming languages can be classified in several overlapping ways. Here's a breakdown of some key classifications:

I. By Level of Abstraction
II. By Level of Abstraction
III. By Execution Method
IV. By Typing System

2. Programming Paradigms

Programming paradigms are a way to classify programming languages based on their features. A language can be classified into multiple paradigms. The Programming Paradigm is mainly classified into two types: Imperative and Declarative.



Imperative Language

In the Imperative type of Language, The programmers instruct the machine How to perform a task by writing step-by-step instructions including variables, conditions, loops etc. The imperative paradigm is further classified into three types: Procedural, Structured and Object Oriented.

Procedural Programming

Procedural language breaks down a task into a collection of procedures (subroutines, functions) to perform a task. It maintains global variables to manage the state of the system.

Example Program: Linear search in C

Structured Programming

Structured programming is a programming paradigm that emphasizes a logical, clear flow of control using well-defined structures like sequence, selection, and iteration, making programs easier to understand, debug, and maintain.

Object Oriented Programming

Object Oriented language breaks down a task into objects that expose behavior (method) and data(member or attribute), objects communicate with each other through message-passing techniques to accomplish a given task

Declarative Language

In the Declarative type of Language, The programmers instruct the machine What to perform without any instructions. The programmer asks for the desired result.

Functional programming

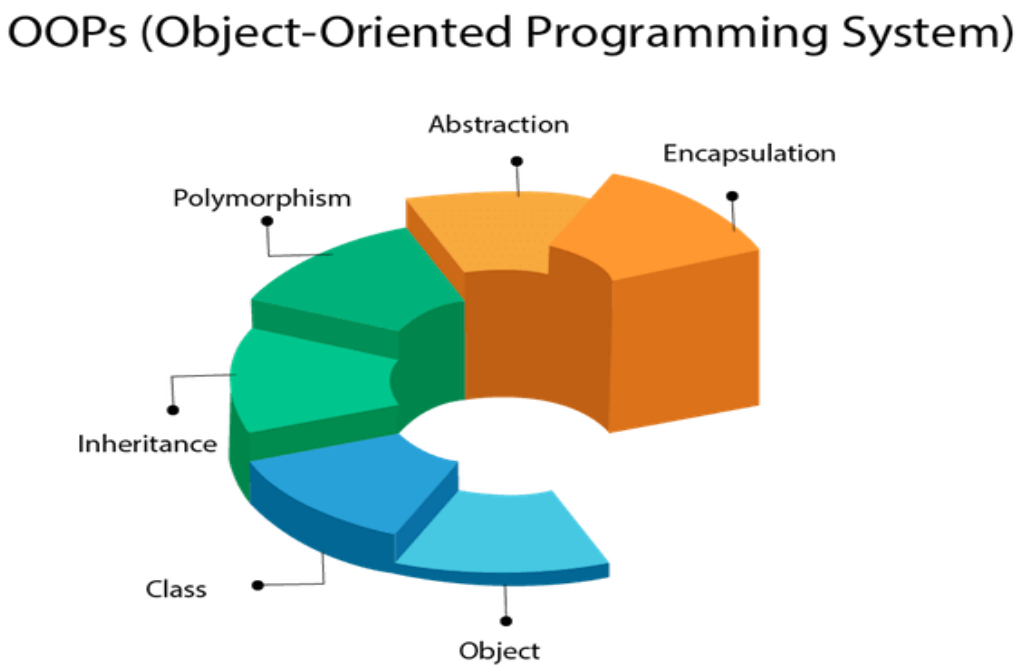
Functional programming contains only functions without mutating the state or data. It means that there are no global data/variables. Composing and applying function is the main driving force in this paradigm. A function can take another as an argument and return a new function.

Logical programming

The Logical paradigm has its foundations in mathematical logic in which program statements express facts and rules. Rules are written as logical clauses. The engine infers the answer to the user's query by using unification and backtracking techniques. The set of facts and rules forms a knowledge base, allowing the system to reason.

3. Object Oriented Programming Concepts

Object-oriented programming (OOP) revolves around the concept of "objects," which combine data (attributes) and actions (methods) that operate on that data. A language is considered object-oriented if it supports key principles like encapsulation, inheritance, abstraction, and polymorphism, along with the fundamental building blocks of objects and classes.



Object

An object is defined as an entity that contains data and its related functions. The functions operate on that data. The object may be either physical or logical.

Class

A class is defined as a collection of objects with same type of data and functions. The functions of the class should be defined. With the help of the class we can create number of objects of type class. All objects occupy equal memory size.

Encapsulation:

Encapsulation is a technique used to protect the information in an object from other objects. In an object data and its functions are encapsulated into a single entity because of other object cannot access directly. This is called data encapsulation or data hiding.

Abstraction:

Abstraction is defined as grouping of essential details ignoring other details. Data abstraction is defined as a named collection of data that describes a object in a class. Class is also called abstract data type because classes use data abstraction.

Inheritance:

Inheritance is the process of deriving new classes from an already existing classes. The derived classes contains all the attributes and functions of existing classes & its own attributes and functions. Existing classes called “base classes” & the inherited classes are called “derived classes”.

Polymorphism:

Polymorphism means “many forms.” It allows objects of different classes to be treated as objects of a common type.

4. **History of Java**

James Gosling is considered the original developer of Java language. He is also known as father of Java. He started on this in early 1990s. He originally named it Greentalk and the extension was .gt. Programming language renamed from Greentalk to Oak: When Green team (James Gosling, Mike Sheridan and Patrick Naughton) started working on it, it was renamed to Oak. Oak is a tree name, which is considered a symbol of strength and is a national tree in many countries such as U.S.A., France, Germany, Romania, etc Again renamed from Oak to Java in 1995:The project is again renamed from Oak to Java in 1995 as Oak was a trademark for other organization. Original and first version of Java: JDK first version alpha & beta was released in 1995 by Sun Microsystems.

Version	Date	Description
JDK Beta	1995	Initial Draft version
JDK 1.0	23 Jan 1996	A stable variant JDK 1.0.2 was termed as JDK 1
JDK 1.1	19 Feb 1997	Major features like JavaBeans, RMI, JDBC, inner classes were added in this release.
JDK 1.2	8 Dec 1998	Swing, JIT Compiler, Java Modules, Collections were introduced to JAVA and this release was a great success.
JDK 1.3	8 May 2000	HotSpot JVM, JNDI, JPDA, JavaSound and support for Synthetic proxy classes were added.
JDK 1.4	6 Feb 2002	Image I/O API to create/read JPEG/PNG image were added. Integrated XML parser and XSLT processor (JAXP) and Preferences API were other important updates.
JDK 1.5 or J2SE 5	30 Sep 2004	Various new features were added to the language like foreach, var-args, generics etc.
JAVA SE 6	11 Dec 2006	1. notation was dropped to SE and upgrades done to JAXB 2.0, JSR 269 support and JDBC 4.0 support added.
JAVA SE 7	7 Jul 2011	Support for dynamic languages added to JVM. Another enhancements included string in switch case, compressed 64 bit pointers etc.
JAVA SE 8	18 Mar 2014	Support for functional programming added. Lambda expressions, streams, default methods, new date-time APIs introduced.
JAVA SE 9	21 Sep 2017	Module system introduced which can be applied to JVM platform.
JAVA SE 10	20 Mar 2018	Unicode language-tag extensions added. Root certificates, threadlocal handshakes, support for heap allocation on alternate memory devices etc were introduced.
JAVA SE 11	5 Sep 2018	Dynamic class-file constants, Epsilon a no-op garbage collector, local-variable support in lambda parameters, Low-overhead heap profiling support added.
JAVA SE 12	19 Mar 2019	Experimental Garbage Collector,Shenandoah: A Low-Pause-Time Garbage Collector, Microbenchmark Suite, JVM Constants API added.

JAVA SE 13	17 Sep 2019	Feature added - Text Blocks (Multiline strings), Enhanced Thread-local handshakes.
JAVA SE 14	17 Mar 2020	Feature added - Records, a new class type for modelling, Pattern Matching for instanceof, Intuitive NullPointerException handling.
JAVA SE 15	15 Sep 2020	Feature added - Sealed Classes, Hidden Classes, Foreign Function and Memory API (Incubator).
JAVA SE 16	16 Mar 2021	Feature added as preview - Records, Pattern Matching for switch, Unix Domain Socket Channel (Incubator) etc.
JAVA SE 17	14 Sep 2021	Feature added as finalized - Sealed Classes, Pattern Matching for instanceof, Strong encapsulation of JDK internals by default. New macOS rendering pipeline etc.
JAVA SE 18	22 Mar 2022	Feature added - UTF-8 by Default, Code Snippets in Java API Documentation, Vector API (Third incubator), Foreign Function, Memory API (Second Incubator) etc.
JAVA SE 19	20 Sep 2022	Feature added - Record pattern, Vector API (Fourth incubator), Structured Concurrency (Incubator) etc.
JAVA SE 20	21 Mar 2023	Feature added - Scoped Values (Incubator), Record Patterns (Second Preview), Pattern Matching for switch (Fourth Preview),Foreign Function & Memory API (Second Preview) etc.
JAVA SE 21	19 Sep 2023	Feature added - String Templates (Preview), Sequenced Collections, Generational ZGC, Record Patterns, Pattern Matching for switch etc.
Java SE 22	19 Mar 2024	Feature added - Region Pinning for G1 garbage collector, foreign functions and memory APIs , multi-file source code programs support, string templates, vector apis (seventh incubator), unnamed variables, patterns, stream gatherers (first preview) etc.
Java SE 23	17 Sep 2024	Feature added - Primitive types in patterns, class file APIs, vector APIs (Eighth incubator), stream gatherers (second preview), ZDC, generation mode by default etc.

5. Java buzzwords

The main objective of Java programming language creation was to make it portable, simple and secure programming language. Java has many buzzwords that are often used

to describe key concepts, features, and philosophies of the language. Here are some of the most common ones:

Simple	Architecture Neutral
Object Oriented	Portable
Platform Independent	Interpreted
Distributed	High Performance
Robust	Multithreaded
Secure	Dynamic

Object-oriented:

Java is a language built upon the object-oriented programming paradigm, using objects to structure and organize code.

Platform-independent:

Java code can run on any device or operating system with a Java Virtual Machine (JVM) installed, thanks to its bytecode format.

Robust:

Java is designed to be robust, with features like automatic memory management and exception handling to ensure stable execution.

Secure:

Java offers security features like bytecode verification and a security manager, making it suitable for applications requiring security.

Multithreaded:

Java supports creating and managing multiple threads of execution, allowing for parallel processing.

Dynamic:

Java is a dynamic language, allowing for changes and updates to be made during runtime, according to WordPress.com.

Architecture-neutral:

Java code is not dependent on the specific hardware architecture of the machine it runs on, making it portable across different platforms.

Portable:

Java's platform independence allows for its code to be easily moved and run on different systems.

Interpreted:

Java code is first compiled into bytecode, and then interpreted by the JVM during execution.

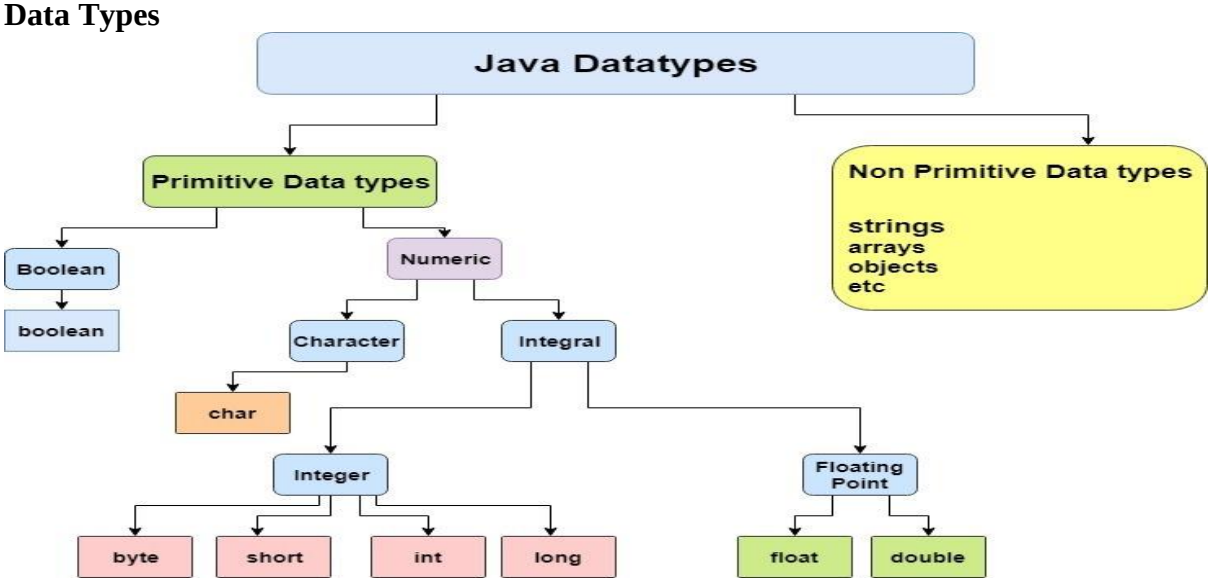
High performance:

Java's Just-In-Time (JIT) compiler and efficient memory management contribute to its high performance capabilities.

Distributed:

Java is suitable for developing distributed applications that can be spread across multiple machines.

6. **Data Types**



Java is a strongly typed programming language, meaning every variable must have a defined data type. Data types specify the size, type, and nature of values that can be stored in variables. In Java, data types are broadly categorized into primitive data types and non-primitive (reference) data types.

1**. Primitive Data Types**

Primitive data types are the fundamental building blocks of data manipulation in Java. These types are predefined by the language and store actual values in memory rather than references. Java provides eight primitive data types, which are classified into different categories based on the type of values they store.

a) **Integer Types**

These types store whole numbers (both positive and negative) and differ based on their memory size and value range:

byte (8-bit): Stores values from -128 to 127. This is typically used when memory conservation is essential, such as for arrays of large numbers.
short (16-bit): Stores values from -32,768 to 32,767. Suitable for applications where memory is a constraint but larger values than byte are required.
int (32-bit): Stores values from -2,147,483,648 to 2,147,483,647. This is the default integer type used in Java programs.
long (64-bit): Stores values from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.
Used when dealing with large numerical values, often requiring an L suffix (e.g., 100L).

b) Floating-Point Types

These types store decimal (floating-point) numbers and are useful when dealing with computations that require precision:
float (32-bit): Stores fractional values with a precision of approximately 6-7 decimal digits. Needs an F suffix (e.g., 3.14F).
double (64-bit): Stores fractional values with a higher precision of up to 15-16 decimal digits. Used as the default type for decimal numbers.

c) Character Type

char (16-bit): Stores a single character using Unicode encoding. It is enclosed in single quotes (e.g., 'A'). Unicode support allows for storing a wide range of characters from different languages.

d) Boolean Type

boolean (1-bit): Stores only two values: true or false. It is used for conditional logic and decision- making in control structures like if-else and loops.

2. Non-Primitive (Reference) Data Types

Non-primitive data types, also known as reference types, store memory references rather than actual data. These include:
a) Strings: A sequence of characters enclosed in double quotes (e.g., "Hello"). Strings are immutable in Java, meaning their values cannot be changed once created.
b) Arrays: A collection of elements of the same data type stored in contiguous memory locations. Arrays in Java are zero-indexed, meaning the first element is accessed at index 0.
c) Classes and Objects: A user-defined blueprint that defines the properties and behaviors of objects. Objects are instances of a class that store state (attributes) and behavior (methods).
d) Interfaces: A contract that defines abstract methods, which implementing classes must define. Interfaces facilitate multiple inheritance and abstraction in Java

7. VARIABLES

A variable is a container which holds the value and that can be changed during the execution of the program. A variable is assigned with a datatype. Variable is a name of memory location.
All the variables must be declared before they can be used. There are three types of variables in java:

local variable,
instance variable and
static variable.

Local variable - A variable defined within a block or method or constructor is called local variable.

Instance Variable - Instance variables are non-static variables and are declared in a class outside any method, constructor or block.

Static variable - Static variables are also known as Class variables. A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class.

```
public class A
{
   static int m=100;//static variable
   void method()
   {
      int n=90;//local variable
   }
   public static void main(String args[])
   {
      int data=50;//instance variable
   }
}//end of class
```

8. Operators

Java provides a rich set of operators to manipulate variables. We can divide all the Java operators into the following groups –

a. Unary Operator
b. Arithmetic Operator
c. Relational Operator
d. Logical Operator
e. Assignment Operator
f. Bitwise Operator
g. Shift Operator and
h. Ternary Operator

a. Unary Operator

The Java unary operators require only one operand. Unary operators are used to perform various operations i.e.: incrementing/decrementing a value by one negating an expression inverting the value of a Boolean

Eg:

```
public class OperatorExample{
public static void main(String args[]){
int x=10;
System.out.println(x++);//10 (11)
System.out.println(++x);//12
System.out.println(x--);//12 (11)
System.out.println(--x);//10
}}
```

b. Arithmetic Operator

Java arithmetic operators are used to perform addition, subtraction, multiplication, and division. They act as basic mathematical operations.

Eg.

```
public class OperatorExample{
public static void main(String args[]){
int a=10;
int b=5;
System.out.println(a+b);//15
System.out.println(a-b);//5
System.out.println(a*b);//50
System.out.println(a/b);//2
System.out.println(a%b);//0
}}
```

c. Relational (Comparison) Operators

Relational operators compare two values and return a boolean result.

== , Equal to.
!= , Not equal to.
< , Less than.
<= , Less than or equal to.
> , Greater than.
>= , Greater than or equal to.

Example:

```
int x = 10, y = 20;
System.out.println(x > y); // false
```

d. Logical Operators

Logical Operators are used to perform "logical AND" and "logical OR" operations, similar to AND gate and OR gate in digital electronics. They have a short-circuiting effect, meaning the second condition is not evaluated if the first is false.

Conditional operators are:
&&, Logical AND: returns true when both conditions are true.
||, Logical OR: returns true if at least one condition is true.
!, Logical NOT: returns true when a condition is false and vice-versa

Example:

```
boolean result = (1 > 3) && (8 > 5); // true
System.out.println (result);
```

e. Assignment Operator

'=' Assignment operator is used to assign a value to any variable. It has right-to-left associativity, i.e. value given on the right-hand side of the operator is assigned to the variable on the left, and therefore right-hand side value must be declared before using it or should be a constant.

The general format of the assignment operator is:

variable = value;

Common compound operators include:

+= , Add and assign.
-= , Subtract and assign.
*= , Multiply and assign.
/= , Divide and assign.
%= , Modulo and assign.

Example:

```
class assignoper {
   // Main Function
   public static void main(String[] args)
   {

      // Assignment operators
      int f = 7;
      System.out.println("f += 3: " + (f += 3)); //10
      System.out.println("f -= 2: " + (f -= 2)); //8
      System.out.println("f *= 4: " + (f *= 4)); //32
      System.out.println("f /= 3: " + (f /= 3)); //10
      System.out.println("f %= 2: " + (f %= 2)); //0
   }
```

f. Bitwise Operators

Bitwise Operators are used to perform the manipulation of individual bits of a number and with any of the integer types.

Types

& (Bitwise AND): returns bit-by-bit AND of input values.
| (Bitwise OR): returns bit-by-bit OR of input values.
^ (Bitwise XOR): returns bit-by-bit XOR of input values.
~ (Bitwise Complement): inverts all bits (one's complement).

A	B	A | B	A & B	A ^ B	~A
0	0	0	0	0	1
1	0	1	0	1	0
0	1	1	0	1	1
1	1	1	1	0	0

g. Shift Operation

Shift operators in Java are used to perform bitwise operations, manipulating the binary representation of numbers. They shift the bits of an operand to the left or right, effectively

Operator	Description	Example
<<	Left shift	a << 2
>>	Right shift	a >> 2
>>>	Unsigned right shift	a >>> 2

h. Ternary operator

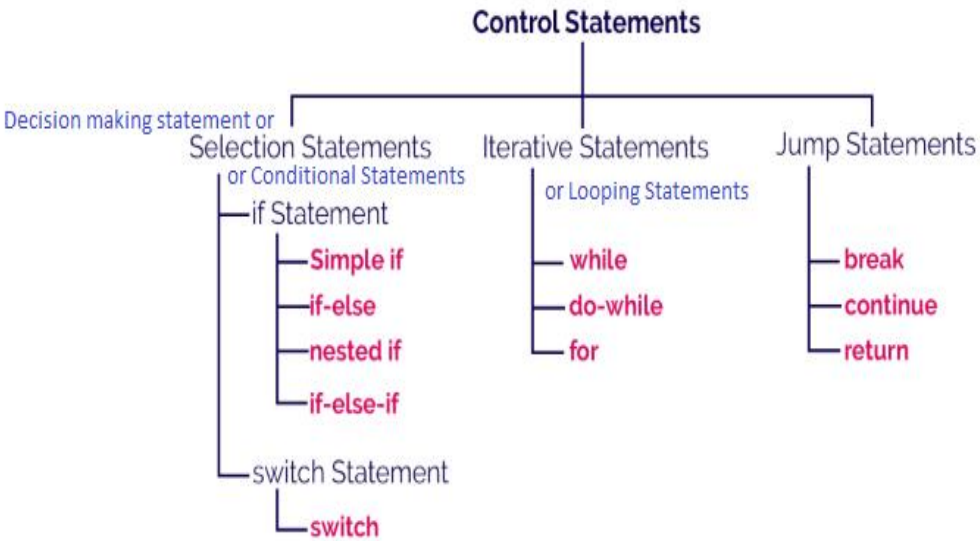
Java Ternary operator is used as one line replacement for if-then-else statement and used a lot in Java programming. It is the only conditional operator which takes three operands**.**

Ternary operator in java is used as one liner replacement for if-then-else statement and used a lot in java programming. it is the only conditional operator which takes three operands.

```
public class OperatorExample{

public static void main(String args[]){

int a=2;

int b=5;

int min=(a<b)?a:b;

System.out.println(min);

}}
```

9. Control Statements in Java

Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java, which provides a smooth flow of program.

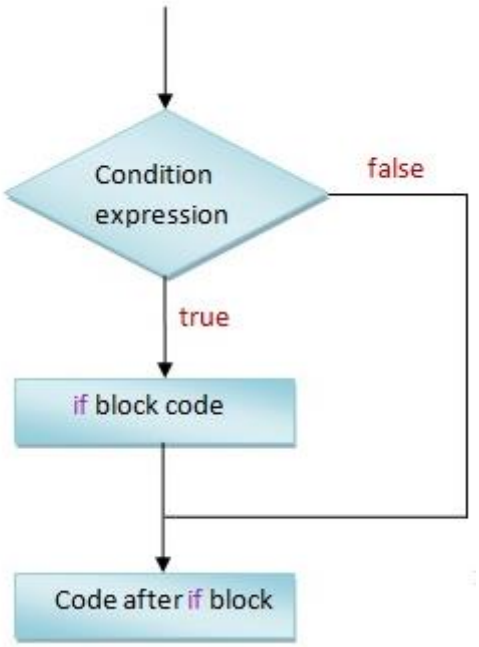


if Statement

if statement is the most simple decision making statement. It is used to decide whether a certain statement or block of statements will be executed or not that is if a certain condition is true then a block of statement is executed otherwise not

Syntax:

```
if(condition)
{
//statements to execute if
//condition is true
}
```



Example:

```
class IfStatement {
  public static void main(String[] args) {
    int number = 10;
if (number < 0) {
      System.out.println("The number is negative.");
    }
    System.out.println("Statement outside if block");
  }
}
```

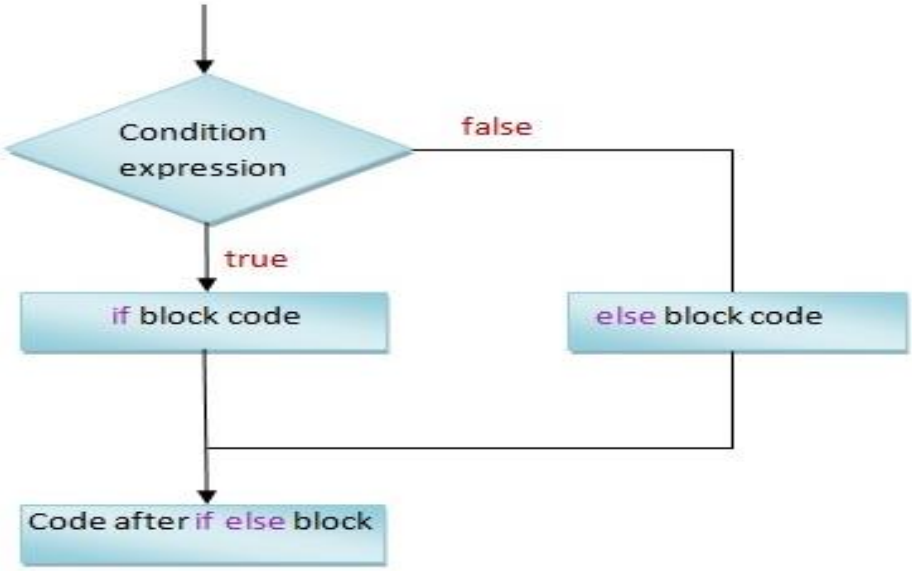
if-else Statement

The Java if-else statement also tests the condition. It executes the if block if condition is true else if it is false the else block is executed

Syntax:.

```
if(condition)
{
TRUE STATEMENT;
}
else
{
```

FALSE STATEMENT;
}



Example:

```
class Main {
  public static void main(String[] args) {
    int number = 10;
    if (number > 0) {
      System.out.println("The number is positive.");
    }
else {
      System.out.println("The number is not positive.");
    }
    System.out.println("Statement outside if...else block");
  }
}
```

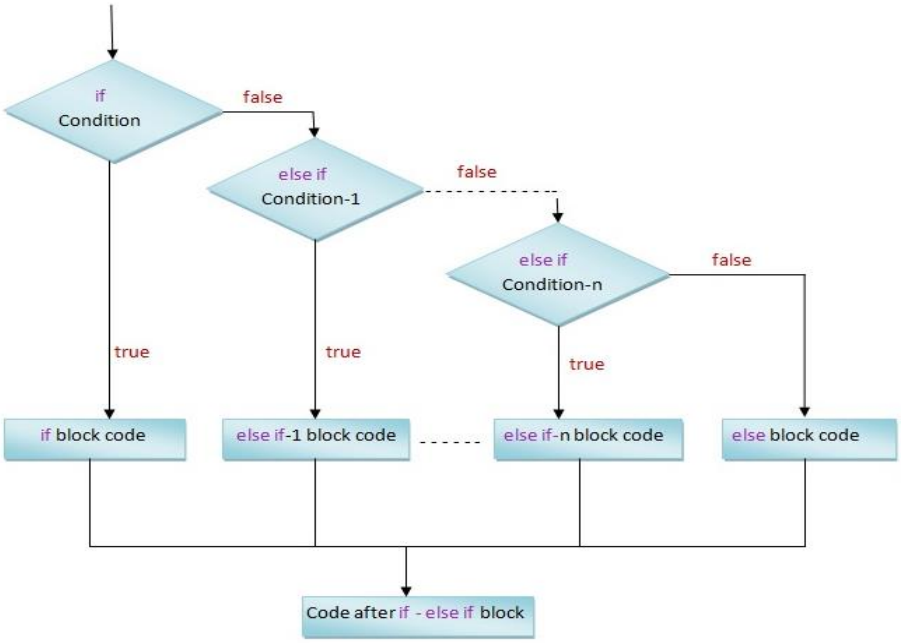
if-else-if ladder:

The if-else-if statement contains the if-statement followed by multiple else-if statements. In other words, we can say that it is the chain of if-else statements that create a decision tree where the program may enter in the block of code where the condition is true.

Syntax;

```
if(condition 1) {

statement 1; //executes when condition 1 is true

}

else if(condition 2) {

statement 2; //executes when condition 2 is true

}

else {
```

```
statement 2; //executes when all the conditions are false
}
```



Example:

```
public class Student
{
public static void main(String[] args)
{
String city = "Delhi";
if(city == "Meerut")
{
System.out.println("city is meerut");
}
else if (city == "Noida")
{
System.out.println("city is noida");
}
else if(city == "Agra")
{
```

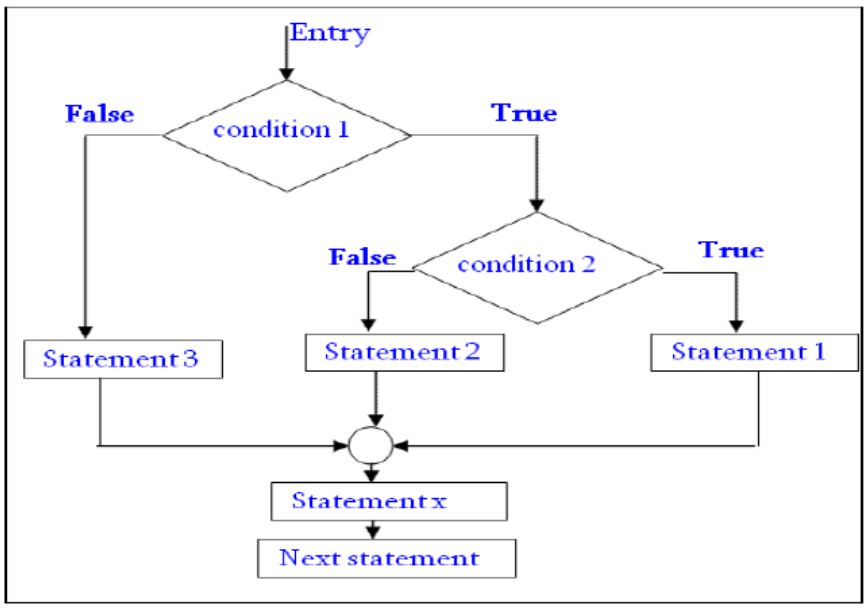
```
System.out.println("city is agra");
}
else
{
System.out.println(city);
}
}
}
```

Nested if Statement

Nested if-else statements, is that using one if or else if statement inside another if or else if statement(s).

Syntax:

```
if(condition 1)
{
statement 1; //executes when condition 1 is true
if(condition 2)
{
statement 2; //executes when condition 2 is true
}
else
{
statement 2; //executes when condition 2 is false
}
}
```



Example:

```
class NestedIfDemo
{
   public static void main(String args[])
   {
      int i = 10;
      if (i == 10)
      {
         // First if statement
         if (i < 15)
            System.out.println("i is smaller than 15");
           // Nested - if statement
         // Will only be executed if statement above
         // it is true
         if (i < 12)
            System.out.println("i is smaller than 12 too");
         else
            System.out.println("i is greater than 15");
      }
   }
}
```

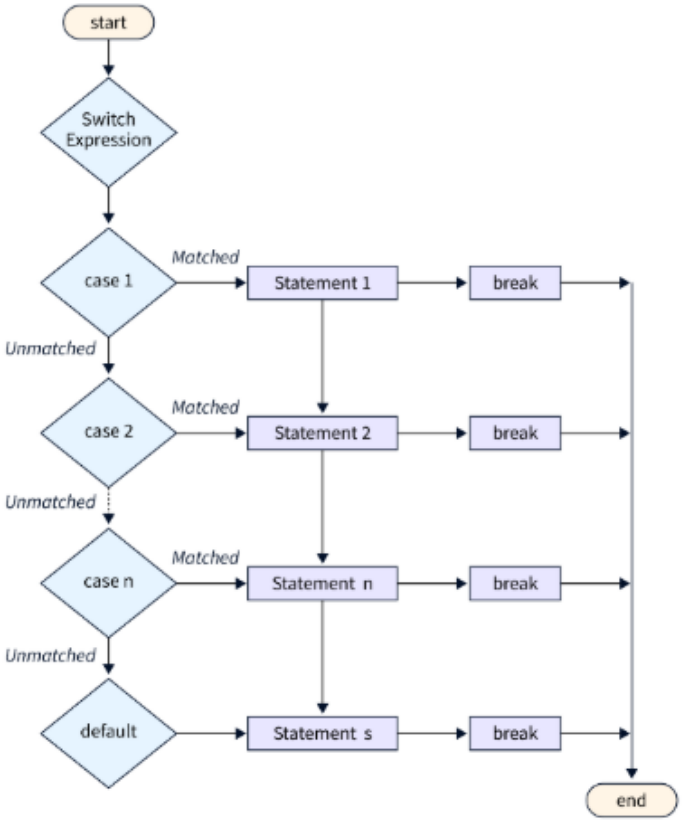
Switch Statements

The **switch** statement is Java's multiway branch statement. It provides an easy way to dispatch execution to different parts of your code based on the value of an expression

Syntax:

```
switch (expression)
        {
        case value1:
        // statement sequence break;
```

```
case value2:
// statement sequence break;
.
.
case valueN :
// statement sequence break;
default:
// default statement sequence
}
```



Example:

```
public class Main {
  public static void main(String[] args) {
    int day = 4;
    switch (day) {
      case 1:
```

```
    System.out.println("Monday");
    break;
  case 2:
    System.out.println("Tuesday");
    break;
  case 3:
  System.out.println("Wednesday");
    break;
  case 4:
    System.out.println("Thursday");
    break;
  case 5:
    System.out.println("Friday");
    break;
  case 6:
    System.out.println("Saturday");
    break;
  case 7:
    System.out.println("Sunday");
    break;
 }  } }
```

Iterative Statements

In programming languages, loops are used to execute a set of instructions/functions repeatedly when some conditions become true.

The looping process involves:

1. Evaluating a condition
2. Executing the code block if the condition is true
3. Repeating the evaluation and execution until the condition becomes false

There are three types of loops in java.

- while loop

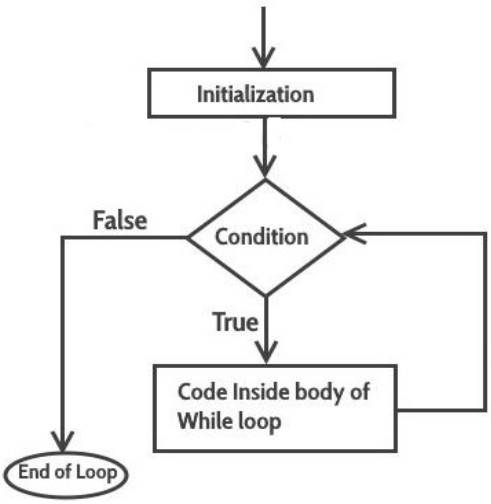
- do-while loop
- For loop

While loop:

A while loop is a control flow statement that allows code to be executed repeatedly based on a given Boolean condition. The while loop can be thought of as a repeating if statement.

Syntax:

```
while (condition)
 {
// body of loop
}
```



Example:

```
class WhileLoopExample
{
   public static void main(String args[])
{
        int i=10;
        while(i>1)
        {
            System.out.println(i);
            i--;
```

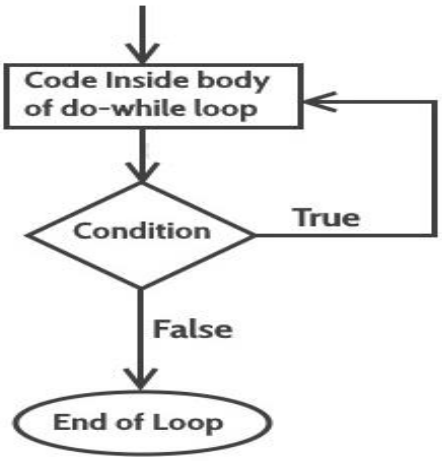
```
    }   } }
```

do – while loop :

do while loop checks for condition after executing the statements, and therefore it is called as Exit Controlled Loop**.**

Syntax:

```
do
{
// body of loop
} while (condition);
```



:

Example:

```
public class GFG
{

   public static void main(String[] args)
 {

      int c = 1;

     // Using do-while loop
      do
     {
```

```
        System.out.println("Do-While Loop: " + c);
        c++;
      } while (c <= 5);
   }
}
```

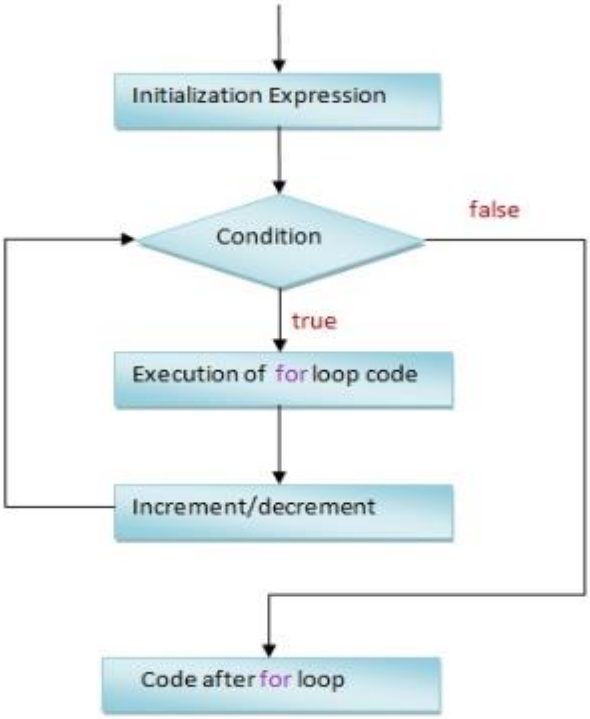
For loop :

Java for loop can be used to repeat execution of statements for a fixed number of times or till a condition is true. The basic syntax of for loop is :

Syntax:

```
for(initialization; condition; increment/decrement)
  {
   // code to be executed inside for loop
  }
```

Everything given inside () after for loop are optional expressions which performs specific tasks. Initialization expression is used to initialize variables, objects etc. for example int i = 0, int count=10 etc. Condition expression is used to test the condition, for example i<=10, count>0 etc. Increment/decrement expression is used to change in variable or object value, for example i++, count-- etc. Everything that comes inside matching curly brackets {} after for keyword is the part of for statement which is also called as for block or for loop body.



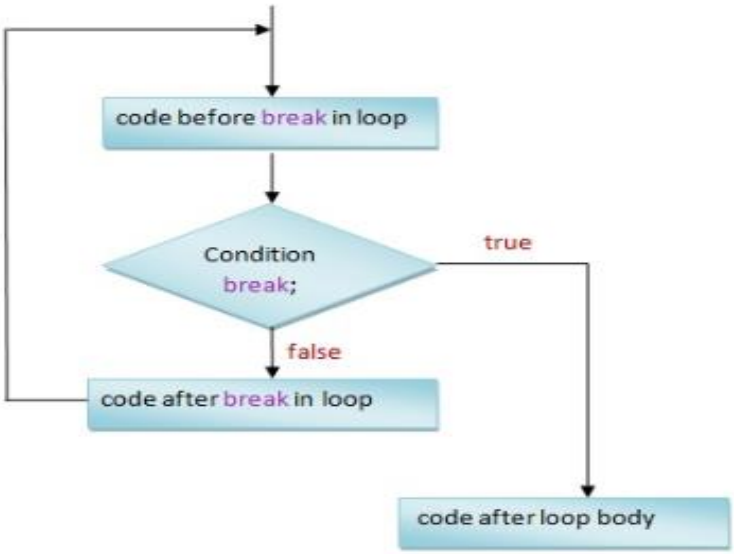
How for loop works

1. Initialization expression is the first expression which get's executed as the loop begins and it is executed only once.
2. Then condition expression is evaluated, it's a boolean expression which returns either true or false.
3. If the condition expression returns as true,
    a. Statements inside the body of for loop is executed.
    b. Then the increment/decrement expression is executed.
    c. Again, the condition expression is evaluated. If it returns as true, again the statements inside the body of for loop is executed and then increment/decrement expression is executed. This process goes on until the condition expression returns as false.
4. If the condition expression returns as false, for loop terminates.Example:

```
class ForLoop
  {
   public static void main(String [] args)
     {
       for(int num=1; num<=10; num++)
        {
         System.out.println(num);
        }
        System.out.println("After the end of for loop");
      }
  }
```

Break statement

Once the execution of a loop statement in a program starts, it keeps repeating the execution of loop instructions as far as a given condition is true. Sometimes it may be required to break the execution flow in between. To break the execution flow of a loop, java provides a keyword called as break.



Example:

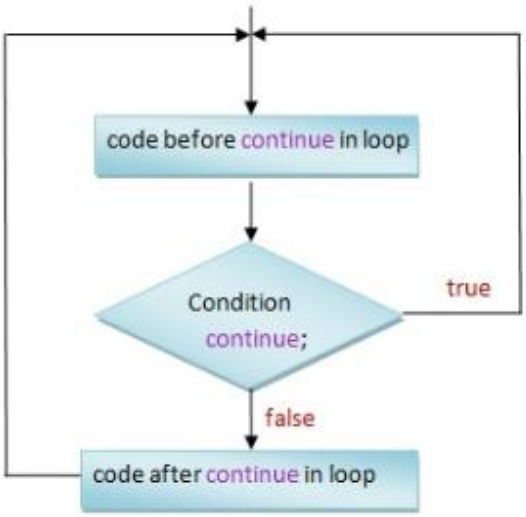
```
class BreakStatement
  {
    public static void main(String [] args)
      {
        for(int i=1; i<=5; i++)
          {
            if(i==4)
              {
                break;
              }
            System.out.println("i = "+i);
          }
        System.out.println("line after for loop");
      }
  }
```

Continue statement

Sometime you may require that for a particular repetition of a loop, you don't want to execute few or all instructions of loop body. To solve such requirements java provides continue keyword which allows programmers to skip the current iteration of a loop and move to the next iteration, which means if you want to skip the execution of code inside a loop for a particular iteration, you can use continue statement. Only the code written after the continue statement will be skipped, code before the continue statement will execute as usual for that iteration

The syntax of continue statement is:

```
continue;
```



.

Example:

```
class ContinueStatement
  {
    public static void main(String [] args)
      {
        for(int i=1; i<=5; i++)
          {
            if(i==4)
              {
                continue;
              }
            System.out.println("i = "+i);
          }
        System.out.println("line after for loop");
      }   }
```

return Statement

As the return keyword itself suggest that it is used to return something. Java return statement is used to return some value from a method. Once a return statement is executed, the control flow exited from current method and returns to the calling method.

The syntax of return statement is:

```
return expression;

return;
```

Example:

```
class ReturnStatement
  {
    public static void main(String [] args)
      {
        int sum = sum(10,20);
        System.out.println("Sum = "+sum);
      }
    public static int sum(int a, int b)
      {
        return a+b;
      }
  }
```

10. Type Conversion and Casting

Java allows conversion between compatible data types through type conversion and type casting.

a) Implicit Type Conversion (Widening Casting)

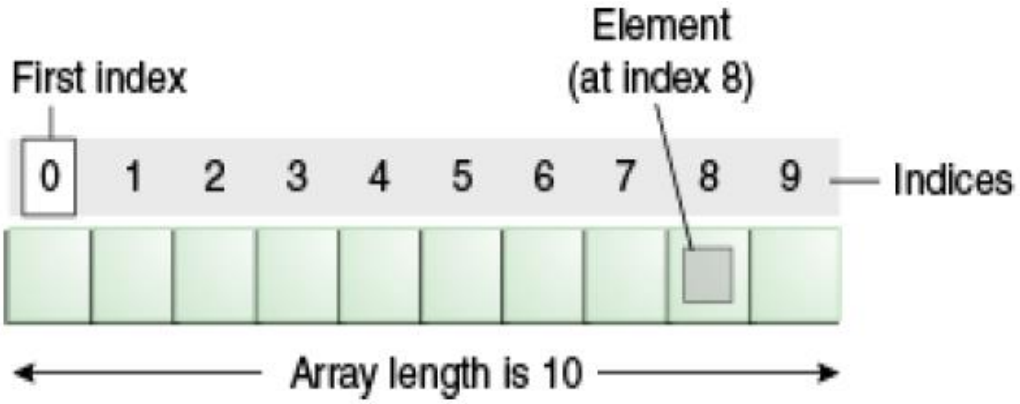
Occurs when a smaller data type is converted into a larger data type automatically without data loss.
Example:
int num = 10;
double d = num; // int to double conversion
Here, the integer num is automatically converted to a double.

b) Explicit Type Casting (Narrowing Casting)

Occurs when a larger data type is explicitly converted into a smaller one, which may lead to data loss.
Example:
double d = 10.5;
int num = (int) d; // double to int conversion (fractional part lost)

11. Array

Array is a collection of similar type of elements that have contiguous memory location.Since arrays are objects in Java, we can find their length using member length. A Java array variable can also be declared like other variables with [] after the data type. The size of an array must be specified by an int value and not long or short.



Advantage of Java Array

a. Code Optimization: It makes the code optimized, we can retrieve or sort the data easily.
b. Random access: We can get any data located at any index position.

Disadvantage of Java Array

Size Limit: We can store only fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in java

Types of Array in java

1. One- Dimensional Array
2. Multidimensional Array

One – Dimensional Array

initialize array

Java arrays initializes array values in a continuous memory location where each memory location is given an index. You can assign or access the value to that memory location using it's index. The index starts from 0 and goes as 0,1,2,3,4...

Example

```
int[] marks = new int[5];
```

Assigning values to array elements

```
 marks[0] = 40;  // initialize first element

 marks[1] = 60;  // initialize second element

 marks[2] = 80;  // initialize third element

 marks[3] = 65;  // initialize fourth element

 marks[4] = 70;  // initialize fifth or last element


 // initializing in one line

 int[] marks = new int[]{40,60,80,65,70};
```

access values of array

Each item in an array is called an element, and each element of array is accessed by its index. The index of first element is 0.

```
marks[0]  // To access element 40

 marks[1]  // To access element 60

 marks[2]  // To access element 80

 marks[3]  // To access element 65

 marks[4]  // To access element 70
```

Example

```
 class ArrayInJava

   {

     public static void main(String [] args)

       {

          int[] marks = new int[5];

          marks[0] = 40;
```

```
        marks[1] = 60;
        marks[2] = 80;
        marks[3] = 65;
        marks[4] = 70;

        System.out.println("Element at index 0 = "+marks[0]);
        System.out.println("Element at index 1 = "+marks[1]);
        System.out.println("Element at index 2 = "+marks[2]);
        System.out.println("Element at index 3 = "+marks[3]);
        System.out.println("Element at index 4 = "+marks[4]);

        // Accessing array elements using for loop;
        System.out.println("Accessing array element using for loop .....");
        for(int i=0; i<5; i++)
        {
          System.out.println("Element at index "+i+" = "+marks[i]);
        }
      }
  }
```

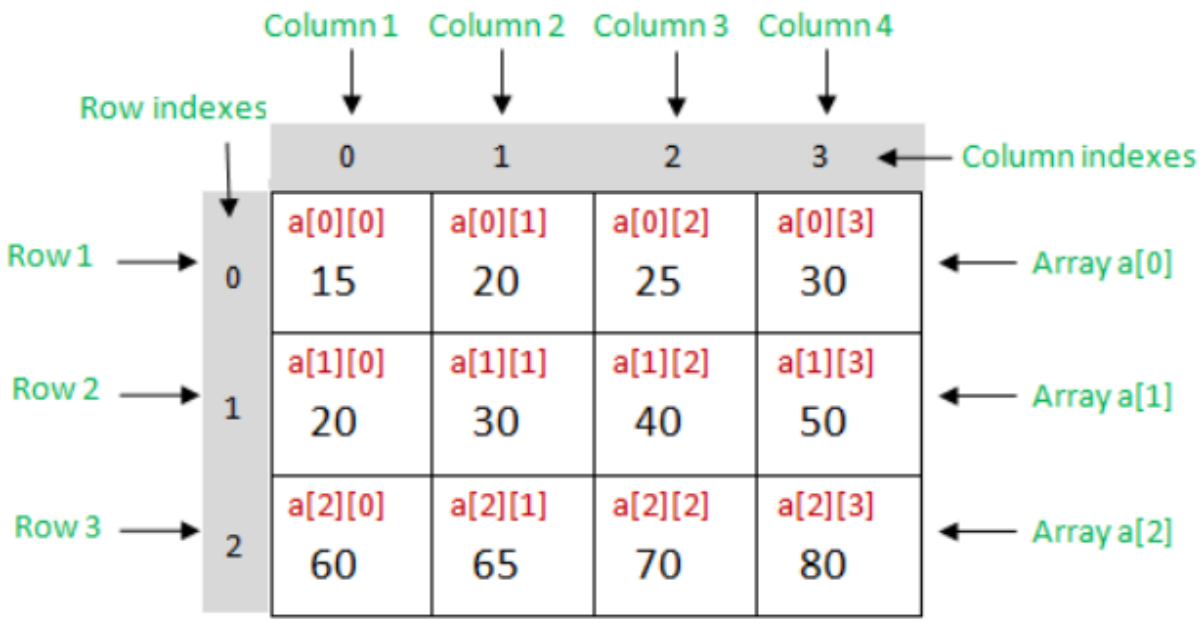
Multidimensional Arrays

In java it is possible to define an array whose elements are itself an array. Such arrays are called multidimensional array. A multidimensional array is an array of arrays which simply means the elements of such arrays will itself be an array.

Multidimensional array could be of different types like two dimensional(2D), three dimensional(3D), four dimensional(4D) and so on. In this tutorial we will cover only two and three dimensional array. After the tutorial you will have enough idea to try other dimensional array as well if you want.

Two Dimensional or 2D Arrays

A 2D array has two dimension which is also called as row and column of 2D array. This type of array has two indexes row index and column index. You can access or assign value in 2D array using the combination of row and column indexes. Row and column indexes starts from 0. 2D array is similar to matrixes in mathematics where we have rows and columns.



declare 2D arrays

syntax of declaring 2D array is :

DataType[][] arrayName;

Example:

int[][] intArray; // declares a 2D array of integer with name as intArray

double[][] doubleArray; // declares a 2D array of double with name as doubleArray

char[][] charArray; // declares a 2D array of char with name as charArray

The double brackets [][] indicates that it's a 2D array. You can use this bracket after the data type or variable name as given in the previous tutorial. The data type can be primitive or non primitive while the name of of array is given as per the programmer's choice.

initialize 2D arrays

```
// First approach

int[][] matrix =  new int[2][3];


matrix[0][0] = 10;

matrix[0][1] = 20;

matrix[0][2] = 30;

matrix[1][0] = 15;

matrix[1][1] = 80;
```

```
 matrix[1][2] = 50;
// 2nd approach
        int[ ][ ] a = {{15,20,25,30},{20,30,40,50},{60,65,70,80}};
```

access values in 2D arrays

To access values in 2D arrays you need to use it's row and column indexes like a[0][0], a[0][1], a[0][2] and so on. In above example a[0][0] will return 15, a[0][1] will return 20 and a[0][2] will return 25.

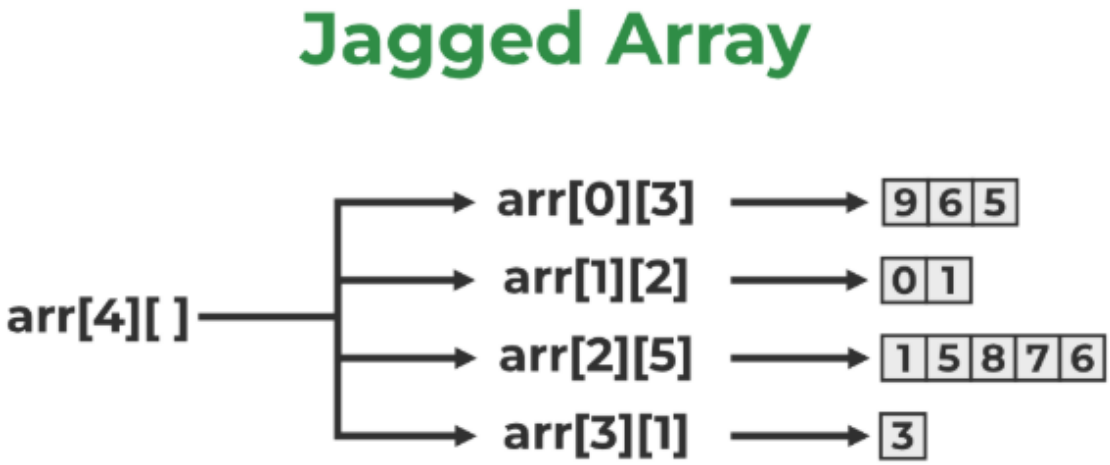
Example:

```
class TwoDArray
  {
    public static void main(String [] args)
      {
        int[][] matrix = new int[2][2];
            matrix[0][0] = 10;
            matrix[0][1] = 20;
            matrix[1][0] = 30;
            matrix[1][1] = 40;


        System.out.println("Element at index 00 = "+matrix[0][0]);
        System.out.println("Element at index 01 = "+matrix[0][1]);
        System.out.println("Element at index 10 = "+matrix[1][0]);
        System.out.println("Element at index 11 = "+matrix[1][1]);
        int[][] a = {{15,20,25},{20,30,40},{50,60,70}};
        System.out.println("\nElement at index 00 = "+a[0][0]);
        System.out.println("Element at index 01 = "+a[0][1]);
        // Accessing array elements using for loop;
        System.out.println("\nAccessing array element using for loop .....");
        for(int i=0; i < a.length; i++)
          {
            for(int j=0; j < a[i].length; j++)
```

```
            System.out.println("Element at index "+i+""+j+" = "+a[i][j]);
        }
    }
}
```

Jagged arrays

A jagged array is an array of arrays, where each inner array can have a different size. Unlike a standard multidimensional array where all inner arrays must be the same length, jagged arrays offer flexibility in structure. This means each row in a jagged array can have a different number of columns.



Declaration and Initialization of Jagged Array

The declaration of Jagged array is:

```
data_type array_name[][] = new data_type[n][];     // n= no. of rows

array_name[0] = new data_type[n1]       //n1= no. of columns in row-1
array_name[1] = new data_type[n2]       //n2= no. of columns in row-2
array_name[2] = new data_type[n3]       //n3= no. of columns in row-3
                    .
                    .
                    .
and so on
```

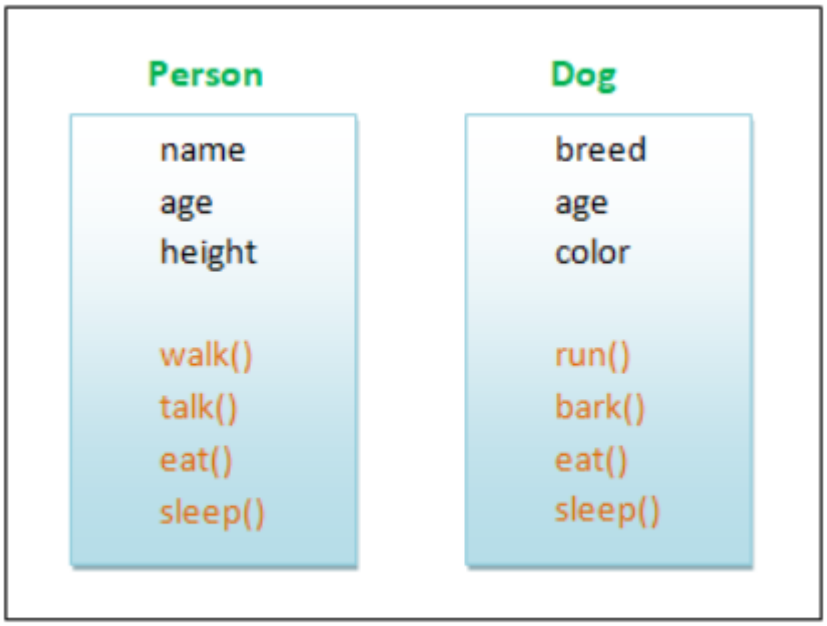
12. Introducing Classes: Class Fundamentals

Classes and objects are the key concepts to understand object oriented programming. Java programming is based on these two terms.

Real time example of class

Some of the real world examples of class are Person, Vehicle, House, Tree, Dog etc. For an example a Person class have properties like name, age, height etc. and behaviors like

walk, talk, eat, sleep etc. Now if you create an object of this class, it will have these properties and behaviors inside it. Once you created a class, you can create multiple objects of that class.



Real world examples of class

The basic syntax of declaring a class is :

```
class ClassName
  {
    // variables, constructors,
     // method declarations
  }
```

Here class is a keyword, used to define a class in java. ClassName is the name of class, given by the programmer. After the class name, it's the class body given inside { }. Variables, methods, constructors etc defined inside the balanced { } after the class name are the part of that class.

Example:

```
class Person
  {
    // Instance variables, describes state/properties of object of this class.
    String name;
    int age;
    int height;
    // methods, describes behaviors of object of this class.
```

```
    public void walk() {
        System.out.println("Hi my name is : "+name+", age : "+age+" year,"
        +" height : "+height+" cm. I can Walk");
      }
    public void talk() {
        System.out.println("Hi my name is : "+name+", age : "+age+" year,"
        +" height : "+height+" cm. I can Talk");
      }
  }

 class ClassDemo
  {
   public static void main(String [] args)
     {
       Person p1 = new Person(); // Creating object of class Person
       p1.name = "Rahul";
       p1.age = 20;
       p1.height = 170;
       p1.walk();
       p1.talk();
     }   }
```

13. Declaring Objects

First, declare a variable of the class type. This variable does not define an object. Instead, it is simply a variable that can refer to an object.

Second, you must acquire an actual, physical copy of the object and assign it to that variable. This is done using the new operator.

The new operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocate

Syntax given below shows the most basic way of creating an object of a class in java. There are other ways as well but we will not discuss that here.

ClassName objName = new ClassName();

Here ClassName is the name of class whose object need to be created and objName is the name of object, given by the programmer. The object name should be unique and should follow the convention given in identifier naming convention. The keyword new is used to create object in java. Using the () with ClassName calls the constructor of that class to create and initialize the object in memory.

Example

```
// Creates an object of Person class with name as P1.

  Person P1 = new Person();

   // Creates an object of MyFirstProgram class with name as mfp.
```

MyFirstProgram mfp = new MyFirstProgram();

14. method in Java

A method is a block of code or collection of statements or a set of code grouped together to perform a certain task or operation. It is used to achieve the reusability of code.

We write a method once and use it many times. We do not require to write code again and again. It also provides the easy modification and readability of code, just by adding or removing a chunk of code. The method is executed only when we call or invoke it.

Method Declaration

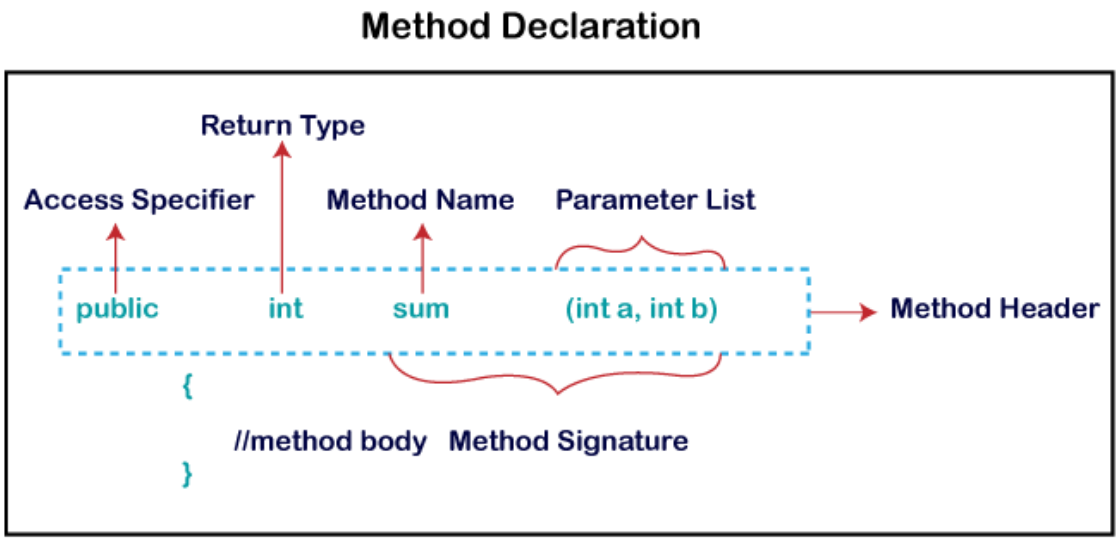
Syntax:

type name(*parameter-list*)

{

// body of method

}



Method Signature: Every method has a method signature. It is a part of the method declaration. It includes the method name and parameter list.

Access Specifier: Access specifier or modifier is the access type of the method. It specifies the visibility of the method. Java provides four types of access specifier.

Public: The method is accessible by all classes when we use public specifier in our application.

Private: When we use a private access specifier, the method is accessible only in the classes in which it is defined.

Protected: When we use protected access specifier, the method is accessible within the same package or subclasses in a different package.

Default: When we do not use any access specifier in the method declaration, Java uses default access specifier by default. It is visible only from the same package only.

Return Type: Return type is a data type that the method returns. It may have a primitive data type, object, collection, void, etc. If the method does not return anything, we use void keyword.

Method Name: It is a unique name that is used to define the name of a method. It must be corresponding to the functionality of the method. Suppose, if we are creating a method for subtraction of two numbers, the method name must be subtraction(). A method is invoked by its name.

Parameter List: It is the list of parameters separated by a comma and enclosed in the pair of parentheses. It contains the data type and variable name. If the method has no parameter, left the parentheses blank.

Method Body: It is a part of the method declaration. It contains all the actions to be performed. It is enclosed within the pair of curly braces.

- Types of Method
- There are two types of methods in Java:
- Predefined Method- **length(), equals(), compareTo(), sqrt(),**
- User-defined Method-

Example

```
import java.util.Scanner;

public class EvenOdd

{

public static void main (String args[])

{

Scanner scan=new Scanner(System.in);

System.out.print("Enter the number: ");

int num=scan.nextInt();

findEvenOdd(num);

}

public  void findEvenOdd(int num)

{

if(num%2==0)

System.out.println(num+" is even");
```

```
else
System.out.println(num+" is odd");
}
}
```

The name of the method is specified by name. The parameter-list is a sequence of type and identifier pairs separated by commas. Parameters are essentially variables that receive the value of the arguments passed to the method when it is called. If the method has no parameters, then the parameter list will be empty. Methods that have a return type other than void return a value to the calling routine using the following form of the return statement:

- **Syntax:**

  return value;

Overloading Methods

When two or more methods within the same class that have the same name, but their parameter declarations are different. The methods are said to be overloaded, and the process is referred to as method overloading.

Method overloading is one of the ways that Java supports polymorphism.

There are two ways to overload the method in java

1. By changing number of arguments

2. By changing the data type

Method Overloading: changing no. of arguments

```
class Adder{
static int add(int a,int b)
{
return a+b;
}
static int add(int a,int b,int c)
{
return a+b+c;
}
}
class TestOverloading1{
```

```
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(11,11,11));
}}
```

Method Overloading: changing data type of arguments

```
class Adder{
static int add(int a, int b)
{
return a+b;
}
static double add(double a, double b)
{
return a+b;
}
}
class TestOverloading2{
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(12.3,12.6));
}}
```

15. Constructors

Constructors are special member functions whose task is to initialize the objects of its class. It is a special member function, it has the same as the class name. Java constructors are invoked when their objects are created. It is named such because, it constructs the value, that is provides data for the object and are used to initialize objects. The constructor in Java cannot be abstract, static, final or synchronized and these modifiers are not allowed for the constructor.

Rules for creating Java constructor

There are two rules defined for the constructor.

Constructor name must be the same as its class name

A Constructor must have no explicit return type

A Java constructor cannot be abstract, static, final, and synchronized

There are two types of constructors:

1. Default constructor (no-arg constructor)

   A constructor having no parameter is known as default constructor and no-arg constructor

Example:

```
class Bike1{

Bike1()

{

System.out.println("Bike is created");

}


public static void main(String args[]){

Bike1 b=new Bike1();

}

}
```

2. Parameterized constructor

A constructor which has a specific number of parameters is called parameterized constructor. Parameterized constructor is used to provide different values to the distinct objects.

```
 class Student4{

   int id;
   String name;
   //creating a parameterized constructor
   Student4(int i,String n){
   id = i;
   name = n;
   }
   //method to display the values
   void display(){
System.out.println(id+" "+name);
}
   public static void main(String args[]){
   //creating objects and passing values
   Student4 s1 = new Student4(111,"Karan");
   Student4 s2 = new Student4(222,"Aryan");
   //calling method to display the values of object
   s1.display();
   s2.display();     } }
```

Constructor	Method
Constructor name must be same as the class name.	Method name may or may not be same as class name.
Constructor must not have return type.	Method must have return type.
Constructor is used to initialize the state of an object.	Method is used to expose behavior of an object.
Constructor is invoked(called) implicitly.	Method is invoked explicitly.
Java compiler provides a defau lt constructor if you don't declare any constructor in your class.	Method is not provided by java compiler in any case.
You can not use keywords like abstract, final, static etc with constructors.	These keywords can be used with methods.

16. Garbage Collection

Objects are dynamically allocated by using the new operator, dynamically allocated objects must be manually released by use of a delete operator. Java takes a different approach; it handles deallocation automatically this is called garbage collection. When no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed. Garbage collection only occurs sporadically (if at all) during the execution of your program. It will not occur simply because one or more objects exist that are no longer used.

Purpse

a. Memory management Technique
b. Process of freeing object
c. No longer reference by the program
d. Find and delete unreachable object
e. Free space as much as possible
f. Free unreferenced object
g. Combat heap fragmentation
h. Relieves the programmer from manual freeing the memory
i. Helps in ensuring program integrity
j. Disadvantages

a. Extra overhead
b. Less control over scheduling of CPU time

17. this keyword

this keyword is used to to refer to the object that invoked it. this can be used inside any method to refer to the current object. That is, this is always a reference to the object on which the method was invoked.
this() can be used to invoke current class constructor.
Example

```
Box(double w, double h, double d)
{
this.width = w;
this.height = h;
this.depth = d;
}
```

Here is given the 6 usage of java this keyword.
this can be used to refer current class instance variable.
this can be used to invoke current class method (implicitly)
this() can be used to invoke current class constructor.
this can be passed as an argument in the method call.
this can be passed as argument in the constructor call.
this can be used to return the current class instance from the method.

Example:

```
class Student
{
int id;
String name;
student(int id, String name)
{
this.id = id;
this.name = name;
}
void display()
{
System.out.println(id+" "+name);
}
public static void main(String args[])
{
Student stud1 = new Student(01,"Tarun");
Student stud2 = new Student(02,"Barun");
stud1.display();
stud2.display();
}
}
```

UNIT – II- METHOD OVERLOADING AND INHERITANCE

18. Method overloading

If a class has multiple methods having same name but different in parameters, it is known as Method Overloading.

How to perform method overloading in Java?

1. Overloading by changing the number of parameters

2. Method Overloading by changing the data type of parameters

Here are the key characteristics of method overloading in Java:

1. Same method name: Multiple methods can have the same name.
2. Different parameter lists: Each method must have a unique parameter list, which includes the number and types of parameters.
3. Same return type: The return type of the methods can be the same or different.
4. Compile Time polymorphism: Method overloading is resolved at compile time, which means the compiler determines which method to call based on the method signature.

Different Ways of Method Overloading in Java

The different ways of method overloading in Java are mentioned below:

a. Changing the Number of Parameters
b. Changing Data Types of the Arguments
c. Changing the Order of the Parameters of Methods

1. Changing the Number of Parameters

Method overloading can be achieved by changing the number of parameters while passing to different methods.

Example:

```
// Importing required classes

import java.io.*;

// Class 1

// Helper class

class Product {

  // Method 1

   // Multiplying two integer values

   public int multiply(int a, int b)
```

```
    {
        int prod = a * b;
        return prod;
    }

    // Method 2
    // Multiplying three integer values
    public int multiply(int a, int b, int c)
    {
        int prod = a * b * c;
        return prod;
    }
}

// Class 2
// Main class
class Geeks {

    // Main driver method
    public static void main(String[] args)
    {
        // Creating object of above class inside main()
        // method
        Product ob = new Product();

        // Calling method to Multiply 2 numbers
        int prod1 = ob.multiply(1, 2);

        // Printing Product of 2 numbers
```

```
        System.out.println(
            "Product of the two integer value: " + prod1);

        // Calling method to multiply 3 numbers
          int prod2 = ob.multiply(1, 2, 3);

        // Printing product of 3 numbers
        System.out.println ("Product of the three integer value: " + prod2);
      }
    }
```

2**. Changing Data Types of the Arguments**

In many cases, methods can be considered overloaded if they have the same name but have different parameter types, methods are considered to be overloaded. Example: This example demonstrates method overloading by changing the data types of parameters in the Prod() method.

Example:

```
// Java Program to Illustrate Method Overloading
// By Changing Data Types of the Parameters
// Importing required classes
import java.io.*;

// Class 1
// Helper class
class Product {

    // Multiplying three integer values
    public int Prod(int a, int b, int c)
    {
        int prod1 = a * b * c;
        return prod1;
```

```
    }

    // Multiplying three double values.
    public double Prod(double a, double b, double c)
    {
        double prod2 = a * b * c;
        return prod2;
    }
}

class Geeks {
    public static void main(String[] args)
    {
        Product obj = new Product();

        int prod1 = obj.Prod(1, 2, 3);
        System.out.println(
            "Product of the three integer value: " + prod1);

        double prod2 = obj.Prod(1.0, 2.0, 3.0);
        System.out.println(
            "Product of the three double value: " + prod2);
    }
}
```

3. Changing the Order of the Parameters of Methods

Method overloading can also be implemented by rearranging the parameters of two or more overloaded methods. For example, if the parameters of method 1 are (String name, int roll_no) and the other method is (int roll_no, String name) but both have the same name, then these 2 methods are considered to be overloaded with different sequences of parameters.

Example: This example demonstrates method overloading by changing the order of parameters in the StudentId() method.

```
// Java Program to Illustrate Method Overloading
// By changing the Order of the Parameters

// Importing required classes
import java.io.*;

// Class 1
// Helper class
class Student {

    // Method 1
    public void StudentId(String name, int roll_no)
    {
        System.out.println("Name: " + name + " "
                           + "Roll-No: " + roll_no);
    }

    // Method 2
    public void StudentId(int roll_no, String name)
    {
        // Again printing name and id of person
        System.out.println("Roll-No: " + roll_no + " "
                           + "Name: " + name);
    }
}

// Class 2
// Main class
```

```
class Geeks {

    // Main function
    public static void main(String[] args)
    {
        // Creating object of above class
        Student obj = new Student();

        // Passing name and id
        // Note: Reversing order
        obj.StudentId("Sweta", 1);
        obj.StudentId(2, "Gudly");
    }
}
```

19. Object as parameters

Object as an argument is use to establish communication between two or more objects of same class as well as different class, i.e, user can easily process data of two same or different objects within function.

Passing Object as Parameters to Methods

Passing parameters is a fundamental concept in programming that allows you to pass values or objects from one part of your code to another. Here are the different ways to pass parameters. In Java, there are two related but distinct concepts: “passing by reference” and “passing an object reference”. Understanding the difference between these two concepts is crucial to writing effective and efficient Java code.

Passing By Reference

Passing by reference refers to a programming paradigm where a function or method receives a reference to a variable as an argument. This means that any changes made to the variable within the function or method affect the original variable. However, Java does not support passing primitive types (e.g., int, boolean, char) by reference. Instead, Java passes primitive types by value.

Example:

```
class Car {
    String model;
```

```
    String color;
    public Car(String model, String color){
        this.model = model;
        this.color = color;
    }

    public void displayCarDetails(){
        System.out.println("Model: " + model);
        System.out.println("Color: " + color);
    }
}

class Garage {
    public void park(Car car){
       System.out.println("Parking a car in the garage:");
       car.displayCarDetails();
    }
}

public class Main {
    public static void main(String[] args) {
        // Create Car objects
        Car car1 = new Car("BMW", "Black");
        Car car2 = new Car("Tesla", "Red");

        // Create Garage object
        Garage garage = new Garage();

        // Pass car objects as parameters to the park method
```

```
        garage.park(car1);
        garage.park(car2);
    }
}
```

20. Returning Objects

In Java, a method can return any type of data, including class types that you can create.

Example:

```
public class MyClass {
  private int value;


  public MyClass(int value)
{
    this.value = value;
  }


  public int getValue()
{
    return value;
  }
}
public class ObjectReturnExample
{
  public static MyClass createMyClassObject(int value)
{
    return new MyClass(value);
  }
  public static void main(String[] args)
{
    MyClass obj = createMyClassObject(10);
```

```
    System.out.println(obj.getValue()); // Output: 10
  }
}
```

21. Recursion

Recursion in java is a process in which a method calls itself continuously. A method in java that calls itself is called recursive method. It makes the code compact but complex to understand.

How Recursion Works?

1. A method calls itself, either directly or indirectly, to solve a problem.

2. The method breaks down the problem into smaller sub-problems of the same type.

3. The method solves each sub-problem recursively until it reaches a base case.

4. The base case is a trivial case that can be solved directly without recursion.

5. The method returns the solution to each sub-problem, and the solutions are combined to solve the original problem.

Syntax:

```
returntype methodname()
{
//code to be executed
methodname();//calling same method
}
```

Example: Factorial Calculation

```
public class RecursionExample3 {
   static int factorial(int n){
        if (n == 1)
          return 1;
        else
          return(n * factorial(n-1));
   }

public static void main(String[] args) {
```

```
System.out.println("Factorial of 5 is: "+factorial(5));
}
}
```

Types of Recursion: There are two types of recursion. They are

1. Direct Recursion: A method calls itself directly, as in the example above.

2. Indirect Recursion: A method calls another method, which in turn calls the original method.

Advantages of Recursion:

Recursion has several advantages.

They are

1. Simplifies complex problems: Recursion can break down complex problems into smaller, more manageable sub-problems.

22. Access Control

The access modifiers in Java specify the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

Java provides several access control modifiers that can be used to control access to classes, methods,

and fields. These modifiers are:

1. public: Members declared as public are accessible from anywhere in the program.

2. private: Members declared as private are accessible only within the same class.

3. protected: Members declared as protected are accessible within the same class and within subclasses of that class.

4. default (or package-private): Members declared without any access modifier are accessible within the same package.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Private Access Control:

A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

Example:

```
class A
{
private int data=40;
private void msg()
{
System.out.println("Hello java");
}
}
public class Simple
{
 public static void main(String args[])
{
   A obj=new A();
   System.out.println(obj.data);//Compile Time Error
   obj.msg();//Compile Time Error
   }
```

Default Access Control:

If you don't use any modifier, it is treated as default by default. The default modifier is accessible only within package.

Example:

```
package pack;

class A{

  void msg()

{

System.out.println("Hello");

}

}

package mypack;

import pack.*;

class B{

  public static void main(String args[]){

   A obj = new A();//Compile Time Error

   obj.msg();//Compile Time Error
```

```
    }
  }
```

Protected Access Control:

The protected access modifier is accessible within package and outside the package but through inheritance only. The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

Example:

```
package pack;
public class A
{
protected void msg()
{
System.out.println("Hello");
}
}
package mypack;
import pack.*;
class B extends A
{
 public static void main(String args[])
{
   B obj = new B();
   obj.msg();
  }
}
```

we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance

Public Access Control:

The public access modifier is accessible everywhere. It has the widest scope among all other modifiers.

Example:

```
package pack;
public class A
{
public void msg()
{
System.out.println("Hello");
}
}
package mypack;
import pack.*;
class B{
  public static void main(String args[])
{
   A obj = new A();
   obj.msg();
  }
}
```

23. Static

The static keyword in Java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes

The static can be:

- Variable (also known as a class variable)
- Method (also known as a class method)
- Block
- Nested class

The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.The static variable gets memory only once in the class area at the time of class loading.

Advantages of static variable

It makes your program memory efficient (i.e., it saves memory).

Java static variable

If you declare any variable as static, it is known as a static variable.

```
class Student{
    int rollno;
    String name;
    String college="ITS";
}
```

Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created. All students have its unique rollno and name, so instance data member is good in such case. Here, "college" refers to the common property of all objects. If we make it static, this field will get the memory only once.

```
class Student
{
  int rollno;//instance variable
  String name;
static String college ="ITS";//static variable
  //constructor
  Student(int r, String n){
  rollno = r;
  name = n;
  }
void display (){
System.out.println(rollno+" "+name+" "+college);}
}
//Test class to show the values of objects
public class TestStaticVariable1{
 public static void main(String args[]){
 Student s1 = new Student(111,"Karan");
 Student s2 = new Student(222,"Aryan");
```

```
 s1.display();
 s2.display();
 }
}
```

If you apply static keyword with any method, it is known as static method.A static method belongs to the class rather than the object of a class.A static method can be invoked without the need for creating an instance of a class.A static method can access static data member and can change the value of it.

```
class Student{
    int rollno;
    String name;
    static String college = "ITS";
    //static method to change the value of static variable
    static void change(){
    college = "BBDIT";
    }
Student(int r, String n){
    rollno = r;
    name = n;
    }
 void display()
{System.out.println(rollno+" "+name+" "+college);
}
}
public class TestStaticMethod{
   public static void main(String args[]){
   Student.change    Student s1 = new Student(111,"Karan");
   Student s2 = new Student(222,"Aryan");
   Student s3 = new Student(333,"Sonoo");
   //calling display method
```

```
    s1.display();
    s2.display();
    s3.display();
    }
}
```

24. Final

The final keyword in java is used to restrict the user. The java final keyword can be used in many context.

Final can be:

- variable
- method
- class

1) Java final variable

```
class Bike9{
 final int speedlimit=90;//final variable
 void run(){
  speedlimit=400;
 }
 public static void main(String args[]){
 Bike9 obj=new  Bike9();
 obj.run();
 }
}//end of class
```

If you make any variable as final, you cannot change the value of final variable(It will be constant).

2) Java final method

If you make any method as final, you cannot override it.

```
class Bike{
  final void run(){
System.out.println("running");}
}
```

```
class Honda extends Bike{
  void run()
{
System.out.println("running safely with 100kmph");}
  public static void main(String args[]){
  Honda honda= new Honda();
  honda.run();
  }
}
```

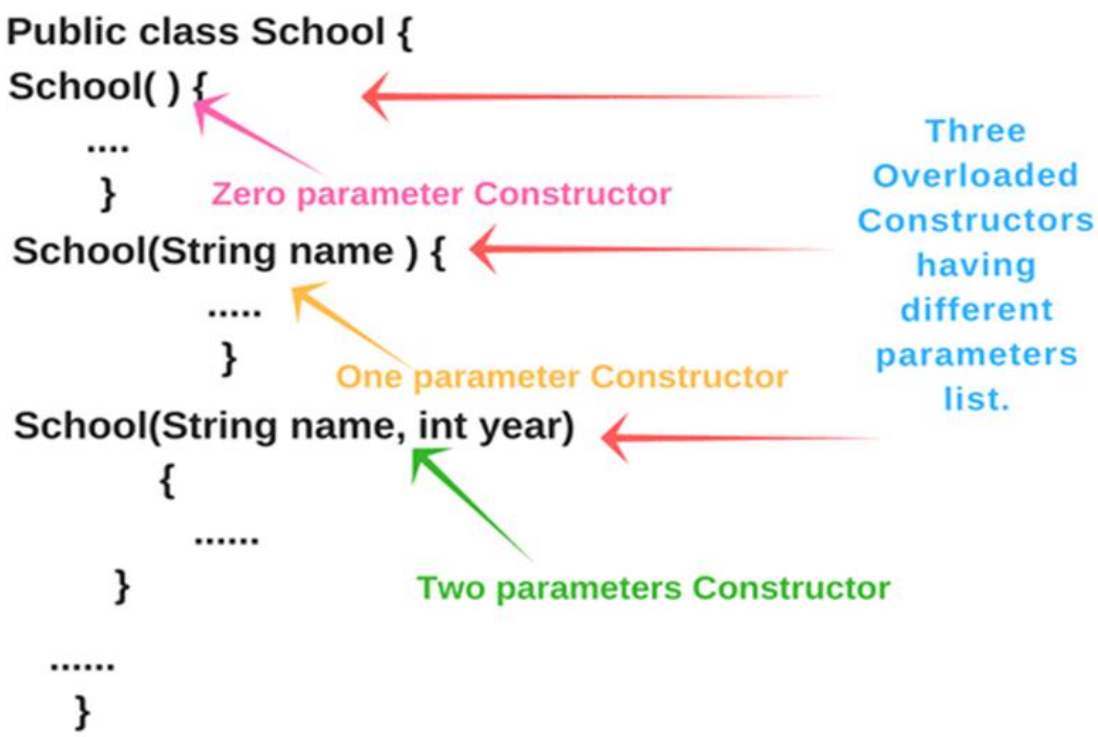
3) Java final class

If you make any class as final, you cannot extend it

```
final class Bike
{
}
class Honda1 extends Bike{
  void run()
{
System.out.println("running safely with 100kmph");
}
public static void main(String args[]){
  Honda1 honda= new Honda1();
  honda.run();
  }
}
```

25. Constructor Overloading

The constructor overloading can be defined as the concept of having more than one constructor with different parameters so that every constructor can perform a different task.



Default constructor (no-arg constructor)

A constructor having no parameter is known as default constructor and no-arg constructor.

Example:

```
class Main {
  int i;
private Mainq()              // constructor with no parameter
{
    i = 5;
    System.out.println("Constructor is called");
  }
  public static void main(String[] args)
{
    // calling the constructor without any parameter
    Main obj = new Main();
    System.out.println("Value of i: " + obj.i);
```

```
  }
}
```

Parameterized Constructors

A constructor which has a specific number of parameters is called parameterized constructor. Parameterized constructor is used to provide different values to the distinct objects.

```
class Main {
  String languages;
Main(String lang)            // constructor accepting single value
{
    languages = lang;
    System.out.println(languages + " Programming Language");
  }
  public static void main(String[] args)
{
    // call constructor by passing a single value
    Main obj1 = new Main("Java");
    Main obj2 = new Main("Python");
    Main obj3 = new Main("C");
  }
}
```

Difference Between Constructor Overloading and Method Overloading in Java

	Constructor	Method
Name	Constructor's name must be same as the name of the class.	Method's name can be anything.
Return Type	Constructor doesn't have a return type.	Method must have a return type.
Call	Constructor is invoked implicitly by the system.	Method is invoked by the programmer.
Main Job	Constructor can be used to initialize an object.	Method consists of Java code to be executed.
Overload	Constructor can be Overload.	Method also can be overload.

Constructor Overloading	Method Overloading
Writing more than 1 constructor in a class with a unique set of arguments is called as Constructor Overloading	Writing more than one method within a class with a unique set of arguments is called as method overloading
All constructors will have the name of the class	All methods must share the same name
Overloaded constructor will be executed at the time of instantiating an object	An overloaded method if not static can only be called after instantiating the object as per the requirement
An overloaded constructor cannot be static as a constructor relates to the creation of an object	Overloaded method can be static, and it can be accessed without creating an object
An overloaded constructor cannot be final as constructor is not derived by subclasses it won't make sense	An overloaded method can be final to prevent subclasses to override the method
An overloaded constructor can be private to prevent using it for instantiating from outside of the class	An overloaded method can be private to prevent access to call that method outside of the class

26. Inheritance Basis

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviours of a parent object. It is an important part of OOPs (Object Oriented programming system). The idea behind inheritance in Java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of the parent class. Moreover, you can add new methods and fields in your current class also. Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

Why to use inheritance in java?

For Method Overriding
For Code Reusability

Terms used in Inheritance

Superclass: The class from which another class inherits. Also known as the parent class.
Subclass: The class that inherits from another class. Also known as the child class.

Inheritance: The process by which one class can inherit the properties and behavior of another class.
Extends: The keyword used to indicate that a subclass inherits from a superclass.
Implements: The keyword used to indicate that a class implements an interface.
Override: The process of providing a different implementation of a method that is already defined in a superclass.
Overload: The process of providing multiple definitions of a method with different parameter lists.
Constructor: A special method that is used to initialize objects when they are created.
Super: A keyword used to access the members of a superclass from a subclass.
This: A keyword used to access the members of the current class.
Polymorphism: The ability of an object to take on multiple forms, depending on the context in which it is used.
Method Hiding: A situation in which a subclass provides a different implementation of a method that is already defined in a superclass, but with a different return type.
Method Overriding: A situation in which a subclass provides a different implementation of a method that is already defined in a superclass, with the same return type.
Final: A keyword used to indicate that a method or variable cannot be overridden or changed.
Abstract: A keyword used to indicate that a class or method cannot be instantiated or implemented directly.

Syntax

```
class Subclass-name extends Superclass-name

{

  //methods and fields

}
```

The extends keyword indicates that you are making a new class that derives from an existing class.

Inheritance Types:

Inheritance in Java can be categorized into different types based on how classes are related to each other. Java supports some types of inheritance while restricting others to avoid complexity. They are:

1. **Single Inheritance:** A subclass inherits from a single superclass.

2. **Multilevel Inheritance:** A subclass inherits from a superclass that itself inherits from another superclass.

3. **Hierarchical Inheritance:** Multiple subclasses inherit from a single superclass.

4. **Multiple Inheritance (Using Interfaces):** A class implements multiple interfaces.

5. **Hybrid Inheritance:** A combination of multiple inheritance and multilevel inheritance.

27. Using super

There are two terms 'super' and 'super()' that are used to access the members of a superclass and to call the constructor of the superclass from a subclass.

1. 'super'

super is a keyword in Java that is used to access the members (methods and fields) of a superclass from a subclass. It is used to refer to the superclass's members, and can be used in various contexts,

such as:
• Accessing superclass fields: super.fieldName
• Calling superclass methods: super.methodName()

2. 'super()'

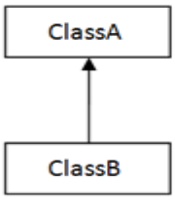
The super() call is a specific use of the super keyword, where it is used to call the constructor of the superclass from a subclass. The super() call must be the first statement in the subclass constructor.

Example for super:

```
public class Animal
{
public void sound()
{
System.out.println("The animal makes a sound.");
}
}
public class Dog extends Animal
{
@Override
public void sound()
{
super.sound(); // Calls the sound() method of the Animal
class
System.out.println("The dog barks.");
}
}
```

28. Single Inheritance

In single inheritance, a subclass is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance. In the below figure, 'A' is a parent class and 'B' is a child class. The class 'B' inherits all the properties of the class 'A'

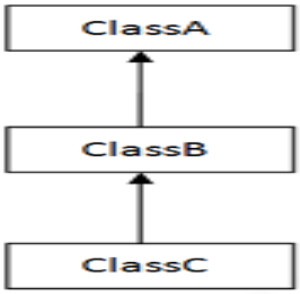


Example:

```
class Animal
{
void eat()
{
System.out.println("eating...");
} }
class Dog extends Animal
{
void bark()
{
System.out.println("barking...");
}  }
class TestInheritance {
public static void main(String args[])
{
Dog d=new Dog();
d.bark();
d.eat();
} }
```

29. Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the grandparent's members.



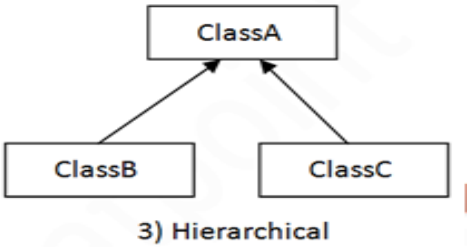
2) Multilevel

Example:

```
class Animal
{
void eat(){
System.out.println("eating...");
}
}
class Dog extends Animal{
void bark(){
System.out.println("barking...");
}
}
class BabyDog extends Dog{
void weep(){
System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

30. Hierarchical Inheritance

In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived classes B and C



When two or more classes inherits a single class, it is known as hierarchical inheritance.

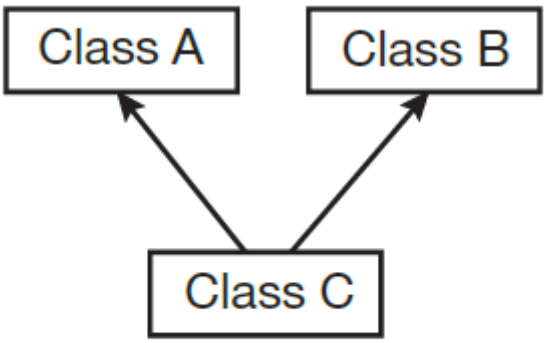
Example:

```
class Animal{
void eat(){
System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){
System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){
System.out.println("meowing...");}
}
}}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
```

```
c.eat();
//c.bark();//C.T.Error
}
}
```

Why multiple inheritance is not supported in java?

Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.



4) Multiple

An interface in Java is a blueprint of a class. It has static constants and abstract methods.The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

Example

```
// Java program to illustrate the
// concept of Multiple inheritance
import java.io.*;
import java.lang.*;
import java.util.*;
interface One
```

```
{
public void print_geek();
}
interface Two
{
public void print_for();
}
interface Three extends One, Two
{
public void print_geek();
}
class Child implements Three
{
@Override public void print_geek()
{
System.out.println("Geeks");
}
public void print_for()
{
System.out.println("for");
}
}
// Drived class
public class Main
{
public static void main(String[] args)
{
Child c = new Child();
c.print_geek();
```

```
c.print_for();

c.print_geek();

}

}
```

31. The Cosmic Superclass

In Java, a superclass (also known as a parent class) is a class that is inherited by another class. The inheriting class is called the subclass (or child class).

Here are the key features of a superclass in Java:

1. Inheritance: A superclass is inherited by one or more subclasses using the 'extends' keyword.
2. Instance Variable / State: A superclass can have state (instance variables) that are inherited by its subclasses.
3. Methods: A superclass can have both abstract and concrete methods that are inherited by its subclasses.
4. Constructors: A superclass can have constructors that are used to initialize its state.

Characteristics of a Superclass

1. Can be instantiated: A superclass can be instantiated, unless it is declared as abstract.

2. Can have state: A superclass can have state (instance variables) that are inherited by its subclasses.

3. Can have methods: A superclass can have both abstract and concrete methods that are inherited by its subclasses.

4. Can be inherited by multiple subclasses: A superclass can be inherited by multiple subclasses.

Types of Superclasses

1. Concrete Superclass: A concrete superclass is a class that can be instantiated and has a complete implementation.

2. Abstract Superclass: An abstract superclass is a class that cannot be instantiated and is designed to be inherited by other classes.

32. Method Overriding

If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java. In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- Method overriding is used to provide **the specific implementation of a method** which is **already provided by its superclass**.
- Method overriding is used for runtime polymorphism
- **Rules for Java Method Overriding**
    - The method must have the same name as in the parent class
    - The method must have the same parameter as in the parent class.
    - There must be an IS-A relationship (inheritance)

Example:

```
class Vehicle{
  //defining a method
void run(){
System.out.println("Vehicle is running");}
}
//Creating a child class
class Bike2 extends Vehicle{
  //defining the same method as in the parent class
  void run(){
System.out.println("Bike is running safely");
}
  public static void main(String args[]){
  Bike2 obj = new Bike2();//creating object
  obj.run();//calling method
  }  }
```

33. Abstract Class

Abstraction is a process of hiding the implementation details and showing only functionality to the user.Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

There are two ways to achieve abstraction in java

Abstract class

Interface

- Abstract method (method having only declaration)
- Definition will be return in immediate derived class
- abstract class contains atleast one abstract method
- Concrete class (class which does not have any abstract method)
- We cannot create object for abstract class

Abstract class

- A class which is declared as abstract is known as an abstract class.
- It can have abstract and non-abstract methods. It needs to be extended and its method implemented.

```
abstract class A
{
}
```

Abstract Method in Java

A method which is declared as abstract and does not have implementation is known as an abstract method.

```
abstract void printStatus();          //no method body and abstract
```

- In this example, Shape is the abstract class, and its implementation is provided by the Rectangle and Circle classes.
- Mostly, we don't know about the implementation class (which is hidden to the end user), and an object of the implementation class is provided by the **factory method**.
- A **factory method** is a method that returns the instance of the class. We will learn about the factory method later.

Example:

```
abstract class Animal {
  abstract void makeSound();
  public void eat() {
    System.out.println("I can eat.");
  } }
class Dog extends Animal {
  // provide implementation of abstract method
  public void makeSound() {
```

```
    System.out.println("Bark bark");
  } }
class Function {
  public static void main(String[] args) {
    // create an object of Dog class
    Dog d1 = new Dog();
    d1.makeSound();
    d1.eat();
  }
}
```

34. Final with Inheritance.

The final keyword is final that is we cannot change. We can use final keywords for variables, methods, and class. If we use the final keyword for the inheritance that is if we declare any method with the final keyword in the base class so the implementation of the final method will be the same as in derived class. We can declare the final method in any subclass for which we want that if any other class extends this subclass.

Final variables can be initialized in the following ways:

At the time of declaration
In a constructor
In an initializer block

Attempting to extend a final class will result in a compiler error.
• A final class cannot be subclassed, so there is no inheritance.
• A final method cannot be overridden, but it can still be inherited by a subclass.

Example:

```
// create a final class
final class FinalClass {
  public void display() {
    System.out.println("This is a final method.");
  }
}
// try to extend the final class
class Main extends FinalClass {
  public  void display() {
    System.out.println("The final method is overridden.");
  }
  public static void main(String[] args) {
    Main obj = new Main();
    obj.display();
  }}
```

UNIT – III - PACKAGES, INTERFACES AND EXCEPTION HANDLING

35. Packages — Importing packages

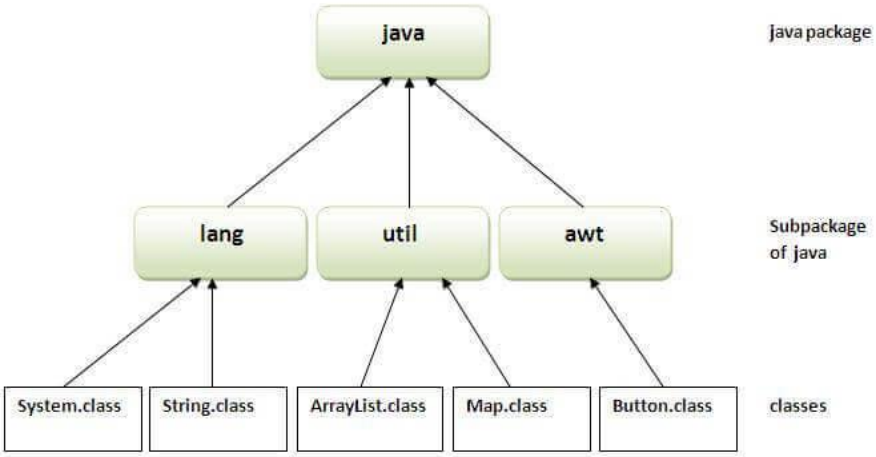
A java package is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package .There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Organization and Modularity:

Packages help structure code by grouping related types, making large projects more manageable and easier to navigate. Think of them as folders in a file system, where related files are kept together.

Advantage of Java Package

1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
2) Java package provides access protection.
3) Java package removes naming collision



Built-in Packages

The Java API is a library of prewritten classes, that are free to use, included in the Java Development Environment.

The library contains components for managing input, database programming, and much more.

Some of the commonly used built-in packages are:

1. Language package - import java.lang.*;
2. Input/output package - import java.io.*;
3. Utilities package - import java.Util.*;
4. Abstract Window toolkit (AWT) - import java.awt.*;
5. Swing package - import java.swing.*;
6. Applet package - import java.applet.*l
7. Networking package - import java.net.*;

Simple example of java package

The package keyword is used to create a package in java.

Syntax

import package.name.Class; // Import a single class

import package.name.* //Import the whole package

Example:

```
package mypack;
public class Simple{
 public static void main(String args[]){
   System.out.println("Welcome to package");
  }
}
```

Import a Package

There are many packages to choose from. In the previous example, we used the Scanner class from the java.util package. This package also contains date and time facilities, random-number generator and other utility classes.

To import a whole package, end the sentence with an asterisk sign (*). The following example will import ALL the classes in the java.util package.

User-defined Packages

These are the packages that are defined by the user. There are two steps for creating the user defined package.

1. Creating a Package
2. Importing or Accessing the package

How to compile java package

Syntax -> javac -d directory javafilename

Example->javac -d Simple.java

The -d switch specifies the destination where to put the generated class file.

How to access package from another package?

- There are three ways to access the package from outside the package.
  1. import package.*;
  2. import package.classname;
  3. fully qualified name

1) Using packagename.*

If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages. The import keyword is used to make the classes and interface of another package accessible to the current package.

Example :

```
package pack;

public class A{

public void msg(){

System.out.println("Hello");

}

}

package mypack;

import pack.*;

class B{

public static void main(String args[]){

  A obj = new A();

  obj.msg();

 }

}
```

2) Using packagename.classname

If you import package. classname then only declared class of this package will be accessible.

Example:

```
package pack;
public class A{
  public void msg()
{
System.out.println("Hello");}
}
import pack.A;
  class B{
  public static void main(String args[])
{
   A obj = new A();
   obj.msg();
  }
}
```

3) Using fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

Example:

```
package pack;
public class A{
  public void msg(){
System.out.println("Hello");}
}
package mypack;
class B{
  public static void main(String args[]){
```

```
    pack.A obj = new pack.A();//using fully qualified name
    obj.msg();
  }
}
```

36. **Interfaces**

- • An **interface in Java** is a **blueprint of a class**. It has **static constants and abstract methods.**
- The interface in Java is *a mechanism to achieve abstraction*
- An Interface is a user defined data type like similar class.

Use of Interface

• It is used to achieve abstraction.

• By interface, we can support the functionality of multiple inheritance.

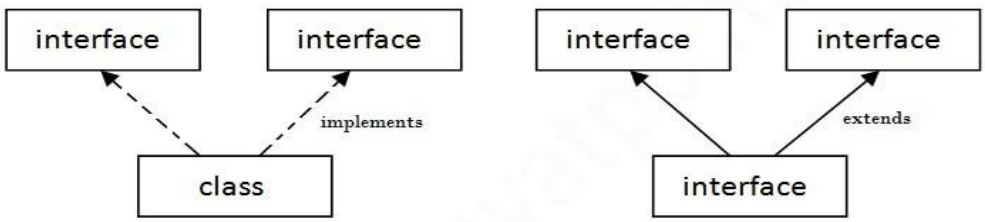
• It can be used to achieve loose coupling.

Difference between Class and Interface

Although Class and Interface seem the same there are certain differences between Classes and Interface.

The major differences between a class and an interface are mentioned below

Class	**Interface**
In class, you can instantiate variables and create an object	In an interface, you must initialize variables as they are final but you can't create an object.
A class can contain concrete (with implementation) methods	The interface cannot contain concrete (with implementation) methods.
The access specifiers used with classes are private, protected, and public	In Interface only one specifier is used- Public.



Multiple Inheritance in Java

Syntax of Interface

```
interface <interface_name>
        {
        // declare constant fields
        // declare methods that abstract
        // by default.
        }
```

Extending the interface

```
Interface< interface name>

{

statements;

}

interface <newinterface .extends <old interfacename>

{

statements;

}
```

Example

```
interface Printable{

void print();

}

interface Showable{

void show();

}

class A7 implements Printable,Showable{

public void print()

{System.out.println("Hello");

}

public void show()

{System.out.println("Welcome");}

public static void main(String args[]){
```

```
A7 obj = new A7();
obj.print();
obj.show();
 }
}
```

37. Static, Default and Private Methods.

Static Methods

Static methods were introduced in Java 8 to allow interfaces to define methods that belong to the interface itself, not to any instance of the interface.

This can be useful for a number of reasons:

a. To provide a way to define utility methods that are related to the interface. For example, the java.util.List interface defines a static method called of(). This method can be used to create a new list containing a specified set of elements.
b. To provide a way to define factory methods that can be used to create instances of classes that implement the interface. For example, the java.util.Optional interface defines a static method called ofNullable(). This method can be used to create a new Optional instance that contains a specified value, or an empty Optional instance if the value is null.
c. To provide a way to define methods that are not related to any specific instance of the interface. For example, the java.lang.Math interface defines a static method called abs(). This method can be used to calculate the absolute value of a number.

Default Methods

Default methods were introduced in Java 8 to allow interfaces to have concrete method implementations. This was a significant change to the Java language, as interfaces had previously only been able to define abstract methods. Default methods are defined using the default keyword.

There are several reasons why default methods were added to Java Interface:

Backward Compatibility: To allow interfaces to evolve without breaking existing code. Prior to Java 8, if a new method was added to an interface, all classes that implemented the interface had to be updated to provide an implementation for the new method. This could be a major undertaking, especially for large codebases. With default methods, new methods can be added to interfaces without requiring changes to existing code.

Common Functionality: To provide a way for interfaces to define common functionality. Default methods can be used to define common functionality that is shared by all classes that implement the interface. This can reduce the amount of code that needs to be written in each class.
Flexibility: To make interfaces more flexible. Default methods allow interfaces to be more flexible, as they can now provide concrete implementations for methods. This can make it easier to use interfaces in a variety of situations.

Private Methods

Private methods in Java interfaces were introduced in Java 9 to provide a way to encapsulate helper code that is not part of the public API of the interface.

This can be useful for a number of reasons:

Code Reusability: Private methods in interfaces can be used to encapsulate and reuse code within the interface, making it more maintainable and avoiding code duplication.

Improved Readability: Private methods allow us to break down complex logic in the interface into smaller, more manageable pieces.

38. Exception — Handling Fundamentals

The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that the normal flow of the application can be maintained. Exception handling in Java allows developers to manage runtime errors effectively by using mechanisms like try-catch block, finally block, throwing Exceptions, Custom Exception handling, etc.

What is Exception & Exception Handling in Java?

In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc.

Hierarchy of Java Exception classes

Exceptions in Java are divided into the two categories – Unchecked exceptions & Check exceptions. In this article we will learn how these works with syntax and examples.

Unchecked Exceptions in Java

In the hierarchy, all the descendant classes of the Runtime Exception come under this category.

Checked Exceptions in Java

The sub-classes of all other classes come under this Category.

As we have learned that there are four possibilities to throw and catch for an exception –

Default Throw and default catch

Default Throw and our catch

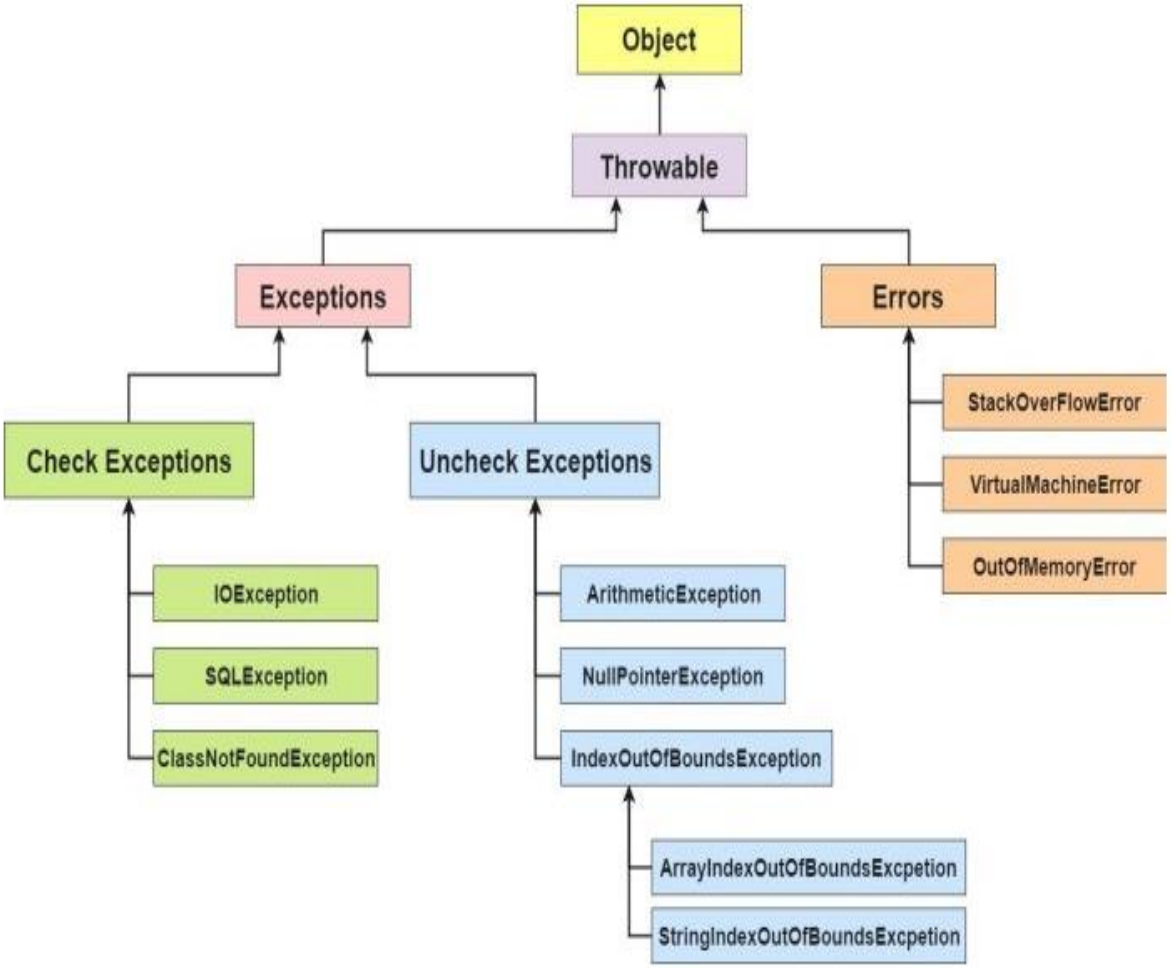
Our Throw and default catch

Our Throw and our catch

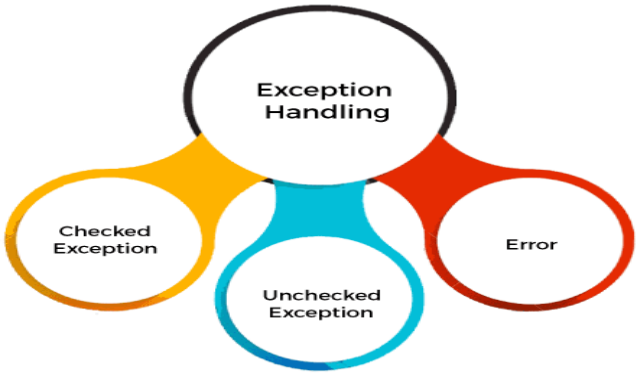
Below, we will understand the Default Throw and our catch –

Default Throw and our catch:

To implement this, we have to use 3 keywords which are tried, catch, and finally.



39. Exception Types



Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc**.**

Java Exception Keywords

try

The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block *must be followed by either catch or finally.*

catch

The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.

finally

The "*finally" block is used to execute the necessary code of the program*. It is executed whether an exception is handled or not.

throw

The "throw" keyword is used to throw an exception.

throws

The "throws" *keyword is used to declare exceptions*. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature**.**

Example:

```
public class JavaExceptionExample{
public static void main(String args[]){
int a=10, b=0;
int c=a/b;
System.out.println(c);
System.out.println("End of Program);
}
}
```

```
public class JavaExceptionExample{
public static void main(String args[]){
int a=10, b=0;
try{
int c=a/b;
}
Catch (Exception e) {
System.out.println("Error Occurred");
}
System.out.println(c);
System.out.println("End of Program);
}
}
```

1) A scenario where ArithmeticException occurs

int a=50/0;//ArithmeticException

2) A scenario where NullPointerException occurs

String s=null;

System.out.println(s.length());//NullPointerException

3) A scenario where NumberFormatException occurs

String s="abc";

int i=Integer.parseInt(s);//NumberFormatException

4) A scenario where ArrayIndexOutOfBoundsException occurs

int a[]=new int[5];

a[10]=50; //ArrayIndexOutOfBoundsException

40. Uncaught Exceptions

When an uncaught exception occurs, the JVM calls a special private method known dispatchUncaughtException( ), on the Thread class in which the exception occurs and terminates the thread.Uncaught Exception occurs when an exception is not caught by a programming construct or by the programmer, it results in an uncaught exception.

Example:

```
import java.util.Scanner;

public class UncaughtExceptionExample {

public static void main(String[] args) {

Scanner read = new Scanner(System.in);

System.out.println("Enter the a and b values: ");

    int a = read.nextInt();

    int b = read.nextInt();

    int c = a / b;

    System.out.println(a + "/" + b +" = " + c);

   }
```

```
}
```

41. Using Try and Catch

Java try block is used to enclose the code that might throw an exception. It must be used within the method.

```
try
{
//code that may throw an exception
}
catch(Exception_class_Name ref)
{

}
```

Syntax of try-finally block

```
try{
//code that may throw an exception
}
finally
{
}
```

Problem without exception handling

Example:

```
public class TryCatchExample1 {
    public static void main(String[] args) {
    int data=50/0; //may throw exception
    System.out.println("rest of the code");
        }     }
```

Solution by exception handling

Example:

```
public class TryCatchExample2 {
   public static void main(String[] args) {
      try
      {
      int data=50/0; //may throw exception
      }
         //handling the exception
      catch(ArithmeticException e)
      {
         System.out.println(e);
      }
      System.out.println("rest of the code");
   } }
```

Java Catch Multiple Exceptions

- A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.
- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for Arithmetic Exception must come before catch for Exception.

Example:

```
public class MultipleCatchBlock1 {

    public static void main(String[] args) {

        try{
            int a[]=new int[5];
            a[50]=30;
           }
           catch(ArithmeticException e)
             {
              System.out.println("Arithmetic Exception occurs");
             }
           catch(ArrayIndexOutOfBoundsException e)
             {
          System.out.println("ArrayIndexOutOfBounds Exception occurs");

             }
           catch(Exception e)
             {
              System.out.println("Parent Exception occurs");
             }
           System.out.println("rest of the code");
    }
}
```

42. Built-in Exceptions

Types of Exceptions in Java Built-in exceptions are the exceptions that are available in Java libraries. These exceptions are suitable to explain certain error situations. Below is the list of important built-in exceptions in Java.

Checked Exceptions

Checked exceptions are called compile-time exceptions because these exceptions are checked at compile-time by the compiler. Examples of Checked Exception are listed below:

1. ClassNotFoundException: Throws when the program tries to load a class at runtime but the class is not found because its not present in the correct location or it is missing from the project.

2. InterruptedException: Thrown when a thread is paused and another thread interrupts it.

3. IOException: Throws when input/output operation fails

4. InstantiationException: Thrown when the program tries to create an object of a class but fails because the class is abstract, an interface, or has no default constructor.

5. SQLException: Throws when there's an error with the database.

6. FileNotFoundException: Thrown when the program tries to open a file that doesn't exist

Unchecked Exceptions

The unchecked exceptions are just opposite to the checked exceptions. The compiler will not check

these exceptions at compile time. In simple words, if a program throws an unchecked exception, and even if we didn't handle or declare it, the program would not give a compilation error. Examples of

Unchecked Exception are listed below:

1**. ArithmeticException:** It is thrown when there's an illegal math operation.

2. ClassCastException: It is thrown when you try to cast an object to a class it does not belongs to.

3. NullPointerException: It is thrown when you try to use a null object (e.g. accessing its methods or fields)

4. ArrayIndexOutOfBoundsException: It occurs when we try to access an array element with an invalid index.

5. ArrayStoreException: It happens when you store an object of the wrong type in an array.

6. IllegalThreadStateException: It is thrown when a thread operation is not allowed in its current state

43. User defined exceptions.

Sometimes, the built-in exceptions in Java are not able to describe a certain situation. In such cases, users can also create exceptions, which are called "user-defined Exceptions".

Java provides us the facility to create our own exceptions by extending the Exception class. Creating our own Exception is known as a custom exception in Java

or a user-defined exception in Java. In simple words, we can say that a User-Defined Custom Exception or custom exception is creating your own exception class and throwing that exception using the "throw" keyword.

Example:

```
class MyException extends Exception {

   public MyException(String m) {

      super(m);

   }

}


// A Class that uses the above MyException
public class setText {
   public static void main(String args[]) {
      try {

         // Throw an object of user-defined exception
         throw new MyException("This is a custom exception");
      }
      catch (MyException ex) {
         System.out.println("Caught");
         System.out.println(ex.getMessage());
      }
   }
}
```

44. String: String Methods

In Java, string is basically an object that represents sequence of char values. An array of characters works same as Java string. For example:

char[] ch={'j','a','v','a','t','p','o','i','n','t'};
String s=new String(ch);
is same as:0→ String s="javatpoint";

Java String class provides a lot of methods to perform operations on strings such as compare(), concat(), equals(), split(), length(), replace(), compareTo(), substring() etc.

The java.lang.String class
implements *Serializable, Comparable* and *CharSequence* interfaces.

CharSequence Interface

The CharSequence interface is used to represent the sequence of characters. String, StringBuffer and StringBuilder classes implement it. It means, we can create strings in Java by using these three classes.

The Java String is immutable which means it cannot be changed. Whenever we change any string, a new instance is created. For mutable strings, you can use StringBuffer and StringBuilder classes

How to create a string object?

- There are two ways to create String object:
  - By string literal
  - By new keyword

1) String Literal

- Java String literal is created by using double quotes. For Example:
  String s="welcome";
- String s1="Welcome";
  String s2="Welcome";//It doesn't create a new instance

2) By new keyword

String s=new String("Welcome");//creates two objects and one reference variable In such case, JVM will create a new string object in normal (non-pool) heap memory, and the literal "Welcome" will be placed in the string constant pool. The variable s will refer to the object in a heap (non-pool).

Example:

```
public static void main(String args[])
{
String s1="java";//creating string by Java string literal
char ch[]={'s','t','r','i','n','g','s'};
String s2=new String(ch);//converting char array to string
String s3=new String("example");//creating Java string by new keyword
System.out.println(s1);
System.out.println(s2);
System.out.println(s3);
}
}
```

45. Operations

No.	Method	Description
1	char charAt(int index)	It returns char value for the particular index
2	int length()	It returns string length
3	static String format(String format, Object... args)	It returns a formatted string.
4	static String format(Locale l, String format, Object... args)	It returns formatted string with given locale.
5	String substring(int beginIndex)	It returns substring for given begin index.
6	String substring(int beginIndex, int endIndex)	It returns substring for given begin index and end index.
7	boolean contains(CharSequence s)	It returns true or false after matching the sequence of char value.
8	static String join(CharSequence delimiter, CharSequence... elements)	It returns a joined string.
9	static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)	It returns a joined string.
10	boolean equals(Object another)	It checks the equality of string with the given object.
11	String toLowerCase()	It returns a string in lowercase.
12	String toLowerCase(Locale l)	It returns a string in lowercase using specified locale.

13	**String toUpperCase()**	**It returns a string in uppercase.**
14	**String toUpperCase(Locale l)**	**It returns a string in uppercase using specified locale.**
15	**String trim()**	**It removes beginning and ending spaces of this string.**
16	**static String valueOf(int value)**	**It converts given type into string. It is an overloaded method.**

Java String charAt()

- The Java String class charAt() method returns *a char value at the given index number*.
- The index number starts from 0 and goes to n-1, where n is the length of the string. It returns StringIndexOutOfBoundsException, if the given index number is greater than or equal to this string length or a negative number.
- **Syntax** **public char charAt(int index)**

Java String concat

- The **Java String class concat()** method *combines specified string at the end of this string*. It returns a combined string. It is like appending another string.
- Signature
- The signature of the string concat() method is given below:
- **public** String concat(String anotherString)

Java String toLowerCase()

- The **java string toLowerCase()** method returns the string in lowercase letter. In other words, it converts all characters of the string into lower case letter.
- The toLowerCase() method works same as toLowerCase(Locale.getDefault()) method. It internally uses the default locale.

Java String toCharArray()

- The java string toCharArray() method converts this string into character array. It returns a newly created character array, its length is similar to this string and its contents are initialized with the characters of this string.

UNIT – IV- MULTITHREAD PROGRAMMING AND INPUT / OUTPUT STREAMS

46. Multithreaded Programming

It is a process of executing multiple threads simultaneously. A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking. However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context- switching between the threads takes less time than process. Java Multithreading is mostly used in games, animation, etc.

Advantages of Java Multithreading

1) It doesn't block the user because threads are independent and you can perform multiple operations at the same time.

2) You can perform many operations together, so it saves time.

3) Threads are independent, so it doesn't affect other threads if an exception occurs in a single thread.

Multitasking

- Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU.
- Multitasking can be achieved in two ways:
  - Process-based Multitasking (Multiprocessing)
  - Thread-based Multitasking (Multithreading)

1) Process-based Multitasking (Multiprocessing)

Each process has an address in memory. In other words, each process allocates a separate memory area. A process is heavyweight. Cost of communication between the process is high.

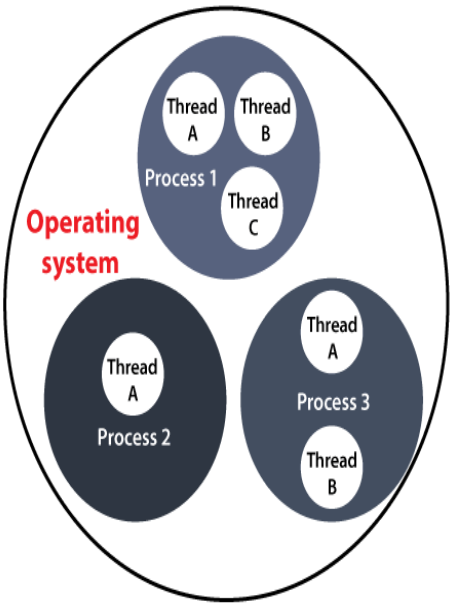
2) Thread-based Multitasking (MultithrCg)

Threads share the same address space .A thread is lightweight. Cost of communication between the thread is low.

What is Thread in java?

- A thread is a lightweight subprocess, the smallest unit of processing.
- It is a separate path of execution.
- Threads are independent. If there occurs exception in one thread, it doesn't affect other threads. It uses a shared memory area.

- In order **to perform complicated tasks in the background, we used the Thread concept in Java.**
- All the tasks are executed *without affecting the main program*.
- In a program or process, all the threads have their own separate path for execution, so each thread of a process is independent.



47. Thread Model

In a single-threaded system, a single control loop continuously checks for events, such as user input or file availability. When an event occurs, it processes it before moving to the next. However, if one task takes too long, the entire program gets stuck, wasting CPU time and making the system unresponsive. This can cause delays, as no other tasks can run until the current one is completed.

Java's multithreading removes the need for a main loop or constant polling. If one thread pauses, it doesn't stop the entire program. For example, while one thread waits for network data or user input, other threads can keep running. This is useful in animations, where a thread can pause between frames without freezing the whole system. If a thread gets blocked, only that threads stops, while the rest of the program continues smoothly.

Threads go through different states during execution. A thread can be running or ready to run when waiting for CPU time. It can be suspended, temporarily pausing its activity, and later resumed to continue from where it stopped. A thread may also become blocked while waiting for a resource.

Finally, a thread can be terminated, which stops it permanently, and once terminated, it cannot be restarted.

48. Thread Creation

In Java, a thread can be created by instantiating an object of the Thread class. There are two primary methods to achieve this:

- Implementing the Runnable interface
- Extending the Thread class

Implementing the Runnable interface:

The simplest way to create a thread is by implementing the Runnable interface. Runnable represents a unit of executable code, and any class that implements it can be used to create a thread. To do this, the class must define the run() method as follows:

public void run()

This method contains the code that will execute when the thread starts. Inside the run() method, you write the actions you want your thread to perform. It works like any other method, allowing you to call other methods, use classes, and declare variables. The key difference is that run() starts a new, separate execution path. The thread will continue running until run() completes.

Thread(Runnable threadObject, String threadName)

This constructor is designed to create a new Thread object that will execute the run() method of a class that implements the Runnable interface.

Runnable threadObject

This parameter expects an object that implements the Runnable interface.This object contains the run() method, which holds the code that the new thread will execute.

String threadName

This parameter allows you to assign a name to the newly created thread. This name is useful for debugging and monitoring purposes.

After creating a new thread, it does not start running immediately. To begin execution, you must call its start() method, which is defined in the Thread class. Internally, start() triggers a call to the run() method, allowing the thread to execute its task.

The start() method is defined in the Thread class and is used to begin the execution of a thread.

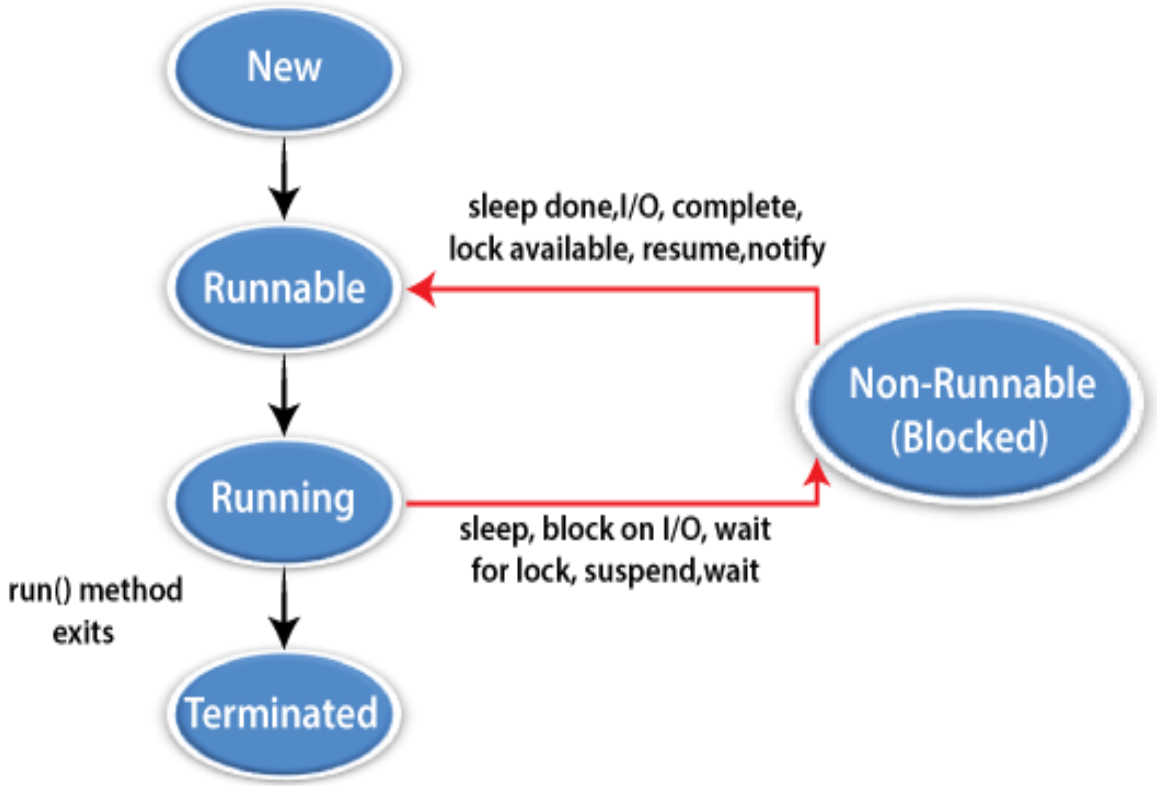
Its syntax is:

public void start()

When start() is called, the thread moves from the New state to the Runnable state, and the JVM schedules it for execution. This method internally invokes the run() method, allowing the thread to perform its defined task.

49. Life Cycle

A thread can be in one of the five states. According to sun, there is only 4 states in thread life cycle in java new, runnable, non-runnable and terminated. There is no running state.



1) New

The thread is in new state if you create an instance of Thread class but before the invocation of start() method.

2) Runnable

The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.

3) Running

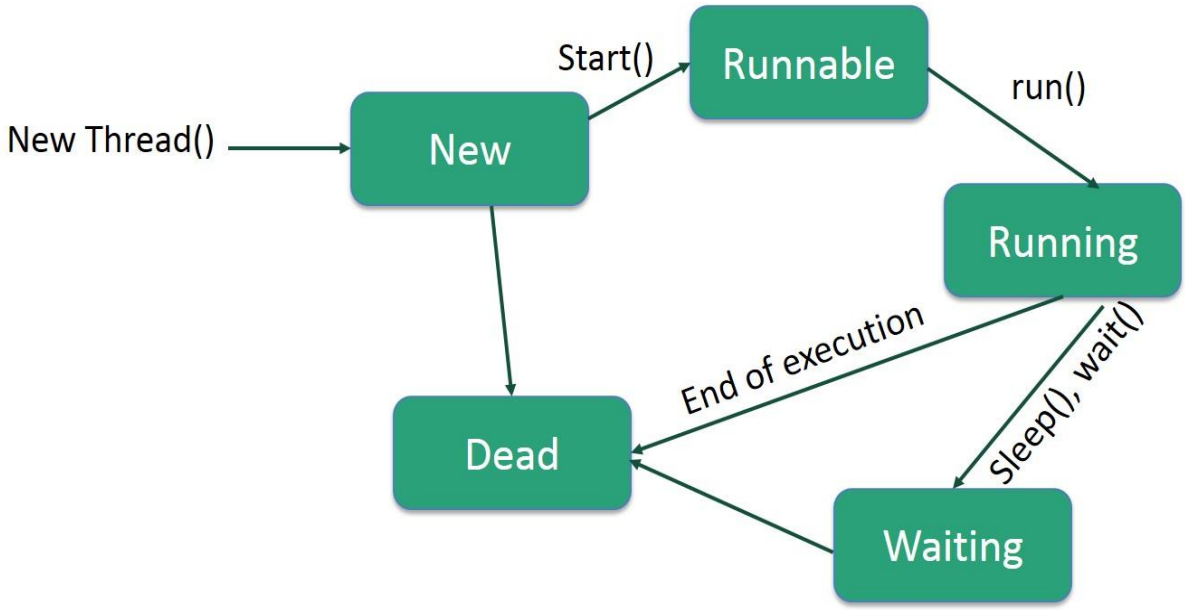
The thread is in running state if the thread scheduler has selected it.

4) Non-Runnable (Blocked)

This is the state when the thread is still alive, but is currently not eligible to run.

5) Terminated

A thread is in terminated or dead state when its run() method exits.



Waiting

Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.

Commonly used methods of Thread class:

1. **public void run():** is used to perform action for a thread.

2. **public void start():** starts the execution of the thread. JVM calls the run() method on the thread.

3. **public void sleep(long miliseconds):** Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.

4. **public void join():** waits for a thread to die.

5. **public void join(long miliseconds):** waits for a thread to die for the specified miliseconds.

6. **public int getPriority():** returns the priority of the thread.

7. **public int setPriority(int priority):** changes the priority of the thread.

8. **public String getName():** returns the name of the thread.

9. **public void setName(String name):** changes the name of the thread.

10. **public Thread currentThread():** returns the reference of currently executing thread.

Java Thread Example by extending Thread class

Example:

```
class Multi extends Thread{
public void run()
{
System.out.println("thread is running...");
}
public static void main(String args[])
{
Multi t1=new Multi();
t1.start();
}
}
```

- ***Output:***
- **thread is running...**

Example:

```
class Multi

{

public void display()

{

System.out.println("thread is running...");

}

public static void main(String args[])

{

Multi t1=new Multi();

t1. display();

}

}
```

- ***Output:***
- **thread is running...**

Java Thread Example by implementing Runnable interface

Example:

```
class abc implements Runnable
{
  public void run()
{
    System.out.println("Thread is running for Runnable Implementation");
  }
  public static void main(String args[])
{
    abc runnable=new abc();
    Thread t1 =new Thread(runnable);
    t1.start();
  }
}
```

Output:

thread is running...

Creating Multiple Threads

- Creating more than one thread to perform multiple tasks is called multithreading in Java.In multiple threading programming, multiple threads are executing simultaneously that improves the performance of CPU because CPU is not idle if other threads are waiting to get some resources. Multiple threads share the same address space in the heap memory. Therefore, It is good to create multiple threads to execute multiple tasks rather than creating multiple processes.

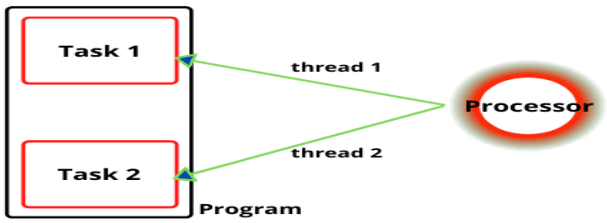


Fig: Thread-based Multitasking

50. Thread Priorities

Each thread has a priority. Priorities are represented by a number between 1 and 10. In most cases, the thread scheduler schedules the threads according to their priority (known as preemptive scheduling).

Setter & Getter Method of Thread Priority

public final int getPriority():

- The java.lang.Thread.getPriority() method returns the priority of the given thread.

public final void setPriority(int newPriority):

- The java.lang.Thread.setPriority() method updates or assign the priority of the thread to newPriority. The method throws IllegalArgumentException if the value newPriority goes out of the range, which is 1 (minimum) to 10 (maximum).

Constants defined in Thread class:

- **public static int MIN_PRIORITY**
- **public static int NORM_PRIORITY**
- **public static int MAX_PRIORITY**
- Default priority of a thread is 5 (NORM_PRIORITY). The value of MIN_PRIORITY is 1 and the value of MAX_PRIORITY is 10

Example:

```
public class Threadpriorityde mo extends Thread
{
public void run()
{
System.out.println("running thread name is:"+Thread.currentThread().
getName());
System.out.println("running thread priority is:"+Thread.
currentThread().getPriority());
}
public static void main(String args[]){
Threadpriorityde mo m1=new Threadpriorityde mo();
```

```
Threadpriritydemo m2=new Threadpriritydemo();
m1.setPriority(Thread.MIN_PRIORITY);
m2.setPriority(Thread.MAX_PRIORITY);
m1.start();
m2.start();
}
}
```

51. Synchronization of Threads

Synchronization in Java is the capability to control the access of multiple threads to any shared resource. Java Synchronization is better option where we want to allow only one thread to access the shared resource.

Why use Synchronization?

- **The synchronization is mainly used to**

    1. To prevent thread interference.
    2. To prevent consistency problem.

Types of Synchronization

- **There are two types of synchronization**

    1. Process Synchronization
    2. Thread Synchronization

- **Here, we will discuss only thread synchronization.**
- There are two types of thread synchronization mutual exclusive and inter-thread communication.
- **1. Mutual Exclusive**

    1. Synchronized method.
    2. Synchronized block.
    3. Static synchronization.

- **2. Cooperation (Inter-thread communication in java)**

Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data. It can be achieved by using the following three ways:

1. By Using Synchronized Method
2. By Using Synchronized Block
3. By Using Static Synchronization

Java Synchronized Method

- If you **declare any method as synchronized**, it is known as synchronized method.
- Synchronized method is **used to lock an object for any shared resource.**
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

Example:

```
class multiplicationTable{

void printTable(int n){

synchronized(this){//synchronized block

for(int i=1;i<=5;i++){

System.out.println(n*i);

try{

Thread.sleep(4000);

}catch(Exception e){System.out.println(e);}

}

}

}//end of the method

}

class MyThread1 extends Thread{

multiplicationTable mt;

MyThread1(multiplicationTable mt){

this.mt=mt;
```

```
}
public void run(){
mt.printTable(5);
}
}
class MyThread2 extends Thread{
multiplicationTable mt;
MyThread2(multiplicationTable mt){
this.mt=mt;
}
public void run(){
mt.printTable(100);
}
}
public class SynchronizedBlockDemo{
public static void main(String args[]){
multiplicationTable obj = new multiplicationTable();//only
one object
MyThread1 mt1=new MyThread1(obj);
MyThread2 mt2=new MyThread2(obj);
mt1.start();
mt2.start();
}
}
```

` **Output:**

```
5
10
15
20
25
```

52. Inter thread Communication

Inter-thread communication or Co-operation is all about allowing synchronized threads to communicate with each other. Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed.

- It is implemented by following methods of **Object class**:
    - wait()
    - notify()
    - notifyAll()

1) wait() method

The wait() method causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

The current thread must own this object's monitor, so it must be called from the synchronized method only otherwise it will throw exception.

2) notify() method

The notify() method wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.

- **Syntax:**
- **public final void notify()**

3) notifyAll() method

Wakes up all threads that are waiting on this object's monitor.

Syntax:

public final void notifyAll()

Why wait(), notify() and notifyAll() methods are defined in Object class not Thread class?

- It is because they are related to lock and object has a lock.

wait()	sleep()
The wait() method releases the lock.	The sleep() method doesn't release the lock.
It is a method of Object class	It is a method of Thread class
It is the non-static method	It is the static method
It should be notified by notify() or notifyAll() methods	After the specified amount of time, sleep is completed.

Example

```
class Customer{
int amount=10000;
synchronized void withdraw(int amount){//15000
System.out.println("going to withdraw...");
if(this.amount<amount){
System.out.println("Less balance; waiting for deposit...");
try{wait();}catch(Exception e){}
}
this.amount-=amount;
System.out.println("withdraw completed...");
}
synchronized void deposit(int amount){//10000
System.out.println("going to deposit...");
this.amount+=amount;
System.out.println("deposit completed... ");
notify();
}
}
class IPCDemo{
```

```
public static void main(String args[]){
final Customer c=new Customer();
new Thread(){
public void run()
{
c.withdraw(15000);
}
}.start();
new Thread(){
public void run(){
c.deposit(10000);
}
}.start();
}}
```

53. Suspending, Resuming and Stopping Threads

In early Java versions, the methods suspend(), resume(), and stop() were used to control thread execution. However, these methods were deprecated because they caused deadlocks and inconsistent

states.

suspend(): When a thread is suspended, it holds onto resources (like locks), causing other threads to be blocked indefinitely. The suspend() method of thread class puts the thread from running to waiting state. This method is used if you want to stop the thread execution and start it again when a certain event occurs. This method allows a thread to temporarily cease execution. The suspended thread can be resumed using the resume() method.

Syntax

```
public final void suspend()
```

Return

This method does not return any value.

Example:

```
public class JavaSuspendExp extends Thread
{
   public void run()
   {
      for(int i=1; i<5; i++)
      {
         try
         {
            // thread to sleep for 500 milliseconds
             sleep(500);
System.out.println(Thread.currentThread().getName());
         }
catch(InterruptedException e)
{System.out.println(e);
}
         System.out.println(i);
      }
   }
public static void main(String args[])
   {
      // creating three threads
      JavaSuspendExp t1=new JavaSuspendExp ();
      JavaSuspendExp t2=new JavaSuspendExp ();
      JavaSuspendExp t3=new JavaSuspendExp ();
       t1.start();
      t2.start();
      t2.suspend();
      t3.start();
   }
}
```

stop(): Terminates a thread abruptly, without allowing it to release resources properly, which can lead to memory leaks and data corruption.

resume(): Since suspend() could freeze a thread indefinitely, resume() became unreliable. Instead of using suspend() and resume(), Java now recommends using inter-thread communication (wait(), notify()) or a volatile flag to control thread execution safely. The resume() method of thread class is only used with suspend() method. This method is used to resume a thread which was suspended using suspend() method. This method allows the suspended thread to start again.

Syntax

```
public final void resume()
```

Return value

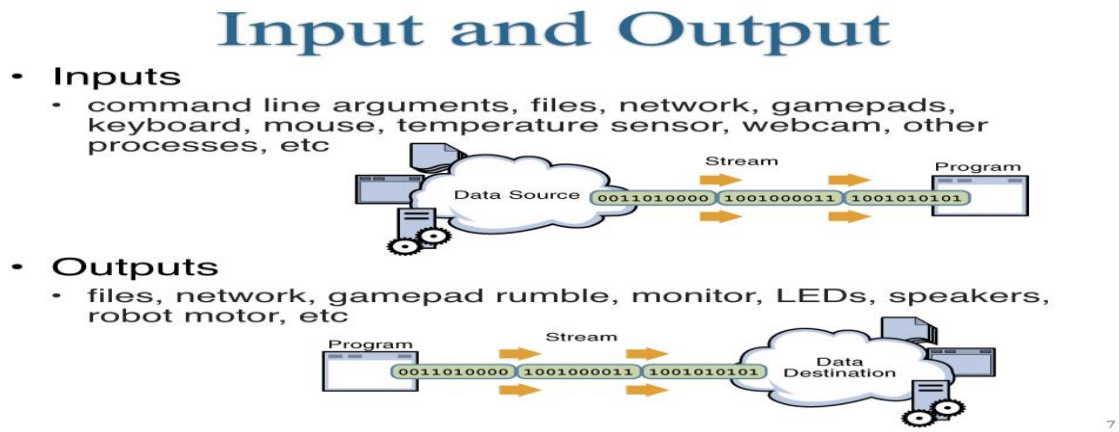
This method does not return any value.

Example

```
public class JavaResumeExp extends Thread
{
   public void run()
   {
      for(int i=1; i<5; i++)
      {
         try
         {
            sleep(500);
            System.out.println(Thread.currentThread().getName());
         }catch(InterruptedException e){System.out.println(e);}
         System.out.println(i);
      }
   }
public static void main(String args[])
   {

    JavaResumeExp t1=new JavaResumeExp ();
  JavaResumeExp t2=new JavaResumeExp ();
   JavaResumeExp t3=new JavaResumeExp ();
      t1.start();
      t2.start();
      t2.suspend();
      t3.start();
      t2.resume();
   }   }
```

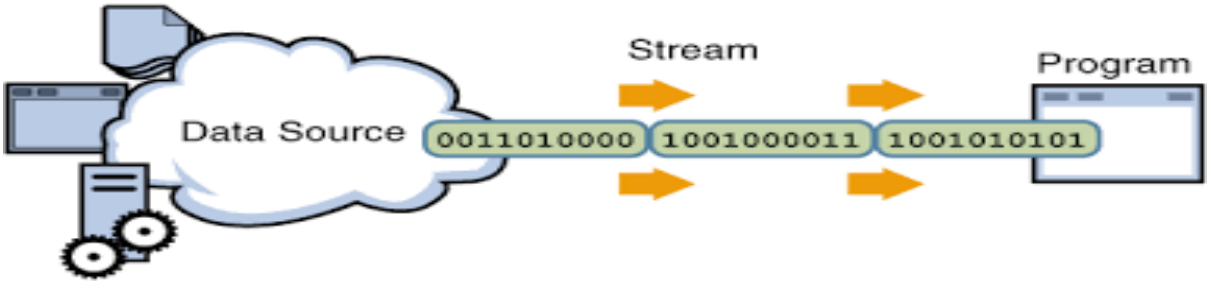
54. I/O Basics

Java I/O (Input and Output) is used to process the input and produce the output. Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.



55. Streams

Stream - A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

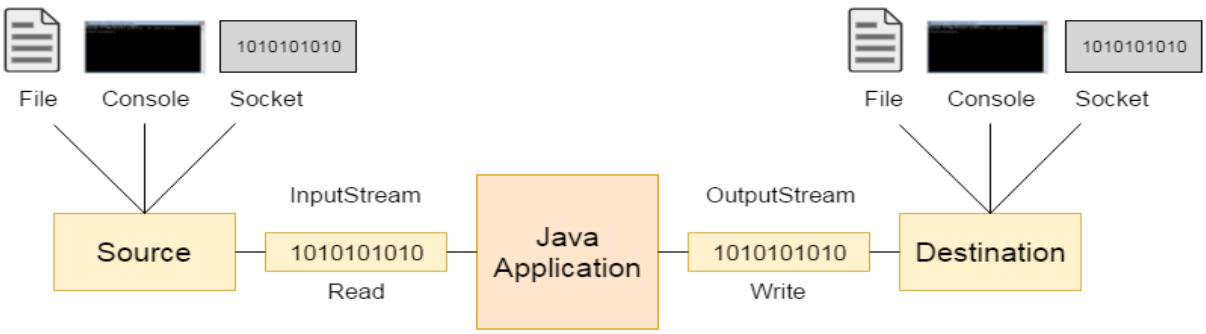


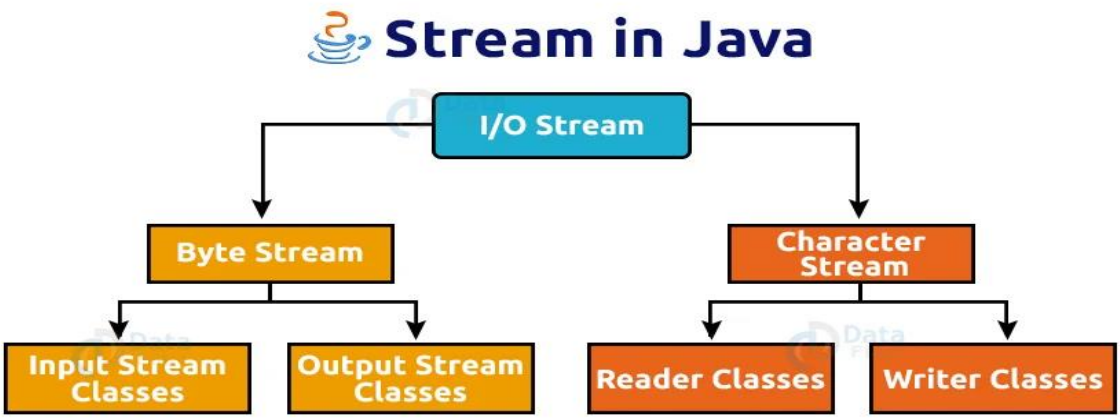
Output Stream

Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.

Input Stream

Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket. Let's understand the working of Java OutputStream and InputStream by the figure given below.





56. Byte streams and Character streams

Byte stream: (Reads data from any source as byte at a time(images, text, video, audio file)

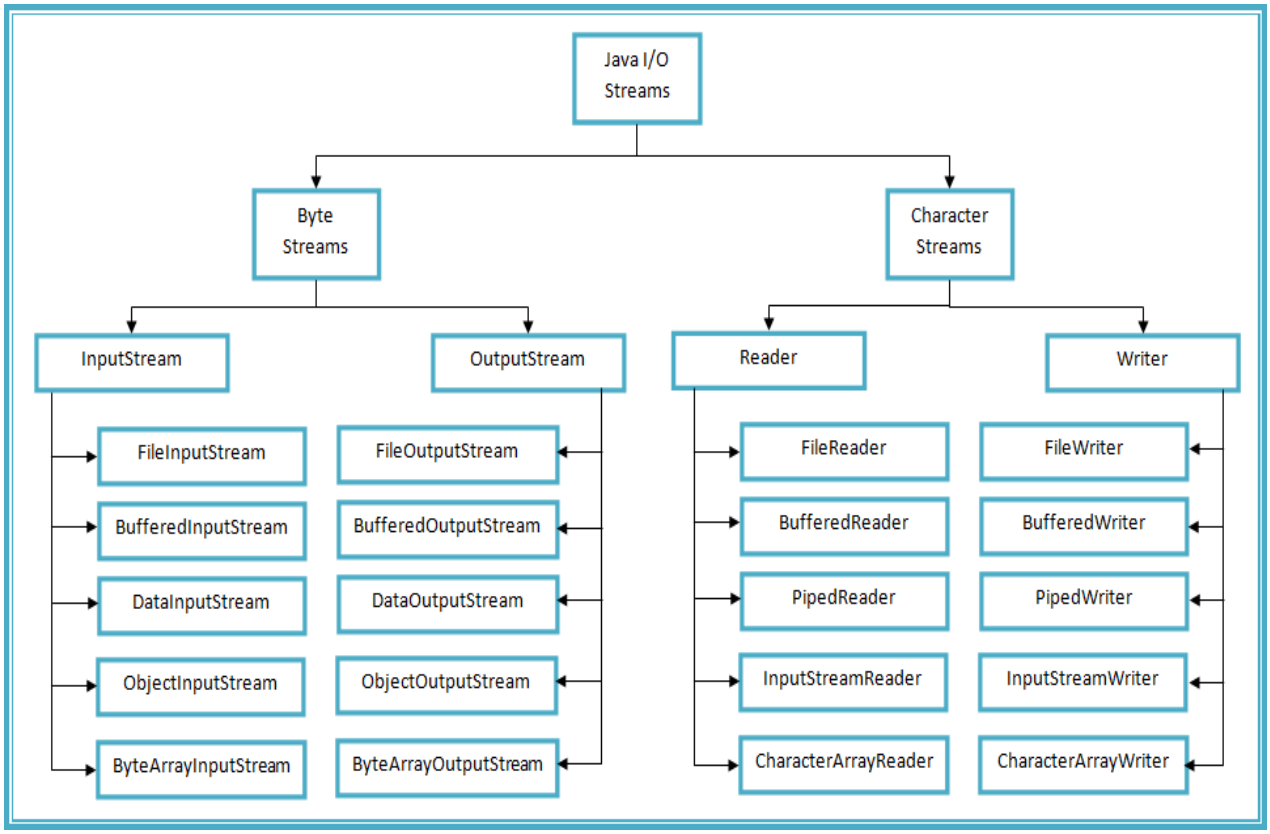
Input Stream: reads data from the source.

Output Stream: writes data to a destination.

Character stream: Reads data from any source as character at a time (one character at a time)

Reader class: reads data from the source

Writer class: writes data to a destination



InputStream class

Input Stream class is an abstract class. It is the superclass of all classes representing an input stream of bytes.

Useful methods of InputStream

Method	Description
1) public abstract int read()throws IOException	reads the next byte of data from the input stream. It returns -1 at the end of the file.
2) public int available()throws IOException	returns an estimate of the number of bytes that can be read from the current input stream.
3) public void close()throws IOException	is used to close the current input stream.

OutputStream class

An OutputStream is an abstract class in the java.io package that represents a stream of bytes used for writing data to an output destination, such as a file, network socket, or console. It provides methods to write bytes sequentially and includes functionalities for flushing and closing the stream.

Useful methods of OutputStream

Method	Description
1) public void write(int)throws IOException	is used to write a byte to the current output stream.
2) public void write(byte[])throws IOException	is used to write an array of byte to the current output stream.
3) public void flush()throws IOException	flushes the current output stream.
4) public void close()throws IOException	is used to close the current output stream.

57. Reading and Writing Console

Reading User Input from Console

In Java, there are four different ways to read input from the user in the command line environment(console).

1. Using Buffered Reader Class

Buffered Reader Class is the classical method to take input, Introduced in JDK 1.0. This method is used by wrapping the System.in (standard input stream) in an InputStreamReader which is wrapped in a BufferedReader, we can read input from

the user in the command line. The input is buffered for efficient reading. The wrapping code is hard to remember.

Example:

```
// Java program to demonstrate BufferedReader
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

public class Geeks {

    public static void main(String[] args)
        throws IOException
    {
        // Enter data using BufferReader
        BufferedReader r = new BufferedReader(
            new InputStreamReader(System.in));

        // Reading data using readLine
        String s = r.readLine();

        // Printing the read line
        System.out.println(s);
    }
}
```

2. Using Scanner Class

Scanner Class is probably the most preferred method to take input, Introduced in JDK 1.5. The main purpose of the Scanner class is to parse primitive types and strings using regular expressions; however, it is also can be used to read input from the user in the command line.

Example

```
// Java program to demonstrate working of Scanner Class
import java.util.Scanner;

class Geeks {
    public static void main(String args[])
    {
        // Using Scanner for Getting Input from User
        Scanner s = new Scanner(System.in);

        String s1 = s.nextLine();
        System.out.println("You entered string " + s1);

        int a = s.nextInt();
        System.out.println("You entered integer " + a);
```

```
        float b = s.nextFloat();
        System.out.println("You entered float " + b);
    }
}
```

3) Using Console Class

Console Class has been becoming a preferred way for reading user's input from the command line, Introduced in JDK 1.6. In addition, it can be used for reading password-like input without echoing the characters entered by the user; the format string syntax can also be used (like System.out.printf()).

Example:

```
// Java program to demonstrate working of System.console()
// Note that this program does not work on IDEs as
// System.console() may require console
import java.util.Scanner;

public class Geeks {
    public static void main(String[] args)
    {
        // Using Scanner for Getting Input from User
        Scanner scanner = new Scanner(System.in);

        System.out.println("Enter a string:");
        String s = scanner.nextLine();

        System.out.println("You entered string " + s);
    }
}
```

4. Using Command line argument

Command line argument has been around since JDK 1.0 and are used in competitive coding. These inputs are passed to the program during execution and are stored as strings in the args[] array. If we need numeric values, we can convert the strings using methods like Integer.parseInt() or Float.parseFloat(). These programs are run from the command line, with inputs provided while executing the program.

Example:

```
// Java Program to check for command line arguments

class Geeks {

    public static void main(String[] args)

    {
```

```
        // check if length of args array is
        // greater than 0
        if (args.length > 0) {
            System.out.println( "The command line arguments are:");
            // iterating the args array and printing
            // the command line arguments
            for (String val : args)
                System.out.println(val);
        }
        else
            System.out.println("No command line   + "arguments found.");
    }
}
```

Writing Console Output in Java

Writing Console Output in Java refers to writing the output of a Java program to the console or any particular file.The methods used for streaming output are defined in the PrintStream class. The methods used for writing console output are print(), println() and write(). Both print() and println() methods are used to direct the output to the console. These methods are defined in the PrintStream class and are widely used.

Both these methods are used with the help of the System.out stream.

- The basic differences between print() and println() methods are as follows:
- **print**() method displays the string in the same line whereas **println**() method outputs a newline character after its execution.
- **print**() method is used for directing output to console only whereas **println**() method is used for directing output to not only console but other sources also.

Example of print() method

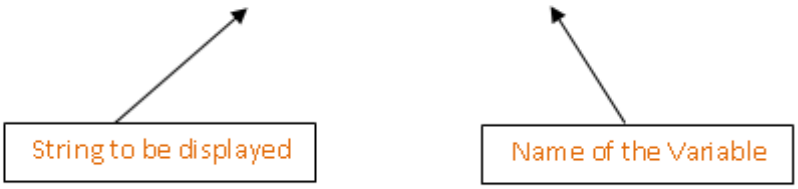
```
class printEg
{
        public static void main(String args[])
        {
                int a;
                for (a=1 ;  a<=10 ;  a++)
                {
                        System.out.print(a);
                }
        } }
```

Example of println() method:

```
class printlnEg
{
        public static void main(String args[])
        {
                int a;
                for (a=1 ; a<=10 ; a++)
                {
                        System.out.println(a);
                }
        }
}
```

In addition, if you want to display the value of any particular variable used in the program, then you have to append the variable name along with the string with a plus (+) symbol.

```
System.out.println("Name entered by you is: " + name);
```



The write() method

- Alternatively, you can make use of the write() method for directing the output of your program to the console.
- The easiest ***syntax*** of the write() method is: **void write(int b);**

Example:

```
class writeEg
{
        public static void main(String args[])
        {
                int a, b;
                a = 'Q';
                b = 65;
                System.out.write(a);
                System.out.write('\n');
                System.out.write(b);
                System.out.write('\n');
        }
}
```

58. The Print Writer Class

Java PrintWriter class is the implementation of Writer class. It is used to print the formatted representation of objects to the text-output stream.

Class declaration

public class PrintWriter extends Writer

Method	Description
void println(boolean x)	It is used to print the boolean value.
void println(char[] x)	It is used to print an array of characters.
void println(int x)	It is used to print an integer.
PrintWriter append(char c)	It is used to append the specified character to the writer.
PrintWriter append(CharSequence ch)	It is used to append the specified character sequence to the writer.
PrintWriter append(CharSequence ch, int start, int end)	It is used to append a subsequence of specified character to the writer.

Example:

```
import java.io.PrintWriter;
class Main {
  public static void main(String[] args) {
  String data = "This is a text inside the file.";
  try
{
     PrintWriter output = new PrintWriter("output.txt");
     output.print(data);
     output.close();
    }
    catch(Exception e) {
     e.getStackTrace();
    }
  }
}
```

59. Reading and Writing Files

Reader, InputStreamReader, FileReader and BufferedReader

1. Reader is the abstract class for reading character streams. It implements the following fundamental methods:

read(): reads a single character.

read(char[]): reads an array of characters.

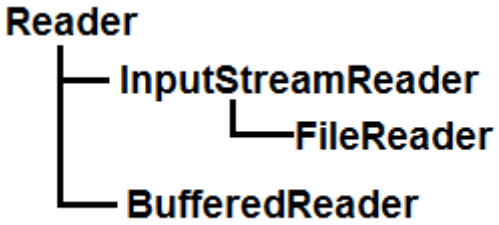
skip(long): skips some characters.

close(): closes the stream.

InputStreamReader is a bridge from byte streams to character streams. It converts bytes into characters using a specified charset. The charset can be default character encoding of the operating system, or can be specified explicitly when creating an InputStreamReader.

FileReader is a convenient class for reading text files using the default character encoding of the operating system.

BufferedReader reads text from a character stream with efficiency (characters are buffered to avoid frequently reading from the underlying stream) and provides a convenient method for reading a line of text readLine().



Writer, OutputStreamWriter, FileWriter and BufferedWriter

Writer is the abstract class for writing character streams. It implements the following fundamental methods:

write(int): writes a single character.

write(char[]): writes an array of characters.

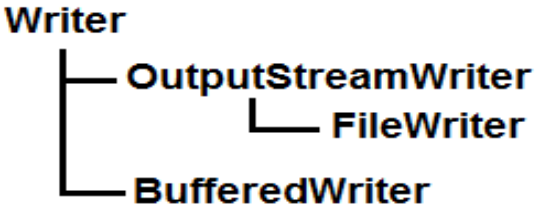
write(String): writes a string.

close(): closes the stream.

OutputStreamWriter is a bridge from byte streams to character streams. Characters are encoded into bytes using a specified charset. The charset can be default character encoding of the operating system, or can be specified explicitly when creating an OutputStreamWriter.

FileWriter is a convenient class for writing text files using the default character encoding of the operating system.

BufferedWriter writes text to a character stream with efficiency (characters, arrays and strings are buffered to avoid frequently writing to the underlying stream) and provides a convenient method for writing a line separator: newLine().



Character Encoding and Charset

When constructing a reader or writer object, the default character encoding of the operating system is used (e.g. Cp1252 on Windows):

```
FileReader reader = new FileReader("MyFile.txt");
FileWriter writer = new FileWriter("YourFile.txt");
```

So if we want to use a specific charset, use an InputStreamReader or OutputStreamWriter instead.

```
InputStreamReader reader = new InputStreamReader(new FileInputStream("MyFile.txt"), "UTF-16");
```

Java Reading from Text File Example

```
import java.io.FileReader;
import java.io.IOException;
public class TextFileReadingExample1 {

    public static void main(String[] args) {
        try {
            FileReader reader = new FileReader("MyFile.txt");
            int character;

            while ((character = reader.read()) != -1) {
                System.out.print((char) character);
            }
            reader.close();

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Java Writing to Text File Example

```
package net.codejava.io;
 import java.io.FileWriter;
import java.io.IOException;

public class TextFileWritingExample1 {

    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("MyFile.txt", true);
            writer.write("Hello World");
            writer.write("\r\n");   // write new line
            writer.write("Good Bye!");
```

```
        writer.close();
      } catch (IOException e) {
        e.printStackTrace();
      }

    }

}
```

60. Scanner Class.

The Scanner class serves as the counterpart to Formatter, allowing formatted input to be read and converted into its binary representation. It can read data from various sources, including the console, files, strings, or any object implementing the Readable interface or ReadableByteChannel. For instance, Scanner can be used to read a number from the keyboard and store it in a variable. Despite its versatility, Scanner remains simple and easy to use.

Input Types

In the example above, we used the nextLine() method, which is used to read Strings. To read other types, look at the table below:

Method	Description
nextBoolean()	Reads a boolean value from the user
nextByte()	Reads a byte value from the user
nextDouble()	Reads a double value from the user
nextFloat()	Reads a float value from the user
nextInt()	Reads a int value from the user
nextLine()	Reads a String value from the user
nextLong()	Reads a long value from the user
nextShort()	Reads a short value from the user

Example:

```
import java.util.Scanner;  // Import the Scanner class

class Main
{
  public static void main(String[] args)
{
    Scanner myObj = new Scanner(System.in);  // Create a Scanner object
    System.out.println("Enter username");

    String userName = myObj.nextLine();  // Read user input
    System.out.println("Username is: " + userName);  // Output user input
  } }
```

UNIT – V-EVENT DRIVEN PROGRAMMING AND NETWORK PROGRAMMING

61. Event Driven Programming: Event Listeners

What is an Event?

- Change in the state of an object is known as event i.e. event describes the change in state of source. Events are generated as result of user interaction with the graphical user interface components.
- For example, clicking on a button, moving the mouse, entering a character through keyboard selecting an item from list, scrolling the page are the activities that causes an event to happen.

What is Event Handling?

- Event Handling is the mechanism that controls the event and decides what should happen if an event occurs.
- This mechanism have the code which is known as event handler that is executed when an event occurs. Java Uses the Delegation Event Model to handle the events.
- This model defines the standard mechanism to generate and handle the events.

Event handling

Any change in the state of any object is called event.
For Example: Pressing a button, entering a character in Textbox, Clicking or dragging a mouse, etc.
The three main components in event handling are:

Events
Events Source
Listeners

- Events: An event is a change in state of an object. For example, mouseClicked,mousePressed.
- Events Source: The source is an object on which event occurs. Source is responsible for providing information of the occurred event to it's handler. Java provide as with classes for source object.
- Listeners: A listener is an object that listens to the event. A listener gets notified when an event occurs.
- When listener receives an event, it process it and then return.
    - Listeners are group of interfaces and are defined in java.awt.event package. Each component has its own listener. For example MouseListener handles allMouseEvent

Event Classes	Generated when	Listener Interfaces
ActionEvent	button is pressed, menu-item is selected,list-item is double clicked	ActionListener
MouseEvent	mouse is dragged, moved, clicked, pressed or released and also whenit enters or exit a component	MouseListener and MouseMotionListener
MouseWheelEvent	mouse wheel is moved	MouseWheelListener
KeyEvent	input is received from keyboard	KeyListener
ItemEvent	check-box or list item is clicked	ItemListener
TextEvent	value of textarea or textfield is changed	TextListener
WindowEvent	window is activated, deactivated, deiconified, iconified, opened or closed	WindowListener
ComponentEvent	component is hidden, moved, resized or set visible	ComponentListener

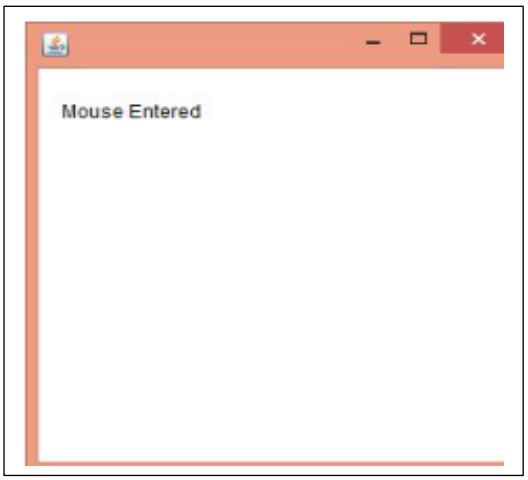
62. Delegation event model: Handling mouse

The MouseListener interface is part of the 'java.awt.event' package. It is used to retrieve and respond to mouse-related events in Java applications. Mouse clicks, mouse button presses and releases, mouse enter and exit events are examples of these events. By implementing the MouseListener interface, you can define specific actions that your application has to perform in response to the interactions with the mouse.

- The Java MouseListener is notified whenever you change the state of mouse. It is notified against MouseEvent. The MouseListener interface is found in java.awt.event package. It has five methods.
  - **public abstract void** mouseClicked(MouseEvent e);
  - **public abstract void** mouseEntered(MouseEvent e);
  - **public abstract void** mouseExited(MouseEvent e);
  - **public abstract void** mousePressed(MouseEvent e);
  - **public abstract void** mouseReleased(MouseEvent e);

Example:

```
import java.awt.*;
import java.awt.event.*;
public class MouseListenerExample extends Frame implements MouseListener{
   Label l;
   MouseListenerExample(){
      addMouseListener(this);
      l=new Label();
      l.setBounds(20,50,100,20);
      add(l);
      setSize(300,300);
      setLayout(null);
      setVisible(true);
   }
   public void mouseClicked(MouseEvent e) {
      l.setText("Mouse Clicked");
   }
   public void mouseEntered(MouseEvent e) {
      l.setText("Mouse Entered");
   }
   public void mouseExited(MouseEvent e) {
      l.setText("Mouse Exited");
   }
   public void mousePressed(MouseEvent e) {
      l.setText("Mouse Pressed");
   }
   public void mouseReleased(MouseEvent e) {
      l.setText("Mouse Released");
   }
public static void main(String[] args) {
   new MouseListenerExample();
}
}
```



63. keyboard events

The Java KeyListener is notified whenever you change the state of key. It is notified against KeyEvent. The KeyListener interface is found in java.awt.event package,Methods of KeyListener interface

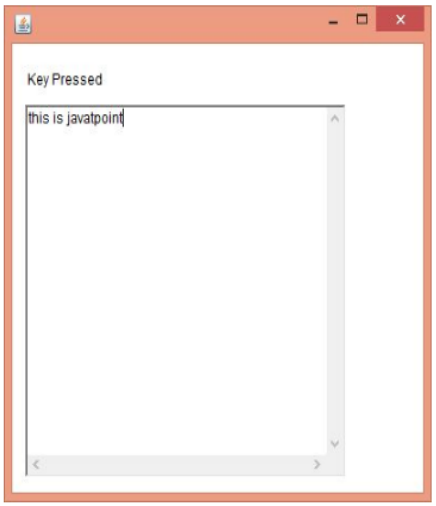
The signature of 3 methods found in KeyListener interface are given below:

Sr. no.	Method name	Description
1.	public abstract void keyPressed (KeyEvent e);	It is invoked when a key has been pressed.
2.	public abstract void keyReleased (KeyEvent e);	It is invoked when a key has been released.
3.	public abstract void keyTyped (KeyEvent e);	It is invoked when a key has been typed.

Example:

```
import java.awt.*;
import java.awt.event.*;
public class KeyListenerExample extends Frame implements KeyListener {
 Label l;
   TextArea area;
   KeyListenerExample()
{
        l = new Label();
      l.setBounds (20, 50, 100, 20);
      area = new TextArea();
      area.setBounds (20, 80, 300, 300);
      area.addKeyListener(this);
      add(l);
add(area);
setSize (400, 400);
      setLayout (null);
      setVisible (true);
   }
   public void keyPressed (KeyEvent e)
 {
      l.setText ("Key Pressed");
   }
   public void keyReleased (KeyEvent e)
{
      l.setText ("Key Released");
   }

   public void keyTyped (KeyEvent e)
{
      l.setText ("Key Typed");
   }
   public static void main(String[] args)
{
      new KeyListenerExample();
   }
}
```

Output:



64. Adapter classes

Java offers a helpful feature known as an adapter class, which can make event handling easier incertain cases. An adapter class provides default (empty) implementations for all the methods in a specific event listener interface. This is particularly useful when you only need to handle a few events from a listener interface, rather than implementing every method it requires.To use this feature, you can create a new class that extends an appropriate adapter class and override only the methods relevant to your needs.

For instance, the MouseMotionAdapter class includes two methods: mouseDragged() and mouseMoved(), which come from the MouseMotionListener interface. If you're only interested in handling drag events, you can extend MouseMotionAdapter and override just the mouseDragged() method. The default implementation of mouseMoved() will take care of the rest without requiring additional code.

Adapter Class	Listener Inte
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener and (as of JDK 6) Mouse MouseWheelListener
MouseMotionAdapter	MouseMotionListener
WindowAdapter	WindowListener, WindowFocusListener,

Example:

```
import java.awt.*;
import java.awt.event.*;
class WindowExample extends Frame {
  WindowExample() {
    addWindowListener(new WindowAdapter(){
      public void windowClosing(WindowEvent e) {
        dispose();
      }
    });
   setSize(400,400);
   setLayout(null);
   setVisible(true);
  }
  public static void main(String[] args) {
    new WindowExample();
  } }
```

65. Swing: Introduction

Java Swing tutorial is a part of Java Foundation Classes (JFC) that is used to create window-based applications. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in java.Unlike AWT, Java Swing provides platform-independent and lightweight components.The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc

Swing is a Java Foundation Classes [JFC] library and an extension of the Abstract Window Toolkit [AWT]. Java Swing offers much-improved functionality over AWT, new components, expanded components features, and excellent event handling with drag-and-drop support

Introduction of Java Swing

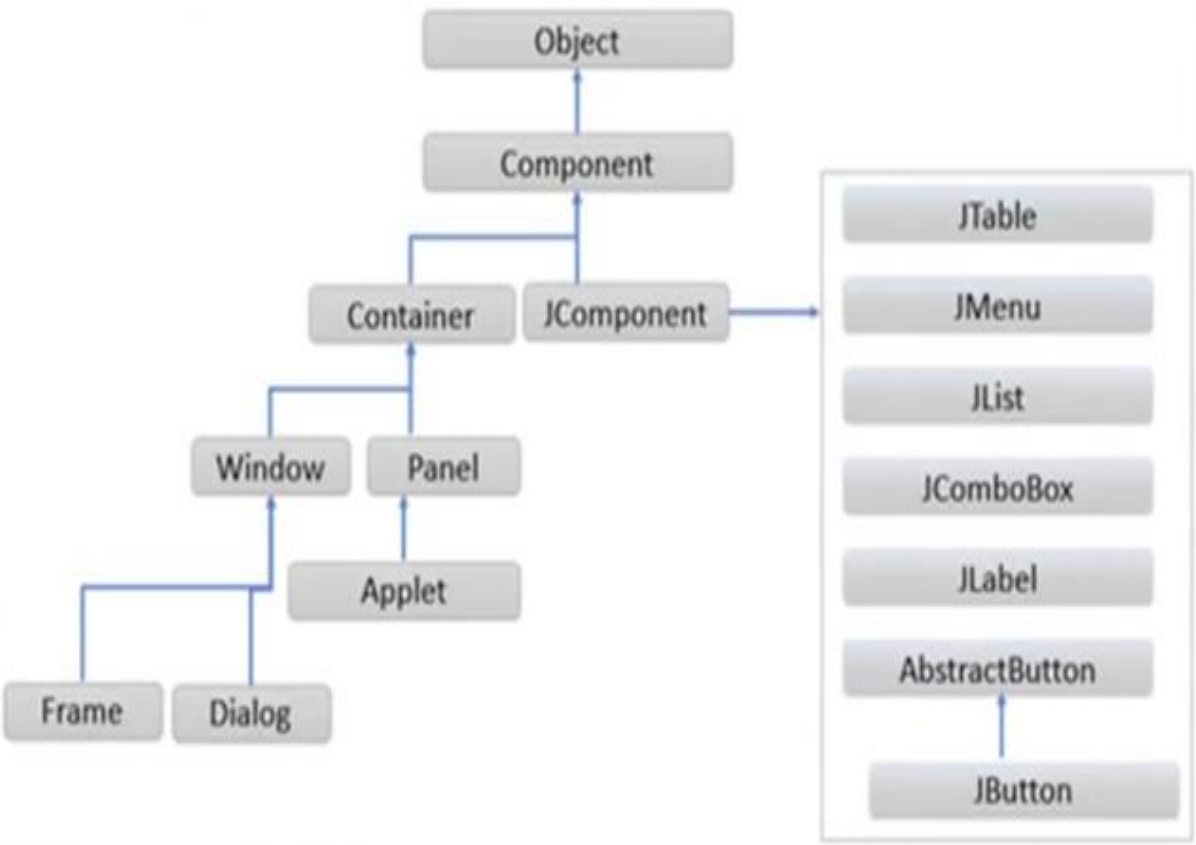
Swing has about four times the number of User Interface [UI] components as AWT and is part of the standard Java distribution. By today's application GUI requirements, AWT is a limited implementation, not quite capable of providing the components required for developing complex GUIs required in modern commercial applications. The AWT component set has quite a few bugs and does take up a lot of system resources when compared to equivalent Swing resources. Netscape introduced its Internet Foundation Classes [IFC] library for use with Java. Its Classes became very popular with programmers creating GUI's for commercial applications.

- Swing is a Set of API (API- Set of Classes and Interfaces)Swing is Provided to Design Graphical User Interfaces
- Swing is an Extension library to the AWT (Abstract Window Toolkit)
- Includes New and improved Components that have been enhancing the looks and Functionality of GUIs'
- Swing can be used to build (Develop) The Standalone swing GUI Apps as Servlets and Applets
- It Employs model/view design architecture.
- Swing is more portable and more flexible than AWT, the Swing is built on top of the AWT.
- Swing is Entirely written in Java.
- Java Swing Components are Platform-independent, and The Swing Components are lightweight.
- Swing Supports a Pluggable look and feel and Swing provides more powerful components.
- such as tables, lists, Scrollpanes, Colourchooser, tabbed pane, etc.
- Further Swing Follows MVC.

No.	Java AWT	Java Swing
1)	AWT components are **platform-dependent**.	Java swing components are **platform-independent**.
2)	AWT components are **heavyweight**.	Swing components are **lightweight**.
3)	AWT **doesn't support pluggable look and feel**.	Swing **supports pluggable look and feel**.
4)	AWT provides **less components** than Swing.	Swing provides **more powerful components** such as tables, lists, scrollpanes, colorchooser, tabbedpane etc.
5)	AWT **doesn't follows MVC**(Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing **follows MVC**.

Hierarchy of Java Swing classes

The Java Swing class hierarchy builds upon the Abstract Window Toolkit (AWT) and extends it with a rich set of lightweight, platform-independent GUI components. The fundamental structure of this hierarchy is as follows



66. MVC Architecture

The Model-View-Controller (MVC) is a well-known design pattern in the web development field. It is way to organize our code. It specifies that a program or application shall consist of data model, presentation information and control information. The MVC pattern needs all these components to be separated as different objects.

What is MVC architecture in Java?

The model designs based on the MVC architecture follow MVC design pattern. The application logic is separated from the user interface while designing the software using model designs.

Model: It represents the business layer of application. It is an object to carry the data that can also contain the logic to update controller if data is changed.

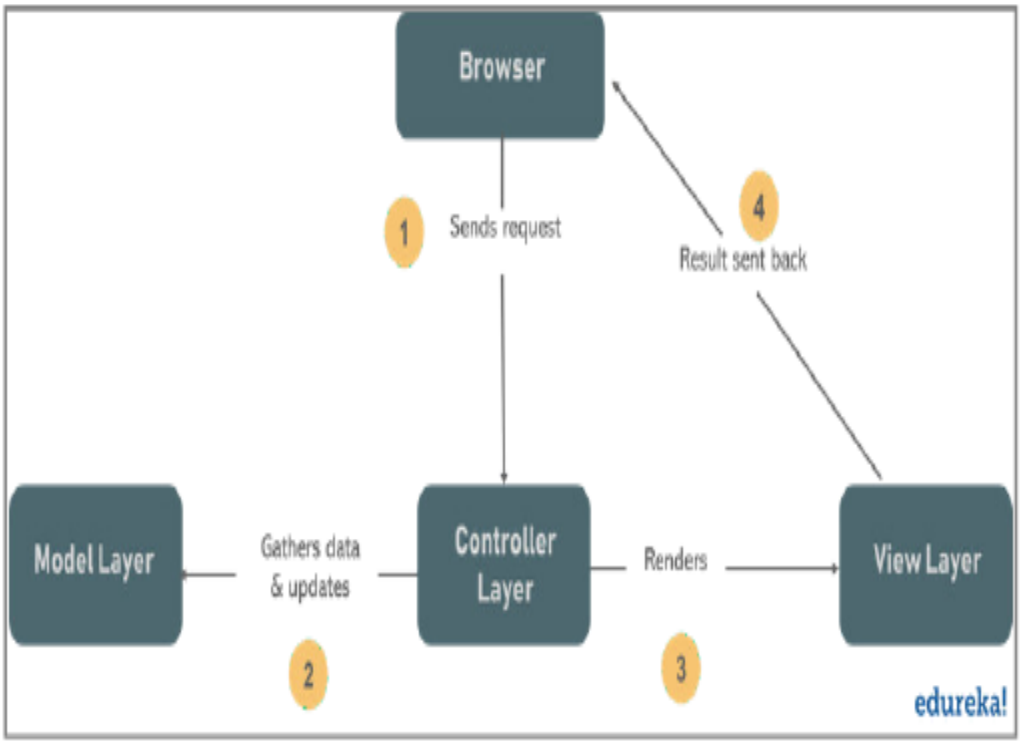
View: It represents the presentation layer of application. It is used to visualize the data that the model contains.

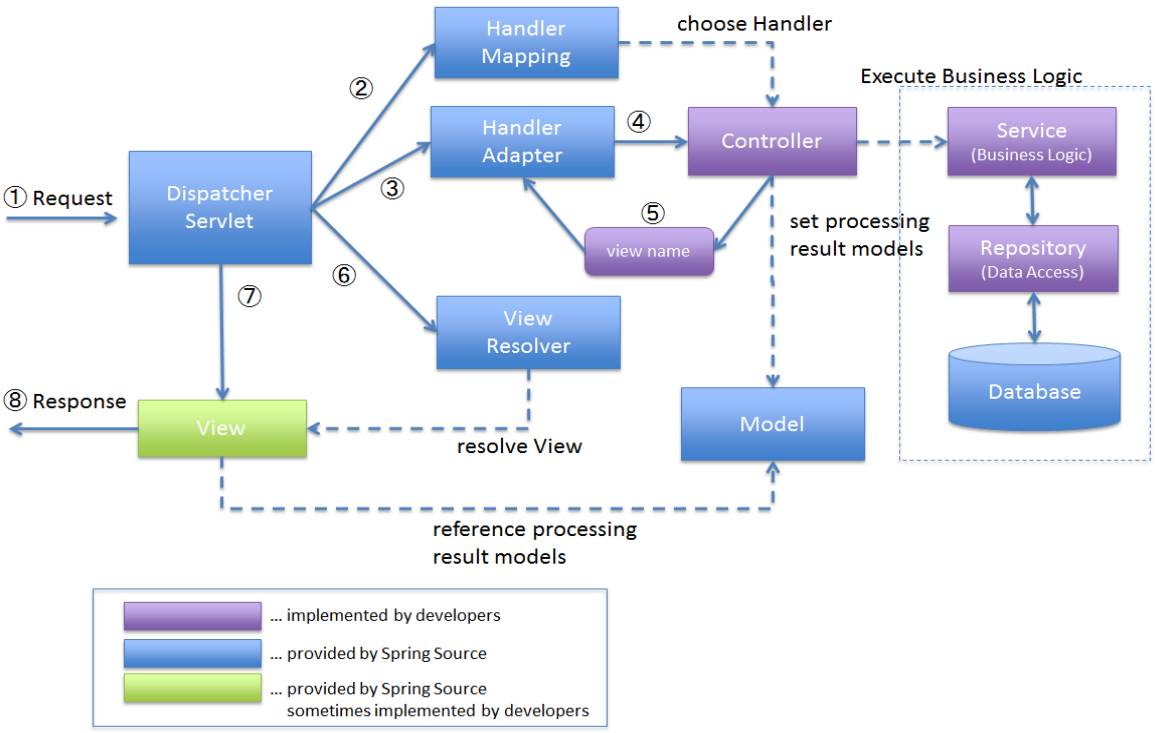
Controller: It works on both the model and view. It is used to manage the flow of application, i.e. data flow in the model object and to update the view whenever data is changed.

In Java Programming, the Model contains the simple Java classes, the View used to display the data and the Controller contains the servlets. Due to this separation the user requests are processed as follows:

Model designs based on MVC architecture **follow the MVC design pattern** and they separate the application logic from the user interface when designing software. As the name implies MVC pattern has three layers, which are:

- **Model –** Represents the business layer of the application
- **View –** Defines the presentation of the application
- **Controller –** Manages the flow of the application





A client (browser) sends a request to the controller on the server side, for a page.

The controller then calls the model. It gathers the requested data.

Then the controller transfers the data retrieved to the view layer.

Now the result is sent back to the browser (client) by the view.

Implementation of MVC using Java

To implement a web application based on MVC design pattern, we will create

- Employee Class, which acts as the model layer
- Employee View Class, which defines the presentation layer (view layer)
- Employee Controller Class, which acts as a controller

The Model Layer

In the MVC design pattern, the model is the data layer which defines the business logic of the system and also represents the state of the application.

The model objects retrieve and store the state of the model in a database Now, let's create a model using Employee Class.

Example:

```
public class Employee {

    // declaring the variables
     private String EmployeeName;
     private String EmployeeId;
     private String EmployeeDepartment;

    // defining getter and setter methods
     public String getId() {
       return EmployeeId;
     }

     public void setId(String id) {
```

```
        this.EmployeeId = id;
      }
 public String getName() {
        return EmployeeName;
      }

      public void setName(String name) {
        this.EmployeeName = name;
      }

      public String getDepartment() {
            return EmployeeDepartment;
          }

      public void setDepartment(String Department) {
            this.EmployeeDepartment = Department;
          }

    }
```

The View Layer

Example:

```
// class which represents the view
public class EmployeeView {

     // method to display the Employee details
public void printEmployeeDetails (String EmployeeName, String EmployeeId, String Emplo
yeeDepartment)
{
        System.out.println("Employee Details: ");
        System.out.println("Name: " + EmployeeName);
        System.out.println("Employee ID: " + EmployeeId);
        System.out.println("Employee Department: " + EmployeeDepartment);
      }
    }
public class EmployeeController {

      private Employee model;
      private EmployeeView view;

     // constructor to initialize
public EmployeeController(Employee model, EmployeeView view) {
        this.model = model;
        this.view = view;
```

```
    }

   // getter and setter methods
    public void setEmployeeName(String name){
      model.setName(name);
    }
      public String getEmployeeName(){
      return model.getName();
    }
     }
```

The Controller Layer

Example:

```
    public void setEmployeeId(String id){
      model.setId(id);
    }

    public String getEmployeeId(){
      return model.getId();
    }

public void setEmployeeDepartment(String Department)
{
          model.setDepartment(Department);
    }

       public String getEmployeeDepartment(){
          return model.getDepartment();
    }

    // method to update view
public void updateView()
{
view.printEmployeeDetails(model.getName(), model.getId(), model.getDepartment());
  }
```

Main Class Java file

Example:

```
public class MVCMain {
public static void main(String[] args) {
  // fetching the employee record based on the employee_id from the database
      Employee model = retriveEmployeeFromDatabase();
// creating a view to write Employee details on console
      EmployeeView view = new EmployeeView();

      EmployeeController controller = new EmployeeController(model, view);

      controller.updateView();

      //updating the model data
      controller.setEmployeeName("Nirnay");
```

```
    System.out.println("\n Employee Details after updating: ");

    controller.updateView();
  }
   private static Employee retriveEmployeeFromDatabase(){
    Employee Employee = new Employee();
    Employee.setName("Anu");
    Employee.setId("11");
    Employee.setDepartment("Salesforce");
    return Employee;
  }
}
```

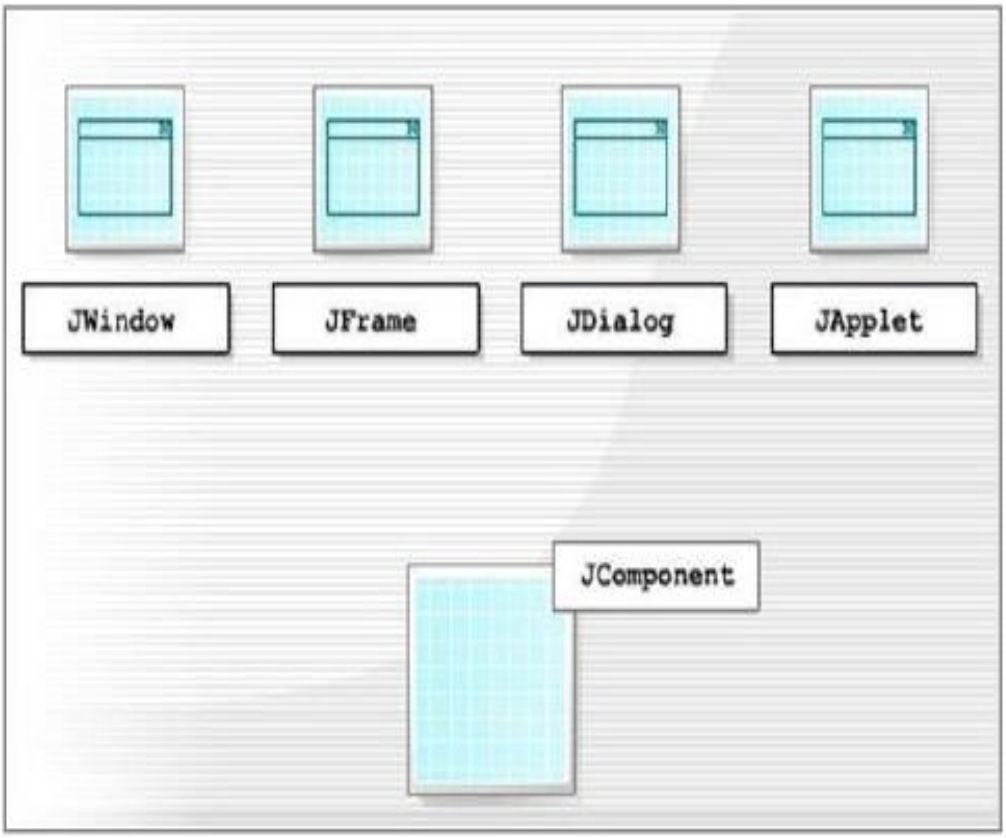
67. Components and containers

In Java, a component is the basic user interface object and is found in all Java applications. Components include lists, buttons, panels, and windows.

Swing provides the following useful top-level containers, all of which inherit from JComponent:

JWindow

- JWindow is a top-level window that doesn't have any trimmings and can be displayed anywhere on a desktop.
- JWindow is a heavyweight component. You usually use JWindow to create pop-up windows and "splash" screens. JWindow extends AWT's Window class.



JFrame

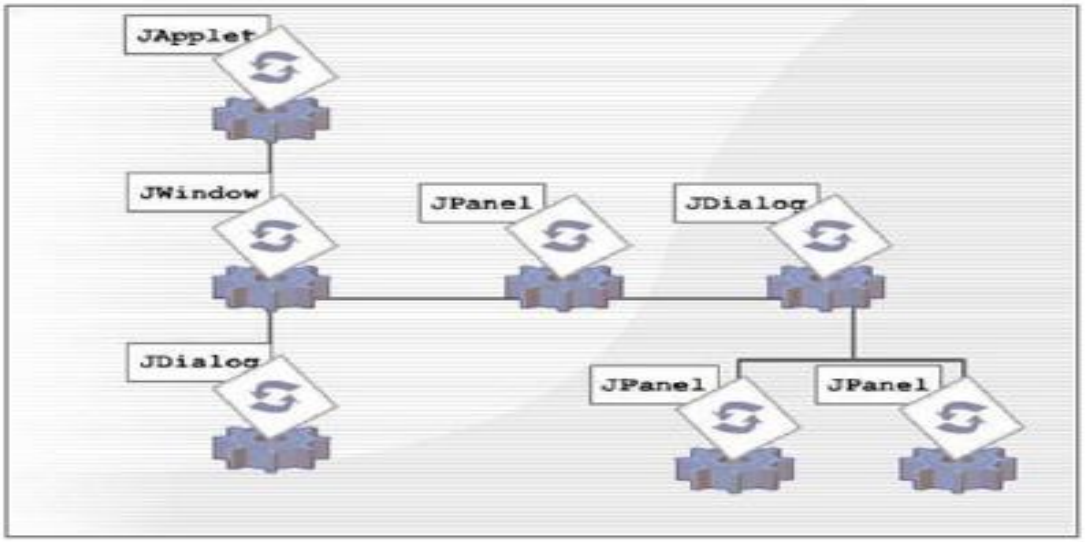
- JFrame is a top-level window that can contain borders and menu bars. JFrame is a subclass of JWindow and is thus a heavyweight component.

JDialog

- JDialog is a lightweight component that you use to create dialog windows. You can place dialog windows on a JFrame or JApplet. JDialog extends AWT's Dialog class.

JApplet

- JApplet is a container that provides the basis for applets that run within web browsers. JApplet is a lightweight component that can contain other graphical user interface (GUI) components. JApplet extends AWT's Applet class.
- All Swing components - including the JApplet and JDialog containers - need to be contained at some level inside a JWindow or JFrame.

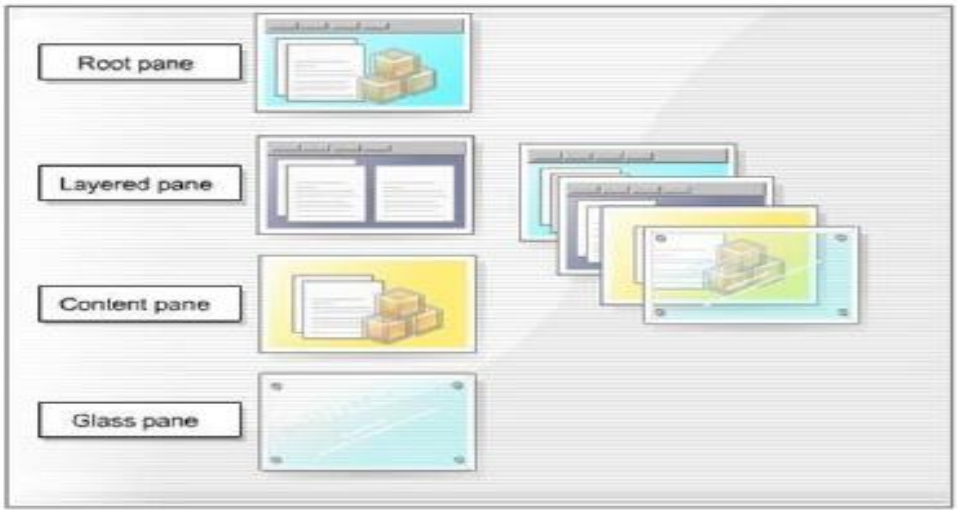


Layered pane

- The layered pane contains the content pane and the optional menu bar. It can also contain other components, which **it arranges so that they overlap each other.**

Content pane

- It covers the visible section of the JFrame or JWindow and you use it to add components to the display area..



68. Exploring Swing Components: JLabel and Image icon

The JLabel class is used to present text, an image, or a combination of both in a GUI. It is a non-interactive component, meaning it does not respond to user actions or inputs.

JLabel provides multiple constructors for different use cases. Some commonly used ones include:

- JLabel(Icon icon) – Displays an icon.
- JLabel(String text) – Displays a text label.
- JLabel(String text, Icon icon, int alignment) – Displays both text and icon, with control over alignment

Example:

```
import javax.swing.*;
public class JLabelWithImage
{
public static void main(String[] args)
 {
// Create a JFrame window
JFrame frame = new JFrame("Example for JLabel and ImageIcon ");
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setSize(400, 400);
frame.setLayout(null); // We use null layout for manual positioning
// Load an image using ImageIcon (make sure the file exists)
ImageIcon icon = new ImageIcon("logo.jpg"); // Use your image file path here
// Create a JLabel with text and image
JLabel label = new JLabel("2024-2028 CSE -2yr-Sec-C");
label.setIcon(icon);
label.setBounds(50, 50, 300, 300); // Position and size
// Text below the image
label.setHorizontalTextPosition(JLabel.CENTER);
label.setVerticalTextPosition(JLabel.BOTTOM);
// Styling
label.setIconTextGap(10); // Gap between icon and text
label.setHorizontalAlignment(JLabel.CENTER);
label.setVerticalAlignment(JLabel.CENTER);
// Add label to frame
frame.add(label);
// Show the frame
frame.setVisible(true);
}}
```

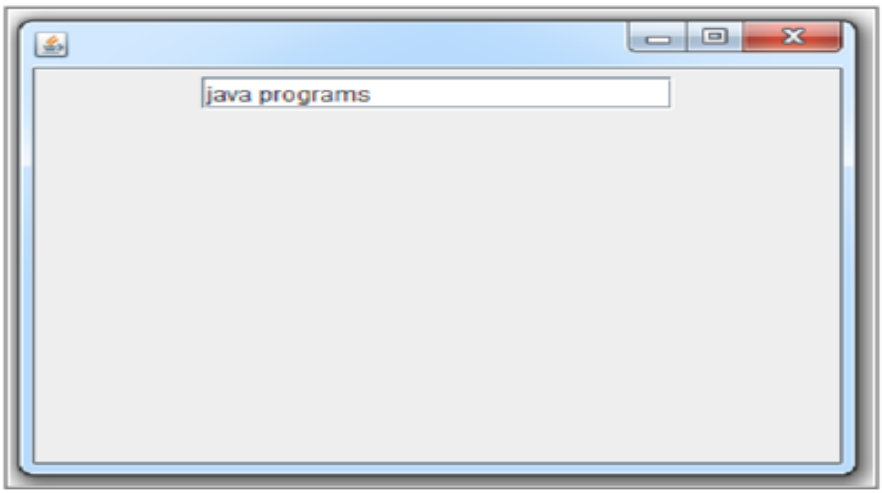
69. JTextField

JTextField is a part of javax.swing package. The class JTextField is a component that allows editing of a single line of text. JTextField inherits the JTextComponent class and uses the interface SwingConstants. The constructor of the class are :

1. **JTextField() :** constructor that creates a new TextField
2. **JTextField(int columns) :** constructor that creates a new empty TextField with specified number of columns.
3. **JTextField(String text) :** constructor that creates a new empty text field initialized with the given string.
4. **JTextField(String text, int columns) :** constructor that creates a new empty textField with the given string and a specified number of columns .
5. **JTextField(Document doc, String text, int columns) :** constructor that creates a textfield that uses the given text storage model and the given number of columns.

Example:

```
import java.awt.event.*;
import javax.swing.*;
public class TextFieldE1 extends JFrame
{
public TextFieldE1()
{
JTextField jtf =newJTextField(20); //creating
JTextField.
add(jtf); //adding JTextField to frame.
setLayout(new FlowLayout());
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setSize(400,400);
setVisible(true);
}
public static void main(String[] args)
{
new TextFieldE1();
}
}
```

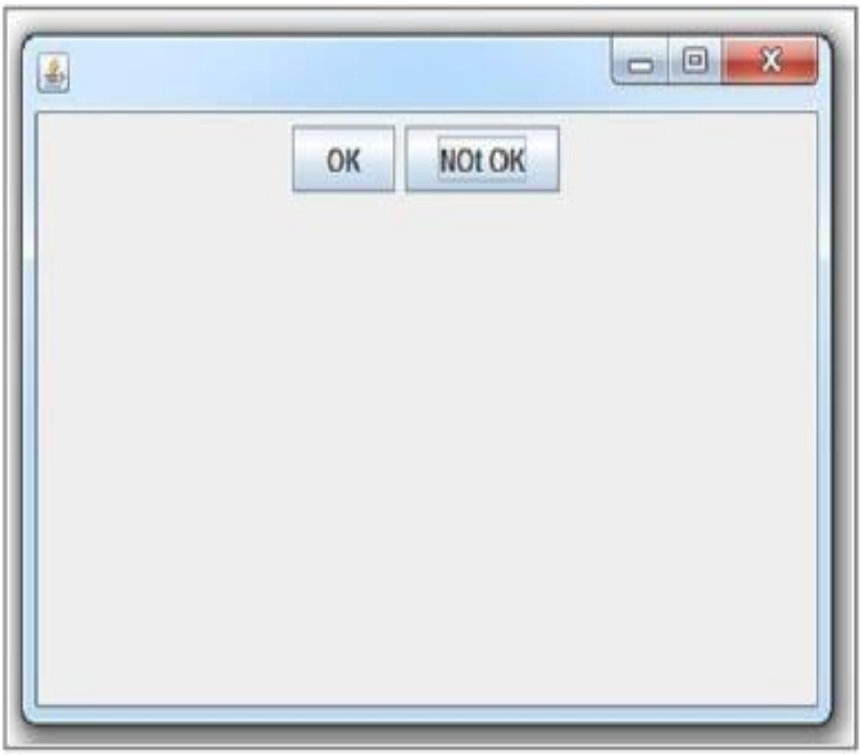


70. The swing Buttons:

JButton class provides functionality of a button. JButton class has three constructors. S JButton(Iconic) S JButton(Stringstr) S JButton(String str, Iconic) It allows a button to be created using icon, a string or both. JButton supports ActionEvent. When a button is pressed an ActionEvent is generated**.**

Example:

```
import javax.swing.*;
import java.awt.event.*;
public class testbutton extends JFrame
        {
testbutton()
{
JButton bt1 = new JButton("OK");
JButton bt2 = new JButton("NOt OK");
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); //setting close operation.
setLayout(newFlowLayout()); //setting layout using FlowLayout object
setSize(400,400); //setting size ofJframe
add(bt1); //adding Yes button toframe.
add(bt2); //adding No button toframe.
setVisible(true);
}
public static void main(String[] args)
{
new testbutton();
}
}
```

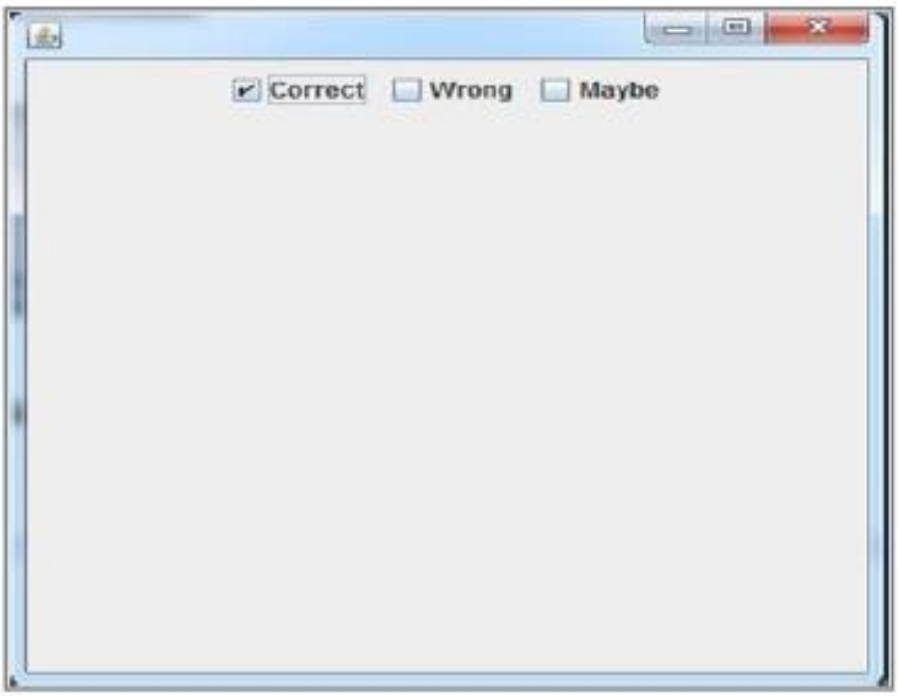


JCheckBox

- JCheckBox class is used to create checkboxes inframe.
- It is used to turn an option on (true) or off (false). Clicking on a CheckBox changes its state from "on" to "off" or from "off" to "on".
- It inherits JToggleButtonclass.

Example:

```
import javax.swing.*;
import java.awt.event.*;
import java.awt.*;
public class Test extends JFrame
{
public Test()
{{
JCheckBox jcb = new JCheckBox("Correct"); //creating JCheckBox.
add(jcb); //adding JCheckBox to frame.
jcb =newJCheckBox("Wrong"); //creating JCheckBox.
add(jcb); //adding JCheckBox toframe.
jcb =newJCheckBox("Maybe"); //creating JCheckBox.
add(jcb); //adding JCheckBox to frame.
setLayout(new FlowLayout());
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setSize(400, 400);
setVisible(true);
}
public static void main(String[] args) {
new Test();
 }
}
```

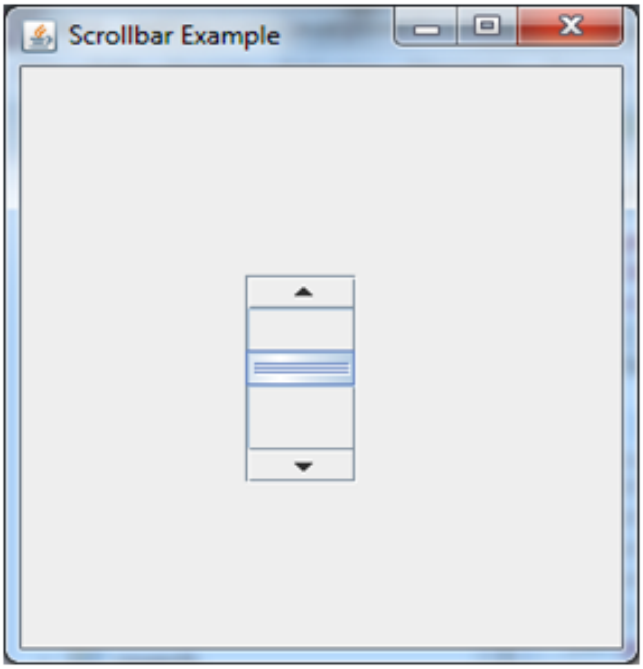


71. JScrollPane

The object of JScrollbar class is used to add horizontal and vertical scrollbar. It is an implementation of a scrollbar. It inherits JComponent class

Example:

```
import javax.swing.*;
class ScrollBarExample
{
ScrollBarExample(){
   JFrame f= new JFrame("Scrollbar Example");
 JScrollBar s=new JScrollBar();
s.setBounds(100,100, 50,100);
f.add(s);
f.setSize(400,400);
f.setLayout(null);
f.setVisible(true);
}
public static void main(String args[])
{
new ScrollBarExample();
}}
```



72. JList and JComboBox

JList

• Displays a list of items.
• Allows single or multiple selections.

Syntax:

String[] items = {“Apple”, “Banana”, “Mango”};
JList<String> list = new JList<>(items);

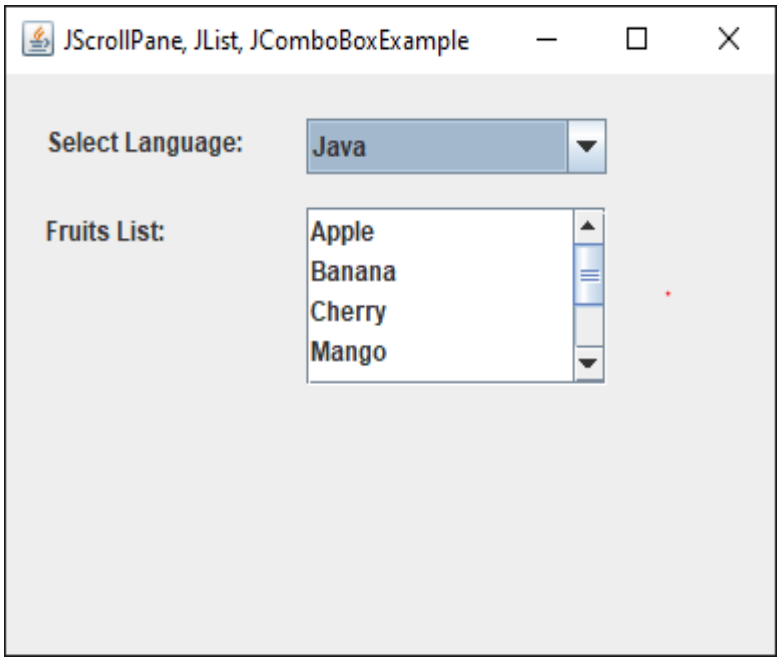
JComboBox

• Also known as a **drop-down menu**.
• Allows the user to choose **one item from a list**.

String[] items = {“Option 1”, “Option 2”};

Example:

```
import javax.swing.*;
public class SwingScrollListComboExample
{
public static void main(String[] args)
{
// Create the frame
JFrame frame = new JFrame("JScrollPane, JList, JComboBoxExample");
frame.setSize(400, 300);
frame.setLayout(null);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
// === JComboBox ===
JLabel comboLabel = new JLabel("Select Language:");
comboLabel.setBounds(20, 20, 150, 20);
frame.add(comboLabel);
String[] languages = { "Java", "Python", "C++", "JavaScript"};
JComboBox<String> comboBox = new JComboBox<>(languages);
comboBox.setBounds(150, 20, 150, 25);
frame.add(comboBox);
JLabel listLabel = new JLabel("Fruits List:");
listLabel.setBounds(20, 60, 150, 20);
frame.add(listLabel);
String[] fruits = { "Apple", "Banana", "Cherry", "Mango","Orange", "Grapes", "Pineapple" };
JList<String> fruitList = new JList<>(fruits);
fruitList.setVisibleRowCount(4); // show only 4 at a time
JScrollPane scrollPane = new JScrollPane(fruitList);
scrollPane.setBounds(150, 60, 150, 80);
frame.add(scrollPane);
// Display frame
frame.setVisible(true);
}
}
```



73. Handling menus

In Java, menus are essentially pull-down lists (similar to combo boxes, known as Choice in Java) that are grouped in one place for convenient user access. To create menus, the java.awt packageprovides four primary classes: MenuBar, Menu, and MenuItem. These classes are not part of the AWT component hierarchy since they do not inherit from java.awt.Component. Instead,they extend java.awt.MenuComponent, which is separate from the Component class lineage.

MenuBar

• The MenuBar class is used to hold and manage menus.

• It is attached to a frame using the setMenuBar() method.

• By default, the menu bar is positioned at the top (north) of the frame and cannot be placed on other sides like the bottom or left.

• **Declaration**:

public class JMenuBar extends JComponent implements MenuElement, Accessible

Menu

- The Menu class represents a drop-down list of menu options and is added to the MenuBar using the add() method.
- Sub-menus can also be nested within a menu.

• **Declaration**:

public class JMenu extends JMenuItem implements MenuElement,Accessible

MenuItem

• MenuItem represents an individual option that a user can choose from within a menu.

• Items are added using the addMenuItem() method.

• You can use the addSeparator() method to insert a separator line, which helps group similar functions such as Cut, Copy, and Paste.

• **Declaration**:

public class JMenuItem extends AbstractButton implements Accessible,MenuElement

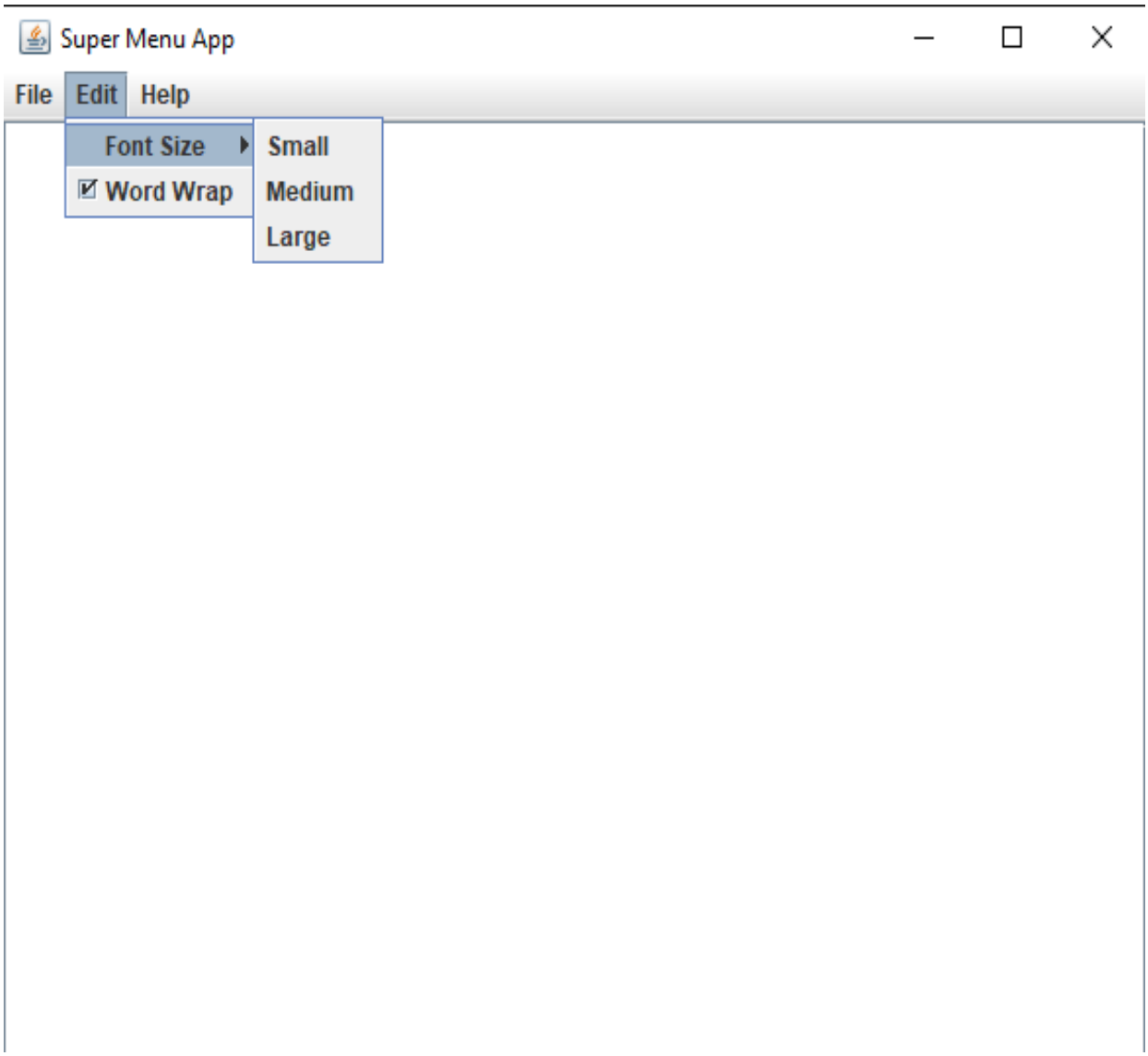
Let me know if you'd like a code example demonstrating how to create a simple menu using these classes!

Example:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
public class SuperMenuApp {
public static void main(String[] args) {
JFrame frame = new JFrame("Super Menu App");
frame.setSize(600, 500);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
JTextArea textArea = new JTextArea();
JScrollPane scrollPane = new JScrollPane(textArea);
frame.add(scrollPane);
JMenuBar menuBar = new JMenuBar();
```

```
// === FILE MENU ===
JMenu fileMenu = new JMenu("File");
JMenuItem openItem = new JMenuItem("Open");
JMenuItem exitItem = new JMenuItem("Exit");
fileMenu.add(openItem);
fileMenu.addSeparator(); // adds a horizontal line
fileMenu.add(exitItem);
// === EDIT MENU with FONT SIZE SUBMENU ===
JMenu editMenu = new JMenu("Edit");
JMenu fontSizeSubMenu = new JMenu("Font Size");
JMenuItem smallSize = new JMenuItem("Small");
JMenuItem mediumSize = new JMenuItem("Medium");
JMenuItem largeSize = new JMenuItem("Large");
fontSizeSubMenu.add(smallSize);
fontSizeSubMenu.add(mediumSize);
fontSizeSubMenu.add(largeSize);
JCheckBoxMenuItem wrapToggle = new JCheckBoxMenuItem("Word Wrap");
editMenu.add(fontSizeSubMenu);
editMenu.add(wrapToggle);
// === HELP MENU with ICON ===
JMenu helpMenu = new JMenu("Help");
ImageIcon icon = new ImageIcon("info.png"); // Put a smallicon image in the same folder
JMenuItem aboutItem = new JMenuItem("About", icon);
helpMenu.add(aboutItem);
// Add menus to menu bar
menuBar.add(fileMenu);
menuBar.add(editMenu);
menuBar.add(helpMenu);
frame.setJMenuBar(menuBar);
// === EVENT HANDLING ===
openItem.addActionListener(e -> {
JFileChooser fileChooser = new JFileChooser();
int option = fileChooser.showOpenDialog(frame);
if (option == JFileChooser.APPROVE_OPTION) {
File file = fileChooser.getSelectedFile();
try {
BufferedReader reader = new BufferedReader(new
FileReader(file));
textArea.read(reader, null);
reader.close();
} catch (IOException ex) {
JOptionPane.showMessageDialog(frame, "Fileerror: " + ex.getMessage());
}
}
```

```
});
exitItem.addActionListener(e -> System.exit(0));
aboutItem.addActionListener(e -> {
JOptionPane.showMessageDialog(frame, "This is a full-featured menu app!");
});
wrapToggle.addActionListener(e -> {
textArea.setLineWrap(wrapToggle.isSelected());
textArea.setWrapStyleWord(wrapToggle.isSelected());
});
smallSize.addActionListener(e -> textArea.setFont(new Font("Arial", Font.PLAIN, 12)));
mediumSize.addActionListener(e -> textArea.setFont(new Font("Arial", Font.PLAIN, 16)));
largeSize.addActionListener(e -> textArea.setFont(new Font("Arial", Font.PLAIN, 20)));
frame.setVisible(true);
}
}
```



74. Networking Basics

Networking refers to the exchange of data between two or more devices, programs, or processes through a network or the internet.

A socket represents one endpoint in a two-way communication link. It plays a crucial role in modern networking because it enables a single machine to manage communication with multiple clients simultaneously and support different types of data exchanges. This is done using ports, which are numbered access points on a device.

Socket communication occurs over specific protocols. The Internet Protocol (IP) is a low-level protocol that handles the routing of data by dividing it into small units called packets and sending them to a specified address. However, IP alone does not guarantee packet delivery.

The Transmission Control Protocol (TCP) operates at a higher level and ensures that data packets are received in the correct order, retransmitting them if necessary.

User Datagram Protocol (UDP), on the other hand, offers a faster, connectionless alternative that does not guarantee delivery, making it suitable for scenarios where speed is more critical than reliability. Once a connection is established, a higher-level protocol operates based on the port number used. TCP/IP designates the first 1,024 port numbers for specific standard services:

Port Number	Service
21	FTP
23	TELNET
25	Email (SMTP)
80	HTTP
119	NetNews

75. Networking Classes and Interfaces

Java enables network communication using the TCP/IP protocol suite by building on its existing stream I/O system and incorporating additional capabilities for handling network-based input and output. It provides built-in support for both TCP (Transmission Control Protocol) and UDP (User Datagram Protocol)

TCP offers a dependable, connection-oriented communication method ideal for continuous data transmission across the network.

UDP, on the other hand, provides a lightweight, faster alternative that relies on a connectionless, datagram-based approach for point-to-point data exchange.

Authenticator	InetAddress	SocketAddress
CacheRequest	InetSocketAddress	SocketImpl
CacheResponse	InterfaceAddress	SocketPermission
ContentHandler	JarURLConnection	StandardSocketOption
CookieHandler	MulticastSocket	URI
CookieManager	NetPermission	URL
DatagramPacket	NetworkInterface	URLClassLoader

DatagramSocket	PasswordAuthentication	URLConnection
DatagramSocketImpl	Proxy	URLDecoder
HttpCookie	ProxySelector	URLEncoder
HttpURLConnection	ResponseCache	URLPermission (Added by JDK 8.)
IDN	SecureCacheResponse	URLStreamHandler
Inet4Address	ServerSocket	
Inet6Address	Socket	

76. InetAddress: TCP/IP Client

The InetAddress class in Java represents both the IP address (such as 142.250.205.68) and the domain name (like www.google.com) of a networked system. Typically, we interact with this class using the domain name rather than the numeric IP, asInetAddress abstracts the IP details internally. It doesn't provide public constructors for object creation.

Instead, factory methods are used to create InetAddress instances. These are special methods that return an object of a class without requiring direct use of the new keyword, simplifying object creation and encapsulating the logic behind it.

Example:

```
import java.net.InetAddress;
import java.net.UnknownHostException;
public class InetAddressExample
{
public static void main(String[] args)
{
try
{
// Creating InetAddress object using a factory method
InetAddress address = InetAddress.getByName("www.google.com");
// Display hostname and IP address
System.out.println("Host Name: " + address.getHost-Name());
System.out.println("IP Address: " + address.getHost-Address());
}
catch (UnknownHostException e)
{
System.out.println("Host not found!");
}
}
}
```

In Java, there are two primary types of TCP sockets: server sockets and client sockets. The ServerSocket class is specifically built to act as a listener—it passively waits for incoming connection requests from clients before taking action. This makes ServerSocket suitable for server-side applications. On the other hand, the Socket class is intended for use by clients. It is used to initiate connections with a server that is listening on a specific port To create client sockets, Java provides the following constructors:
Socket(String hostName, int port)
This constructor establishes a connection to the specified host and port. If the host cannot be resolved, an UnknownHostException is thrown.
Socket(InetAddress ipAddress, int port)
This version allows you to use an existing InetAddress object to connect to a server on the given port. It throws an IOException if a connection cannot be made Methods:

- InetAddress getInetAddress() - Returns the InetAddress associated with the Socket object. It returns null if the socket is not connected.
- int getPort() - Returns the remote port to which the invoking Socket object is connected. It returns 0 if the socket is not connected.
- int getLocalPort()- Returns the local port to which the invoking Socket object is bound. It returns –1 if the socket is not bound

Example:

```
// File: Client.java
import java.io.*;
import java.net.*;
public class Client {
public static void main(String[] args)
 {
try
{
// Connecting to server at localhost, port 5000
Socket socket = new Socket("localhost", 5000);
System.out.println("Connected to server.");
// Output stream to send data to server
PrintWriter out = new PrintWriter(socket.getOutput-Stream(), true);
// Input stream to receive data from server
BufferedReader in = new BufferedReader(new Input-
StreamReader(socket.getInputStream()));
// Sending message to server
out.println("Hello Server, I'm the Client!");
// Reading server response
String serverResponse = in.readLine();
System.out.println("Server says: " + serverResponse);
// Closing socket
socket.close();
}
catch (IOException e)
{
e.printStackTrace();
}
}
}
```

77. Cookies

The java.net package provides several classes and interfaces designed to handle cookies, enabling the creation of stateful HTTP sessions in Java applications. These tools help manage cookie behavior and storage during client-server communication.

Classes

CookieHandler: Acts as the base class for cookie management. It defines methods that control how cookies are handled across HTTP requests.

CookieManager: Implements the CookieHandler and provides a default implementation for storing and retrieving cookies.

HttpCookie: Represents an individual HTTP cookie and provides methods to inspect and manipulate cookie attributes (such as name, value, domain, path, and expiry).

Interfaces

CookiePolicy: Defines the policy that determines whether cookies should be accepted or rejected based on rules (e.g., only from original servers).

CookieStore: Represents a storage for cookies. It allows adding, retrieving, and removing cookies.

78. TCP/IP Server Sockets

Java provides a specialized class called Server Socket for developing server-side network applications. Unlike the standard Socket class used for client-side communication, Server Socketis designed to wait for incoming connections from client programs, whether they are on the same machine or remote systems, using predefined ports. When a ServerSocket is instantiated, it signals the system that it's ready to accept client connections. Unlike regular sockets, it does not initiate a connection; instead, it listens for one. The constructor options for ServerSocket specify the port number to listen on and optionally, the size of the connection request queue.

Here are three commonly used constructors:

ServerSocket(int port) throws IOException: Initializes the server socket to listen on a specific port with a default queue length of 50.

ServerSocket(int port, int maxQueue) throws IOException: Initializes the server socket on a designated port with a specified maximum number of queued connection requests.

ServerSocket(int port, int maxQueue, InetAddress localAddress) throws IOException: This version is useful on systems with multiple network interfaces (multihomed hosts). It binds the server socket to a specific IP address and sets the connection queue size.

Example:

```
// File: Server.java
import java.io.*;
import java.net.*;
public class Server {
public static void main(String[] args) {
try {
// Server is listening on port 5000
ServerSocket serverSocket = new ServerSocket(5000);
System.out.println("Server started. Waiting for client...");
Socket socket = serverSocket.accept(); // Accept client
connection
System.out.println("Client connected.");
// Input stream to receive data from client
```

```
BufferedReader in = new BufferedReader(new Input-
StreamReader(socket.getInputStream()));
// Output stream to send data to client
PrintWriter out = new PrintWriter(socket.getOutput-
Stream(), true);
String clientMessage = in.readLine();
System.out.println("Client says: " + clientMessage);
// Sending response to client
out.println("Hello from Server!");
// Closing connection
socket.close();
serverSocket.close();
}
catch (IOException e)
{
e.printStackTrace();
}
}
}
```

79. Datagrams

While TCP/IP-style networking is commonly used due to its predictable, reliable, and ordered data transmission, it's not always the most efficient. TCP relies on complex congestion control algorithms and assumes a high possibility of packet loss, which adds overhead and reduces performance in some scenarios.

An alternative is datagram-based communication, which sends discrete packets of information— known as datagrams—between machines. Think of it like a blindfolded player throwing a ball to third base: the throw happens whether or not someone's there to catch it. There's no guarantee the data will arrive, arrive intact, or that the sender will still be available to hear back.

In Java, this datagram-style communication is implemented using the UDP protocol. Java provides two primary classes for this:

DatagramPacket: Acts as the container for the data being transmitted.

DatagramSocket: The tool that sends and receives DatagramPacket instances.

Together, they support lightweight, connectionless communication with lower latency—ideal for scenarios where speed is prioritized over reliability

DatagramSocket

The DatagramSocket class in Java offers four public constructors, each providing different ways to create a socket:

• DatagramSocket()Creates a socket bound to any available port on the local machine.

DatagramSocket(int port) Binds the socket to a specific port provided as an argument.

• DatagramSocket(int port, InetAddress ipAddress) Binds the socket to a specific port and IP address.

• DatagramSocket(SocketAddress address)Binds the socket using a SocketAddress object. This abstract class is implemented by InetSocketAddress, which pairs an IP address with a port. Each constructor can throw a SocketException if the socket creation fails due to issues like port unavailability.

Two of the key methods provided by DatagramSocket are:

• void send(DatagramPacket packet) throws IOException Sends the specified datagram packet through the socket.

• void receive(DatagramPacket packet) throws IOException Waits for an incoming datagram and stores it in the provided packet.
These methods are essential for transmitting and receiving data using the User Datagram Protocol (UDP) in Java.

UDP Client side

```
// Java program to illustrate Client side
// Implementation using DatagramSocket
import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.util.Scanner;

public class udpBaseClient_2
{
   public static void main(String args[]) throws IOException
   {
      Scanner sc = new Scanner(System.in);

      // Step 1:Create the socket object for
      // carrying the data.
      DatagramSocket ds = new DatagramSocket();

      InetAddress ip = InetAddress.getLocalHost();
      byte buf[] = null;

      // loop while user not enters "bye"
      while (true)
      {
         String inp = sc.nextLine();

         // convert the String input into the byte array.
         buf = inp.getBytes();

         // Step 2 : Create the datagramPacket for sending
         // the data.
         DatagramPacket DpSend =
              new DatagramPacket(buf, buf.length, ip, 1234);

         // Step 3 : invoke the send call to actually send
         // the data.
         ds.send(DpSend);

         // break the loop if user enters "bye"
         if (inp.equals("bye"))
            break;
      }
   }
}
```

Server Side

```
// Java program to illustrate Server side
// Implementation using DatagramSocket
import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.net.SocketException;

public class udpBaseServer_2
{
    public static void main(String[] args) throws IOException
    {
        // Step 1 : Create a socket to listen at port 1234
        DatagramSocket ds = new DatagramSocket(1234);
        byte[] receive = new byte[65535];

        DatagramPacket DpReceive = null;
        while (true)
        {

            // Step 2 : create a DatgramPacket to receive the data.
            DpReceive = new DatagramPacket(receive, receive.length);

            // Step 3 : revieve the data in byte buffer.
            ds.receive(DpReceive);

            System.out.println("Client:-" + data(receive));

            // Exit the server if the client sends "bye"
            if (data(receive).toString().equals("bye"))
            {
                System.out.println("Client sent bye.....EXITING");
                break;
            }

            // Clear the buffer after every message.
            receive = new byte[65535];
        }
    }

    // A utility method to convert the byte array
    // data into a string representation.
    public static StringBuilder data(byte[] a)
    {
        if (a == null)
            return null;
        StringBuilder ret = new StringBuilder();
        int i = 0;
        while (a[i] != 0)
```

```
            {
                ret.append((char) a[i]);
                i++;
            }
            return ret;
        }
    }
```

OUTPUT